

SO-ARM101 on PYNQ-Z2 — Detailed Working Plan

20-week plan · 21 Sep 2026 – 7 Feb 2027 · 5-week ramp-up + 13-week project + 2-week holiday buffer · Hiwonder SO-ARM101 kit

One capstone, worked end to end the way a senior hardware engineer would own it: requirements, architecture, RTL, a custom PCB, bring-up, characterisation, safety analysis, and a design review you can defend in an interview.

Budget	Rhythm	Gates	Output
~12 h / week	4 × 2 h weeknights + 4 h Saturday	R5 · W4 · W8 · W10 · W13	Full design package

Live page: <https://claude.ai/code/artifact/566fe14c-fa43-47a2-8405-08c87fa4bba5>

Contents

- [How to use this document](#)
- [The capstone](#)
- [Hiwonder kit notes: what differs from the community build](#)
- [Calendar](#)
- [Week 0 · Setup \(runs alongside R1–R2\)](#)

Phase 0 · Ramp-up

- [R1 · Python foundations, pointed at the project](#)
- [R2 · Python for instruments and hardware](#)
- [R3 · Async Python, digital logic, and your first Verilog](#)
- [R4 · Verilog on the PYNQ-Z2: clocks, buttons, and a UART transmitter](#)
- [R5 · UART receiver, the PS–PL bridge, and Gate 0 — Gate 0](#)

Month 1 · Foundations

- [W01 · PYNQ-Z2 bring-up and the Zynq PS/PL split](#)
- [W02 · Both arms running, then characterised](#)
- [W03 · The HX servo protocol and your own driver](#)
- [W04 · Requirements, architecture, first design review — Gate 1](#)

Month 2 · Build

- [W05 · RTL: half-duplex UART and the packet engine](#)
- [W06 · Overlay integration, timing, on-chip debug](#)
- [W07 · Shield schematic in KiCad](#)
- [W08 · Layout, DFM, release, bring-up plan — Gate 2](#)
- [Holiday buffer](#)

Month 3 · Prove

- [W09 · Board bring-up and the rev-B list](#)
- [W10 · Hardware teleoperation without the PS — Gate 3](#)
- [W11 · The characterisation bench](#)
- [W12 · Safety: FMEA, trips, fault injection](#)
- [W13 · Final review, portfolio, interview readiness — Gate 4](#)

Reference

- [Competency matrix](#) · [Kit and tools](#) · [Reading list](#)
- [Interview bank](#) · [Risks](#)
- [Appendix A](#) · [Standing procedures](#) — rail check, power-on, scope hygiene, energised work, anomaly handling
- [Appendix B](#) · [The measurement ledger](#) — all 40 measurements, where taken, what consumes them
- [Appendix C](#) · [Weekly cadence template](#)

How to use this document

Every week — ramp-up weeks R1–R5 and project weeks W1–W13 alike — is broken into five sessions that match the rhythm above: **S1–S4** are the 2-hour weeknight blocks, **S5** is the 4-hour Saturday block. Four weeks (W5, W7, W8, W10) are budgeted at 14 h; those add a sixth short session or extend Saturday, and say so.

Each session gives you:

- **Numbered steps** in the order to do them. A step is sized so that stopping between steps is safe — you never leave the bench with a servo energised mid-procedure.
- **Exact commands and code** where the command is the point. Where a value depends on your hardware (pin names, measured currents), the step tells you where to get it rather than inventing a number.
- **Done when** — the observable condition that closes the session. If it is not true, the session is not finished; roll the remainder into S5 rather than starting the next session.

Each week closes with a **measurement table** (what to record and where), a **deliverables checklist**, and a **failure-mode table** of the things that actually go wrong, so you recognise them in ten minutes instead of two hours.

Three conventions used throughout

1. **Nothing is "done" until it is written down.** Every measurement lands in the repo the same day, in the file named by the week's Ship list. A number in a notebook you will "write up later" does not exist.
2. **Rail check before every power-on.** Both arms run on 12 V, and every servo in the kit is rated 9–12.6 V — so the hazard is not a mixed-up rail but **over-voltage**. A bench supply left at 13.8 V from another job, a 15 V laptop brick, or a 24 V adapter with the same barrel plug will take all twelve servos past their maximum at once. Say it out loud: *twelve volts, and the display agrees*. This check appears as an explicit step in every procedure that energises an arm, and it stays there even when it feels tedious.
3. **Current limit before voltage.** Every bench supply is set to its limit *first*, with the output off, then the voltage, then the output on. The limits are given per procedure.

Version drift. This plan is written against PYNQ v3.1.1, Vivado/Vitis 2026.1, KiCad 8, and the LeRobot and OpenCV APIs current at time of writing. Tool CLIs move — LeRobot in particular has renamed its entry points more than once, and OpenCV moved ArUco pose estimation out of `estimatePoseSingleMarkers`. Where a step names a command, check `--help` once at the start of the week and record the actual invocation in your notes. Treat any mismatch as a finding worth a line in `docs/decisions/`, not an obstacle.

The capstone: an FPGA-native servo controller with hardware teleoperation

The Hiwonder SO-ARM101 ships driven by BusLinker USB-to-serial boards and a Python loop. The PYNQ-Z2 replaces that path with hardware you design yourself. By week 13 the pair of arms is driven by:

- **A two-bus servo bus-master IP** in the Zynq-7020 programmable logic (PL), one bus for the leader and one for the follower, talking Hiwonder's HX magnetic-encoder bus-servo protocol (half-duplex TTL, 1 Mbps) with an AXI-Lite register interface.
- **A custom shield PCB** on the Arduino header: two independently switched 12 V rails (leader and follower) behind an active over-voltage cutoff, two bus drivers, current sense on each rail, a hardware e-stop and watchdog-held load switches.

- A **hardware teleoperation loop** that reads the leader and writes the follower entirely in PL, with an HLS trajectory filter, so the Linux side can be slow and jittery without the arms noticing. End-to-end latency is measured against the USB path.
- An **automated characterisation bench** built on the same SCPI/VISA instrument stack you already use for RF ATE, with the spare OBSBOT camera as an independent optical measurement of the follower, producing a per-joint datasheet.
- An **FMEA and fault-injection results** with measured reaction times.
- A **LeRobot dataset and policy** recorded with the kit's wrist and external cameras, trained on a PC, and run against your overlay as the actuator backend.

Why this project maps to a senior role: every week produces a document or artifact that a reviewer could pick apart. Senior engineers are judged on requirements ownership, trade-off reasoning, verification depth, bring-up discipline and the ability to run a review, not on whether the arm moves.

Assumptions

- Hardware in hand: the **Hiwonder SO-ARM101 kit, assembled** (Standard or Advanced) — a follower arm with six **HX-30HM** servos, a leader arm with six **HX-10HM** servos, BusLinker V3.0 debugging board(s), the 12 V 5 A adapter, the 480p wrist camera and the 1080p external camera. Plus a spare OBSBOT camera and a PYNQ-Z2.
- If you have the Starter or DIY variant, which ship without cameras, use the OBSBOT as the scene camera and borrow any UVC webcam for the W11 optical instrument.
- Toolchain: PYNQ v3.1.1 image, which was built on the 2024.1 toolchain, with the current Vivado and Vitis release (2026.1 at time of writing). Custom overlays build cleanly on a newer Vivado than the image; only a rebuild of the base overlay itself needs its IP cores upgraded.
- You are **new to both Python and Verilog**, and comfortable with SCPI instruments and lab measurement from RF test work. Phase 0 (R1–R5) takes you from zero to the level W1 assumes, and ends with Gate 0 — an honest check before the project clock starts. KiCad may also be new; R5 includes an optional hour on it.
- **Primary host: a dedicated Linux laptop** (Ubuntu LTS) for everything — Vivado, simulation, Python, LeRobot and instruments. A **Windows desktop** serves three narrower roles: ServoStudio, which is Windows-only (W2, W12); GPU training in W13 if that is where the NVIDIA card is; and a fallback host if the laptop fails.
- **Vivado Basic (free) tier, 2026.1.** It supports Linux, but its license is node-locked and renewed annually, its simulator is limited (the plan uses Icarus/Verilator with cocotb instead), and its ILA is **instantiation-only: one ILA core, at most five probes**, with no System ILA and no `mark_debug` insertion flow. W6 and W10 are written to fit inside that.

Senior-level thread: each week names the competency being exercised. By the end you should be able to tell a concrete story for every row in the competency matrix.

Hiwonder kit notes: what differs from the community build

Most SO-ARM101 material online — including the LeRobot docs and most tutorials — assumes the community build: Feetech STS3215 servos, a 7.4 V leader powered at 5 V, and Waveshare-style adapters. Your kit is a different electrical system, and the plan is written for what you actually have.

Topic	Community build	Your Hiwonder kit	What it changes in this plan
Follower servos	Feetech STS3215, 12 V	Hiwonder HX-30HM , magnetic encoder, rated 9–12.6 V, 3 A stall, 100 mA no-load	Power budget uses HX numbers; W2 compares measured against these
Leader servos	Feetech STS3215, 7.4 V at 5 V	Hiwonder HX-10HM , magnetic encoder, also high-voltage	Both arms on 12 V. No 5 V leader rail, no buck on the shield
Main electrical hazard	12 V reaching the 7.4 V leader	Over-voltage above the 12.6 V maximum, on all twelve servos at once	Shield gains an active over-voltage cutoff; rail check becomes a voltage check
Protocol	Feetech STS/SCS	Hiwonder magnetic-encoder bus-servo protocol, 1 Mbps default, IDs 1–253, Write / SyncWrite / RegWrite	W3 works from Hiwonder's protocol document ; no Feetech register address is assumed
Position	0–4095 per turn	0–4095 = 0–360°, plus multi-turn to about ± 7.5 turns; turn count is cleared on power loss	Positions are signed; wrist roll needs care at the 0/4095 seam
Servo-side motion shaping	Speed/acceleration registers	Built-in trapezoidal acceleration and PID, with Accel / Speed parameters	W10 must show the HLS filter earns its place; W11 records profile settings
Built-in protection	Basic	Configurable current limit with hold time, overload, temperature, under- and over-voltage	W12 uses these as a defence-in-depth layer and verifies them
USB adapter	Waveshare, FTDI or CH340	BusLinker V3.0 , CH341, with a TTL header for an external controller	W3 baseline is "BusLinker + CH341 + Python"; the TTL header is a W6 fallback
Vendor tool	Feetech's debug utility	ServoStudio (scan, config, monitor, firmware) — Windows only	W2 uses it on the Windows desktop for ID, config and firmware inventory
Servo connector	JST-style 3-pin	5264-3P (the BusLinker also has PH2.0 and 1.25T ports)	W0 calipers it; W7 footprints it
Cameras	Your own	480p wrist, 1080p external	W4 camera budget covers the 1080p stream

Two cautions that follow from this. First, never take a register address from a Feetech datasheet or an STS3215 forum post and apply it to an HX servo — the instruction names match, which makes it tempting, but the memory table is Hiwonder's to define. Second, find out in W2 how Hiwonder's LeRobot integration talks to these servos (upstream LeRobot motor-bus code, or a Hiwonder-supplied fork or plugin). The answer tells you how close to Feetech the protocol really is, and it is the first fact W3's spec records.

Calendar

Weeks run Monday to Sunday. The fab release at the end of W8 lands right before the holiday buffer, so the boards are made while you are away from the bench.

Week	Dates	Focus	Gate
R1	21 – 27 Sep 2026	Python foundations (W0 ordering and installs alongside)	
R2	28 Sep – 4 Oct	Python for instruments and hardware	
R3	5 – 11 Oct	Async Python, digital logic, first Verilog	
R4	12 – 18 Oct	Verilog on the PYNQ-Z2, UART transmitter	
R5	19 – 25 Oct	UART receiver, AXI-Lite, readiness check	Gate 0
W1	26 Oct – 1 Nov	PYNQ-Z2 bring-up, PS/PL split	
W2	2 – 8 Nov	Both arms running and characterised	
W3	9 – 15 Nov	HX protocol and your own driver	
W4	16 – 22 Nov	Requirements, architecture, first review	Gate 1
W5	23 – 29 Nov	RTL and verification	
W6	30 Nov – 6 Dec	Overlay integration, timing, ILA	
W7	7 – 13 Dec	Shield schematic	
W8	14 – 20 Dec	Layout, release to fab by Sat 19 Dec	Gate 2
—	21 Dec – 3 Jan 2027	Holiday buffer; boards in fab	
W9	4 – 10 Jan	Board bring-up, rev-B list	
W10	11 – 17 Jan	Hardware teleoperation in PL	Gate 3
W11	18 – 24 Jan	Characterisation bench	
W12	25 – 31 Jan	FMEA and fault injection	
W13	1 – 7 Feb	Final review and portfolio	Gate 4

R1 starts the week this plan was revised, so it begins immediately. If you would rather start R1 next Monday, shift everything by one week and take it from the holiday buffer — the fab release then falls on Boxing Day, which still works but leaves one buffer week instead of two.

Week 0: Setup (runs alongside R1–R2)

Budget 6–8 h, spread across R1 and R2 on top of their sessions. None of it is intellectually hard and all of it is on the critical path — a missing microSD card costs you W1 entirely. **Do S1 and S2 in the first two days of R1** so deliveries arrive during the ramp-up; do S3's Ubuntu and Python install before R1 S1 — ideally over the first weekend — and start the long Vivado install early in R2 so it is ready for R3.

S1 · Inventory, labelling and the over-voltage hazard (1 h)

1. Lay out everything on the bench and photograph it. That photo is the first image in `docs/arm-bringup.md`.
2. Set the bench supply to **12.0 V, limit 5 A**, output off. If you have two supplies, set both identically — one per arm makes W2's current measurements cleaner, but either arrangement works.
3. Label every supply and adapter that could reach the arms: **12.0 V – HX SERVOS MAX 12.6 V**. Check the rating on anything with a matching barrel plug in your drawer; if an adapter above 12.6 V fits the same jack, label it **NOT FOR ARMS** or move it out of the room.

- Label the arm cables `LEADER` and `FOLLOWER` as flags. This is no longer about voltage — both are 12 V — but about bus identity: from W6 onwards the PL drives two buses, and commanding the leader as if it were the follower moves an arm someone's hand is on.
- Identify the servos by reading one label on each arm: **HX-30HM** on the follower, **HX-10HM** on the leader. They look alike, and a follower joint fitted with an HX-10HM has about a third of the torque. Photograph both labels.
- Count six servos per arm. Locate or order one spare **HX-30HM** and one **HX-10HM** and bag them, labelled.
- Check the cable connector on an actual servo cable. Hiwonder specifies **5264-3P**; caliper the pitch and confirm the pin order (GND / VIN / SIG) against the servo documentation. The BusLinker V3.0 also has PH2.0 and 1.25T ports — note which ones your kit's cables use. W7 footprints this connector, and a wrong footprint is the most common "the board arrived and nothing plugs in" failure.
- Record the kit adapter: Hiwonder lists **12 V 5 A with a 4.0 × 1.7 mm plug**, while the BusLinker V3.0 manual shows a 5.5 × 2.1 mm DC jack and a screw terminal. Check how your kit actually distributes power — there may be an adapter cable or a different board — and write it down with a photo.
- Count your BusLinker boards. The W3 USB baseline needs one per arm running simultaneously; if the kit came with one, add a second to the S2 order.

Done when: every supply that could reach the arms is labelled, the connector is calipered, and the kit's power distribution is photographed in `docs/`.

S2 · Ordering (1 h)

Order everything in one go; the long pole is shipping, not money.

Item	Qty	Why	Watch for
microSD, 32 GB, A1 class	1	PYNQ image	A1 rating matters for boot time
Ethernet cable	1	Board is headless over Ethernet	—
USB supply, 2 A, barrel-to-PYNQ	1	Board power	Board draws more than a PC port likes
Powered USB hub	1	Two cameras, one host port	Must be <i>powered</i> , not bus-powered
SN74LVC1T45 or 74LVC1G125 breakout	4	Two per bus, interim drivers	Buy spares; they die on miswiring
INA226 breakout	2	One per rail	Check the shunt value fitted
Mushroom e-stop, latching NC	1	Safety	Normally-closed contact, latching
XT30 pairs / barrel jacks	4	12 V input and rail connectors	Rated above the W4 current
Breadboard + jumpers	1 set	R4–R5 and W6 interim bus	—
USB-UART adapter, 3.3 V logic (CP2102, CH340 or FTDI)	1	R2 loopback, R4–R5 UART exercises	Must be 3.3 V — PYNQ PL pins are not 5 V tolerant
Spare HX-30HM and HX-10HM	1 each	Stall and over-voltage testing	Match the kit's servo models exactly
Second BusLinker V3.0	0–1	One per arm for the W3 baseline	Only if the kit shipped with one
5264-3P housings, crimps, pre-crimped leads	1 set	W6 breadboard and W7 prototype cables	Match the calipered pitch

- Place the order. Record the order date and expected arrival in `docs/risks.md` — this is the first entry in your risk register, before the register exists.

2. While the cart is open, bookmark the LCSC/JLPCB part pages for the INA226 and the buffer; W7 reuses them.

Done when: order placed, arrival dates logged.

S3 · Laptop and toolchain install (5–6 h, partly unattended)

This is longer than a Windows install would be, and it is time well spent: every problem you hit here is one you would otherwise hit in the middle of R4 or W1. Keep a running log in `docs/toolchain.md` as you go — the commands you ran and the errors you fixed. It becomes your rebuild guide if the laptop ever dies.

1. **Check the laptop.** 16 GB of RAM is workable for this design and 32 GB is comfortable; at least 200 GB of free SSD for the OS plus Vivado; an Ethernet port or a USB-Ethernet adapter for the PYNQ-Z2; and count your USB ports — the BusLinkers, USB-UART adapter, logic analyser, JTAG cable and cameras will outnumber them, so plan on a powered hub on the laptop too.
2. **Install Ubuntu LTS.** Before choosing the version, open AMD's *Vivado Release Notes, Installation and Licensing* (UG973) for 2026.1 and check its supported-OS table; pick the newest Ubuntu LTS listed there. A clean, full-disk install is simplest since the laptop is dedicated. Take system updates at weekends, never the night before a measurement session — a kernel update that breaks a USB driver mid-week costs a session.
3. **Learn the terminal basics in the first hour** if Linux is new: `cd`, `ls`, `pwd`, `mkdir`, `cp`, `mv`, `cat`, `less`, `grep`, tab completion, `sudo`, and `apt`. Everything in this plan happens in a terminal.
4. **Base packages:**

```
sudo apt update
sudo apt install build-essential git make python3-venv python3-dev \
    iverilog gtkwave verilator libusb-1.0-0 pulseview
```

Check `verilator --version` against what current cocotb requires. Ubuntu's packaged Verilator can lag well behind; if it is too old, build it from source or stay on Icarus. Record the decision. 5. **Serial port permissions, and the one Ubuntu trap.**

```
sudo usermod -aG dialout $USER      # log out and back in afterwards
sudo apt remove brlTTY              # see below
```

The BusLinker uses a CH341 USB-serial chip, and its driver is built into the Linux kernel — nothing to install. But Ubuntu's **braille-display service brlTTY claims some CH340/CH341 devices on sight**, and the port appears for a second and then vanishes. Removing `brlTTY` fixes it (unless you use a braille display). Plug in a BusLinker, run `dmesg | tail`, and confirm you see a `ch341` device attached to `/dev/ttyUSB0`. Then look in `/dev/serial/by-id/` — those names stay stable across reboots and unplugging, unlike `ttyUSB0` and `ttyUSB1`, and they are what your scripts should use. 6. **Vivado + Vitis 2026.1, Basic tier.**

- Create or log in to your AMD account and generate the free **Basic** license. It is **node-locked** to this laptop's host ID and **expires after a year**; put the renewal date in `docs/risks.md` now. The Windows desktop needs its own license if you ever fall back to it.
- Run the Linux installer, selecting Vivado, the Zynq-7000 device family and Vitis HLS, into `/tools/Xilinx`. Then run the `installLibs.sh` script the installer provides to pull in the system libraries Vivado needs.
- Follow Hackster.io's *AMD FPGA Tools 2026.1 Install on Ubuntu* walkthrough for the Ubuntu-specific fixes, notably the license-manager library link that newer Ubuntu releases need.
- **Install the JTAG cable drivers** with the `install_drivers` script under the Vivado installation's `data/xicom/cable_drivers/lin64/` folder, run with `sudo`, then unplug and replug the board. Without this, Hardware Manager never sees the PYNQ-Z2, and R4 stalls on day one.
- Add `source /tools/Xilinx/2026.1/Vivado/settings64.sh` (adjust to your install path) to `~/.bashrc` so `vivado` works from any terminal.
- If Vivado reports no valid license, the usual cause on a laptop is the license manager not finding the network interface the host ID was generated from. Check the host ID Vivado's License Manager reports against the one on your AMD license page before anything else.

7. **PYNQ v3.1.1 image.** Download, verify the checksum with `sha256sum`, and write to the microSD with `dd` or balenaEtcher. Do not skip the checksum: a truncated image boots far enough to waste an evening.
8. **KiCad 8 or newer**, from KiCad's official Ubuntu PPA or Flatpak rather than Ubuntu's own package, which is usually a version behind. Add the JLCPCB/LCSC library plugin via the Plugin and Content Manager.
9. **Python environment** on the laptop:

```
cd ~/arm-on-zynq
python3 -m venv .venv
source .venv/bin/activate
pip install lerobot pyserial cocotb pytest pyvisa pyvisa-py pyusb opencv-python numpy matplotlib
pip freeze > requirements.txt
```

Check Hiwonder's LeRobot tutorial for the Python version it expects. If it differs from Ubuntu's, create the environment with `uv` or `pyenv` at the required version instead of the system `python3`. **Instruments on Linux:** NI-VISA's Linux support is narrow, so use the pure-Python **pyvisa-py** backend (`pyvisa.ResourceManager("@py")`). LAN-connected instruments work immediately. USB-TMC instruments need a small udev rule granting your user access to the device; the `pyvisa-py` documentation shows it.

10. **Hiwonder tooling, on the Windows desktop.** ServoStudio is a Windows program. Install it and the CH341 serial driver on the desktop from the BusLinker V3.0 download folder. On the laptop, download into `docs/vendor/` the **Hiwonder Magnetic Encoder Bus Servo Communication Protocol** document, the HX-30HM and HX-10HM product documents, and Hiwonder's SO-ARM101 LeRobot tutorial, recording each file's date or version.

11. **Simulator decision.** Icarus Verilog (`iverilog -g2012`) or Verilator, both now installed natively. Verilator is the better choice if you expect to use `always_ff`, packed structs or interfaces; Icarus is lighter and enough for the plan as written. Record it as your first decision record.

12. Verify each install with one command and paste the output into `docs/toolchain.md`: `lsb_release -a`, `uname -r`, `vivado -version`, `kicad-cli version`, `iverilog -V`, `verilator --version`, `python -c "import cocotb, pyvisa, cv2; print('ok')"`, and on the desktop, ServoStudio's version from its Settings page.

Done when: `docs/toolchain.md` lists a version string for every tool, a BusLinker shows up under `/dev/serial/by-id/`, and Vivado opens without a license warning.

S4 · Repository skeleton (1 h)

1. Create `arm-on-zynq` and the folder structure:

```
git init arm-on-zynq && cd arm-on-zynq
mkdir -p docs/decisions hw/shield rtl overlay sw test/cocotb test/bench
printf '*.jou\n*.log\n*.Xil\n*.bit\n*.hwh\n*.venv\n*__pycache__/\n' > .gitignore
```

2. Reconsider `*.bit` and `*.hwh` in `.gitignore` — you *do* want released bitstreams tracked, so instead ignore build scratch and commit tagged overlay outputs deliberately under `overlay/vx.Y/`. Write this as a one-line note in the README; it is exactly the kind of small ownership decision reviewers notice.
3. Seed `docs/decisions/0001-simulator-choice.md` using this template, which every later decision record reuses:

```
# 0001 · Simulator choice
Date: 2026-09-xx   Status: accepted
## Context
## Options considered
## Decision
## Consequences
## How this could be revisited
```

4. Seed `docs/risks.md` with the PCB lead-time risk and the supply-hazard risk.
5. Commit and push.

Done when: a clean `git log` with one commit, and the decision-record template exists.

W0 checklist

- ☒ Every supply that fits the arms labelled; nothing above 12.6 V within reach
- ☒ Connector confirmed as 5264-3P from a real cable
- ☒ Kit power distribution photographed; BusLinker count known
- ☒ Hiwonder protocol document and tutorials saved with dates
- ☒ Order placed, arrival logged in the risk register
- ☒ Every tool version captured from the tool
- ☒ Laptop: in the `diayout` group, `br1tty` removed, Vivado licensed with its renewal date logged, JTAG cable drivers installed
- ☒ ServoStudio installed on the Windows desktop
- ☒ Repo skeleton committed, decision template in place

Phase 0 · Ramp-up: Python and Verilog from zero to project-ready (5 weeks)

Why this phase exists. The main plan assumes you can already write a Python class with tests and a Verilog state machine with a testbench. Without those, W3, W5, W6, W10 and W11 each run to two or three times their budget, and the slips compound. These five weeks close that gap — and nothing in them is throwaway. Every exercise is a smaller version of something the project needs, and the UART you build in R4 and R5 becomes the core of W5.

How the phase is shaped. Python comes first because the Verilog testbenches (cocotb) are themselves written in Python. The two tracks overlap in R3, and R4–R5 put your own logic on the PYNQ-Z2. Each week ends with something working on real hardware or a real instrument, because that is what keeps a ramp-up from feeling like homework.

Three rules for the phase.

1. **Type, don't paste.** Every listing in this phase is short enough to type. Typing is how the syntax lands; pasting is how you arrive at W5 unable to write a `for` loop without a reference.
2. **Predict before you run.** Before every script or simulation, write down in one line what you expect to see. When the result differs, the difference is the lesson — work out why before changing anything.
3. **Everything goes in the repo.** Create `ramp/` in `arm-on-zynq` with one folder per week. It is your evidence at Gate 0, and it becomes a useful appendix in W13's portfolio: "I learned this from scratch, here is the trail."

Week 0 runs alongside R1 and R2. Do W0 S1 and S2 (inventory and ordering) in the first days of R1, because deliveries take time. Do W0 S3's Ubuntu and Python setup before R1 S1, and start the Vivado install early in R2 so it is ready for R3.

R1 · Python foundations, pointed at the project (12 h)

Goal: write and run scripts that handle bytes, files and plots — the three things W2 and W3 lean on hardest. **You will have by Sunday:** a script that decodes and checksums servo-style packets from a file and plots a histogram.

S1 · Environment, and your first twenty lines (2 h)

1. Create the project virtual environment exactly as W0 S3 describes, and activate it with `source .venv/bin/activate` — in every new terminal, every time. Install VS Code (the official `.deb` from Microsoft, or the Snap) with the Python extension, and point it at `.venv`; it gives you inline errors, which halves the time you spend confused.
2. In `ramp/r1/`, create `hello.py` and run it from the terminal, not the editor's run button — W2 onwards is all terminal.
3. Work through the core types by writing tiny experiments in a file, running each, and predicting the output first: integers, floats, strings, lists, dictionaries, `if / elif / else`, `for` and `while` loops, and functions with arguments and return values.
4. Use the official Python tutorial (docs.python.org, sections 3–5) as your reference. Read one short section, then close it and write something that uses it.

- Set up git properly: `git config` your name and email, commit `ramp/r1/`, and push. Commit at the end of every session from now on.

Done when: you can write a function that takes a list of numbers and returns the minimum, maximum and mean — without looking anything up.

S2 · Bytes, hex and bits — the language of serial protocols (2 h)

This session matters more than any other in R1. W3 is entirely bytes.

- Experiment with `bytes`, `bytearray`, hex literals and conversions until each line's output stops surprising you:

```
pkt = bytes.fromhex("ff ff 01 04 02 38 02 be")
print(len(pkt), pkt[0], hex(pkt[2]), pkt[2:-1].hex(" "))

body = pkt[2:-1]                # everything between header and checksum
s = sum(body)
print(hex(s), hex(~s & 0xFF))    # ~s is negative in Python; & 0xFF keeps the low byte

value = 0x0834
lo, hi = value & 0xFF, value >> 8
print(hex(lo), hex(hi), (hi << 8) | lo)
print(int.from_bytes(b"\x34\x08", "little"), int.from_bytes(b"\x34\x08", "big"))
```

- Work out **by hand**, on paper, why `~s & 0xFF` gives the checksum byte. This is the checksum rule from W3; understanding it now means W3 S1 takes twenty minutes instead of two hours.
- Two's complement.** HX servo positions are signed. Convert by hand: what is `0xFFFFE` as a signed 16-bit number? Then check:

```
raw = b"\xfe\xff"
print(int.from_bytes(raw, "little", signed=False), int.from_bytes(raw, "little", signed=True))
print((-2).to_bytes(2, "little", signed=True).hex())
```

- Write `checksum(body: bytes) -> int` and `is_valid(packet: bytes) -> bool` in `ramp/r1/packets.py`.

Done when: you can explain, without notes, why `0xFFFFE` is 65534 unsigned and `-2` signed, and your `is_valid` accepts the packet above.

S3 · Files and data (2 h)

- Create `ramp/r1/packets.txt` with ten hex packets, one per line — some valid, some with a deliberately wrong checksum. Build them with your S2 function so you know which is which.
- Write a script that reads the file, validates each line, and prints a summary: how many valid, how many invalid, which line numbers failed.
- Add `try / except` around the parsing so a malformed line (odd number of hex digits, a stray letter) is reported rather than crashing the script. This is the same shape as W3's driver: expect bad data, handle it explicitly.
- Write the results to a CSV with the `csv` module: line number, packet hex, valid yes/no. Open it in Excel to check it looks right. Your W11 bench will write hundreds of these.

Done when: the script handles a malformed line gracefully and writes a correct CSV.

S4 · numpy and matplotlib (2 h)

- Generate fake latency data — `numpy.random.normal(300, 20, 1000)` plus a few large outliers you add by hand — and save it to CSV.
- Load it back, then compute median, 99th percentile and maximum with `np.median`, `np.percentile` and `.max()`. Predict each before printing.

3. Plot a histogram with matplotlib, with a title, axis labels and units, and save it as a PNG. This is the exact figure W3 S4 asks for — you are rehearsing it with fake data.
4. Add a second, differently-shaped dataset and overlay the two histograms with a legend. That overlaid figure is the one W6 calls "the best single image in your design package".

Done when: you have a labelled, overlaid two-histogram PNG in the repo.

S5 · Saturday: a small tool, start to finish (4 h)

Pull the week together into one program you would actually use.

1. Write `ramp/r1/pktview.py` , run as `python pktview.py packets.txt` , using `sys.argv` or `argparse` for the filename. It should validate every packet, print a table of the valid ones broken into fields (header, ID, length, instruction, parameters, checksum), write a CSV, and plot a histogram of packet lengths.
2. Split it into functions of a few lines each, with a `main()` and the `if __name__ == "__main__":` guard. Structure is what makes the next step possible.
3. Refactor once: look for any repeated code and turn it into a function.
4. Write a short `README.md` for the tool: what it does, how to run it, an example output.
5. Spend the last half-hour on the W0 checklist — make sure the order has gone in.

Done when: a friend could clone the repo, run `pktview.py` on your sample file, and get the table, CSV and plot.

Failure modes

Symptom	Likely cause	Fix
<code>ModuleNotFoundError</code> for numpy	Virtual environment not activated	Activate <code>.venv</code> in every new terminal
Checksum comes out negative	Forgot <code>& 0xFF</code> after <code>~</code>	Python integers are unbounded; mask to a byte
<code>ValueError: non-hexadecimal number</code>	Stray character or odd digit count	That is the malformed-line case — catch it
Plot window never appears	Running headless or blocked by a script end	Save to PNG with <code>plt.savefig</code> ; do not rely on <code>show()</code>

R2 · Python for instruments and hardware (12 h)

Goal: classes, exceptions, tests, serial ports and your bench supply — the toolkit W2, W3, W6 and W11 are built from. **You will have by Sunday:** a tested `Psu` class driving your real bench supply, and a latency histogram from a real serial loopback. **Hardware:** a 3.3 V USB-UART adapter (added to the W0 order) with its TX and RX pins joined by a jumper.

S1 · Classes and exceptions (2 h)

1. Read the classes section of the Python tutorial, then build a class that models something simple you already understand — a bank account, a temperature log — with `__init__` , a few methods and some internal state.
2. Define your own exceptions by subclassing `Exception` , raise them for invalid operations, and catch them. The W3 driver defines `BusTimeout` , `ChecksumError` and `StatusError` in exactly this way.
3. Rewrite R1's packet functions as a `Packet` class: construct from bytes, expose `id` , `instr` , `params` and `checksum` attributes, and raise a custom `ChecksumError` from the constructor if the packet is invalid.
4. Move the class into its own module, `ramp/r2/packet.py` , and import it from a separate script. Imports and modules are how the project's code is organised.

Done when: `Packet(bytes.fromhex(...))` works for a valid packet and raises your own exception for an invalid one.

S2 · Testing with pytest (2 h)

1. Install `pytest` (it is already in the venv from W0) and write `ramp/r2/test_packet.py` with at least six tests: a valid packet parses, each field is correct, a bad checksum raises, a truncated packet raises, an empty input raises, and a round trip (`build` → `bytes` → `parse`) preserves everything.
2. Run `pytest -v`. Break your code on purpose and watch a test fail; restore it and watch it pass. A test you have never seen fail proves nothing.
3. Use `pytest.raises` for the error cases, and `@pytest.mark.parametrize` to run one test over several packets.
4. Learn fixtures by writing one that provides a known-good packet to several tests. Fixtures are the backbone of W11's bench, so meet them here in a harmless setting.

Done when: six or more tests pass, and you have watched each one fail at least once.

S3 · Your bench supply from Python (2 h)

You already know SCPI; this session puts it inside a Python class.

1. Find the supply's VISA address with `pyvisa.ResourceManager("@py").list_resources()` and send `*IDN?`. A LAN-connected supply shows up as a `TCP::` address with no further setup; a USB one needs the udev rule from W0 S3 first.
2. Write `ramp/r2/psu.py`:

```
import pyvisa

class Psu:
    def __init__(self, address):
        self.inst = pyvisa.ResourceManager("@py").open_resource(address)  # pyvisa-py backend on Linux

    def idn(self):
        return self.inst.query("*IDN?").strip()

    def configure(self, volts, amps):
        self.inst.write("OUTP OFF")
        self.inst.write(f"CURR {amps}")  # limit before voltage – always
        self.inst.write(f"VOLT {volts}")

    def on(self): self.inst.write("OUTP ON")
    def off(self): self.inst.write("OUTP OFF")

    def measure(self):
        return float(self.inst.query("MEAS:VOLT?")), float(self.inst.query("MEAS:CURR?"))

    def __enter__(self): return self
    def __exit__(self, *exc): self.off()  # output goes off even if the script crashes
```

Adjust the SCPI commands to your supply's manual. 3. Use it only with a **resistor load or nothing connected** — never the arms in this phase. Configure 5 V at 0.1 A, switch on, measure, switch off. 4. Test the `with` block: put a deliberate error inside `with Psu(addr) as psu:` and confirm the output switches off anyway. That context-manager behaviour is the whole of W11's "teardown always de-energises" rule, in one line of Python.

Done when: a crashing script provably leaves the supply output off.

S4 · Serial ports and timing (2 h)

1. Plug in the 3.3 V USB-UART adapter with TX jumpered to RX. Find its stable name under `/dev/serial/by-id/` (or check `dmesg | tail`), then open it with `pyserial` at 115200 baud, write some bytes, and read them back. A `Permission denied` error means you are not yet in the `dialout` group — or have not logged out and back in since joining it.

2. Measure the round-trip time of one byte with `time.perf_counter()` , 1000 times, and plot the histogram using your R1 code. This is a rehearsal for W3 S4 with real hardware underneath.
3. Repeat at 1,000,000 baud, the servo bus rate, if the adapter supports it. Compare the two histograms and explain the difference in the medians — most of it is not the wire. That sentence is the W3 insight, arriving three weeks early.
4. Set a read timeout, remove the jumper, and confirm that `read()` returns short rather than hanging. Handling "no reply" is half of W3's driver.

Done when: you have two overlaid loopback histograms and a written explanation of why the medians differ by less than the bit rates suggest.

S5 · Saturday: an instrument-backed test (4 h)

1. Write a pytest suite in `ramp/r2/test_bench.py` that uses your `Psu` class through a **session fixture**: the fixture configures the supply, yields it, and switches it off in teardown.
2. Tests, all with a resistor load: output voltage within 1 % of setpoint at three settings; current limit respected (choose a resistor so the limit is reached); output reads zero after `off()` .
3. Log every measurement to a timestamped CSV in `ramp/r2/data/` , with the supply's `*IDN?` string in the header. This is W11's pattern at toy scale — DUT identity, timestamped data, teardown safety.
4. Write a **fake** `Psu` class with the same methods but no hardware, and run the same tests against it with a command-line option. Being able to test logic without the instrument attached is the trick W3 and W6 depend on.
5. Commit, then read through your week's code once and write three notes on what you would now do differently.

Done when: `pytest` passes against both the real and the fake supply, and the real run leaves a CSV with identity in its header.

Failure modes

Symptom	Likely cause	Fix
<code>VisaIOError</code> timeout	Wrong address, or a command the supply does not know	Check <code>list_resources()</code> ; check the manual's command syntax
USB supply not listed at all	No udev permission for the USB-TMC device	Add the udev rule from the pyvisa-py docs, replug; or use the supply's LAN port
Permission denied on <code>/dev/ttyUSB0</code>	Not in the <code>dialout</code> group	<code>sudo usermod -aG dialout \$USER</code> , then log out and in
Serial port appears, then disappears	<code>brl1tty</code> claimed the device	<code>sudo apt remove brl1tty</code>
Loopback reads nothing	Jumper missing, or wrong port	Check the port name; confirm TX and RX are joined
First read contains old bytes	Input buffer not flushed	<code>reset_input_buffer()</code> before each transaction
Tests pass without hardware attached	Fixture is using the fake by default	Make the real/fake choice explicit and visible in test output

R3 · Async Python, digital logic, and your first Verilog (12 h)

Goal: enough asyncio to read cocotb testbenches, the digital-logic ideas Verilog assumes, and your first simulated modules.

You will have by Sunday: a counter and an adder in Verilog, each verified by a plain testbench and by a cocotb test. **Install before S3:** Icarus Verilog and GTKWave (or Verilator), per W0 S3.

S1 · asyncio, just enough for cocotb (2 h)

cocotb testbenches are Python `async` functions. You do not need all of asyncio — just the shape.

1. Work through this, predicting the output order before running it:

```
import asyncio

async def ticker(name, period, n):
    for i in range(n):
        print(f"{name} tick {i}")
        await asyncio.sleep(period)

async def main():
    task = asyncio.create_task(ticker("fast", 0.1, 5)) # runs in the background
    await ticker("slow", 0.25, 2)                    # runs here, in the foreground
    await task                                         # wait for the background one to finish

asyncio.run(main())
```

2. Understand three ideas: `async def` defines a coroutine; `await` pauses until something finishes; `create_task` starts something that runs alongside. In cocotb these become: a test is an `async def`; `await RisingEdge(clk)` waits for a clock edge; `cocotb.start_soon(...)` starts a clock or a servo model running alongside the test.
3. Write a small program where one coroutine "produces" bytes into an `asyncio.Queue` and another "consumes" them. That is the shape of W5's behavioural servo model: one coroutine plays the servo while the test plays the master.

Done when: you can predict the interleaved output of the listing above and explain why.

S2 · Digital logic, on paper (2 h)

Verilog describes hardware, not steps. These ideas have to be in place first, or Verilog looks like a strange programming language.

1. **Combinational logic:** outputs depend only on current inputs. Draw a 2-to-1 multiplexer and a 4-bit adder as gates or blocks.
2. **Sequential logic:** a D flip-flop copies its input to its output on the clock edge and holds it otherwise. Draw a 4-bit counter as four flip-flops plus an adder feeding back.
3. **Clocks, and why everything waits for the edge.** Sketch a timing diagram for your counter over eight clock cycles, including a reset.
4. **Finite state machines.** Draw, as circles and arrows, a machine that detects the byte sequence `FF FF` in a stream — states IDLE, GOT_ONE, FOUND. That is the first thing W5's packet receiver does.
5. **Metastability, in one paragraph.** Read a short explanation of why an asynchronous input (a button, a serial line) must pass through two flip-flops before your logic uses it. Write the explanation in your own words. You will meet it on hardware next week and in an interview in W13.

Done when: your counter timing diagram and your `FF FF` detector state diagram are drawn and committed as photos.

S3 · First Verilog: combinational modules in simulation (2 h)

1. Write `ramp/r3/mux2.sv` and `adder4.sv` using `assign` statements and an `always_comb` block. Keep them tiny.
2. Write a plain SystemVerilog testbench that drives every input combination, prints the outputs with `$display`, and dumps a waveform:

```

module tb_adder4;
  logic [3:0] a, b;
  logic [4:0] sum;
  adder4 dut (.a(a), .b(b), .sum(sum));
  initial begin
    $dumpfile("adder4.vcd"); $dumpvars(0, tb_adder4);
    for (int i = 0; i < 16; i++)
      for (int j = 0; j < 16; j++) begin
        a = i; b = j; #1;
        if (sum !== i + j) $display("FAIL %d + %d = %d", i, j, sum);
      end
    $display("done");
    $finish;
  end
endmodule

```

3. Compile and run: `iverilog -g2012 -o sim adder4.sv tb_adder4.sv && vvp sim`, then open `adder4.vcd` in GTKWave and find one addition in the waveform.
4. Break the adder on purpose (drop the carry) and watch the testbench catch it.

Done when: the testbench reports failures for the broken adder and none for the correct one, and you have seen the waveform.

S4 · Sequential Verilog: the counter (2 h)

1. Write `ramp/r3/counter32.sv` — this is the same module W1 S3 uses, so you are building W1's first piece now:

```

module counter32 (
  input logic      clk,
  input logic      rstn,
  output logic [31:0] count
);
  always_ff @(posedge clk) begin
    if (!rstn) count <= '0;
    else      count <= count + 1'b1;
  end
endmodule

```

2. Understand `<=` (non-blocking, used in `always_ff`) against `=` (blocking, used in `always_comb`). The rule to memorise: **flip-flops get `<=`, combinational logic gets `=`**. Break it on purpose later and see what goes wrong.
3. Write a plain testbench with a generated clock (`always #5 clk = ~clk;`), hold reset for two cycles, release it, run ten cycles, and print the count. **Predict the number first**. If you are off by one, work out whether reset released before or after a clock edge — that off-by-one is the lesson, and it is the same one that bites the turnaround timing in W6.
4. Look at the waveform in GTKWave and match it against your paper timing diagram from S2.

Done when: your prediction of the count matches the simulation, and you can explain why.

S5 · Saturday: your first cocotb tests (4 h)

1. Install cocotb (already in the venv) and set up `ramp/r3/cocotb_counter/` with a Makefile:

```

SIM ?= icarus
TOPLEVEL_LANG ?= verilog
VERILOG_SOURCES = ../counter32.sv
TOPLEVEL = counter32
MODULE = test_counter
COMPILE_ARGS += -g2012
include $(shell cocotb-config --makefiles)/Makefile.sim

```

2. Write `test_counter.py` :

```
import cocotb
from cocotb.clock import Clock
from cocotb.triggers import ClockCycles

@cocotb.test()
async def counts_after_reset(dut):
    cocotb.start_soon(Clock(dut.clk, 10, unit="ns").start()) # cocotb < 2.0: units="ns"
    dut.rstn.value = 0
    await ClockCycles(dut.clk, 2)
    dut.rstn.value = 1
    await ClockCycles(dut.clk, 10)
    predicted = 10 # write your own prediction here, from S4
    assert int(dut.count.value) == predicted, f"got {int(dut.count.value)}"
```

3. Run `make` . Make the assertion fail on purpose, read cocotb's error output, and fix it. Learning to read cocotb's failure messages now saves hours in W5.
4. Add three more tests: the count holds at zero while reset is held; reset mid-count returns to zero; the counter wraps from `0xFFFFFFFF` to `0` . For the wrap test you cannot wait four billion cycles — work out how to test it anyway (hint: parameterise the width, or force a value). Working that out is a real verification skill.
5. Write the adder's cocotb test too, checking all 256 combinations in a Python loop. Compare it with the plain SystemVerilog testbench: which is easier to extend? Write two sentences on it.

Done when: four counter tests and one adder test pass in cocotb, and each has been seen to fail.

Failure modes

Symptom	Likely cause	Fix
<code>iverilog</code> syntax errors on <code>logic</code> or <code>always_ff</code>	Missing <code>-g2012</code>	Add the flag; or switch to Verilator
Count is off by one from your prediction	Reset released relative to the edge	Draw it; this is the lesson, not a bug
cocotb can't find the module	<code>TOPLEVEL</code> or <code>MODULE</code> name mismatch	Names must match the module and the Python file exactly
<code>TypeError</code> on <code>Clock(... unit=...)</code>	cocotb version before 2.0	Use <code>units="ns"</code> and note your version

R4 · Verilog on the PYNQ-Z2: clocks, buttons, and a UART transmitter (12 h)

Goal: your own logic running on the board, and a UART transmitter talking to your PC. **You will have by Sunday:** the PYNQ-Z2 sending "Hello" to a terminal on your PC when you press a button. **Setup:** set the PYNQ-Z2 boot jumper to **JTAG** for these standalone exercises, and program the PL from Vivado's Hardware Manager over the micro-USB cable — which needs the JTAG cable drivers from W0 S3. W1 goes back to SD boot and PYNQ. **Clock:** the PYNQ-Z2 feeds a 125 MHz clock into the PL. Take its pin name from the board's master XDC, along with the LEDs, buttons and a Pmod pin. **Safety:** your USB-UART adapter must be a 3.3 V one. PYNQ-Z2 PL pins are not 5 V tolerant. Connect grounds first.

S1 · First bitstream: a blinking LED (2 h)

1. Create a Vivado project for the PYNQ-Z2 part, add your `counter32.sv` and a top module that connects the counter's bit 26 to LED 0. At 125 MHz, bit 26 toggles about every half-second — work out the exact period before building.
2. Copy the PYNQ-Z2 master XDC into the project and uncomment only the clock, LED 0 lines.

3. Run synthesis, implementation and bitstream generation. Use the GUI for this first one — W1 converts it to a script — but save the project's Tcl journal (`vivado.jou`) and look at the commands the GUI ran on your behalf.
4. Program the board from Hardware Manager and watch the LED. Time ten blinks with a stopwatch and compare with your predicted period.
5. Drive all four LEDs from four different counter bits and check the ratios.

Done when: the measured blink period matches your prediction within stopwatch error.

S2 · Buttons, synchronisers and debouncing (2 h)

1. Connect button 0 directly to LED 0 and check it works. Then connect it to a counter that increments on each press, shown in binary on the four LEDs. Press it ten times and count what the LEDs show — it will likely jump by more than one per press. That is contact bounce, and you are about to fix it.
2. Add a **two-flip-flop synchroniser** on the button input — the metastability protection from R3 S2 — then a debouncer: a counter that only accepts a new button state after it has been stable for about 10 ms. At 125 MHz, compute the count for 10 ms.
3. Add an edge detector so the press counter increments once per press, not once per clock while held.
4. Simulate the debouncer in cocotb before building: drive a bouncy input (a few fast toggles, then steady) and check it outputs one clean edge.
5. Build and test on the board. Ten presses should now read exactly ten.

Done when: ten presses give exactly ten counts, and the debouncer has a passing cocotb test.

S3 · UART transmitter in simulation (2 h)

1. Read about the UART frame: idle high, one start bit (low), eight data bits least-significant first, one stop bit (high). Draw one byte — `0x48` , the letter H — as a timing diagram.
2. Write `ramp/r4/uart_tx.sv` with this interface, which deliberately matches W5's:

```
module uart_tx (
    input logic      clk, rstn,
    input logic [15:0] clks_per_bit, // 125 MHz / 115200 ≈ 1085
    input logic [7:0] data,
    input logic      valid,
    output logic     ready,
    output logic     tx,
    output logic     active
);
```

Write it yourself as a state machine — IDLE, START, DATA, STOP — before you look at the W5 S1 listing. Then compare. Differences are fine; understanding them is the point. 3. Write a cocotb test that sends one byte and **decodes it from the tx line** by sampling at the middle of each bit period, then asserts the decoded byte equals the sent one. That decoder is the seed of W5's servo model. 4. Test several bytes, including `0x00` , `0xFF` and `0x55` . Each catches a different class of bug.

Done when: the cocotb test decodes every byte you send, including the three edge cases.

S4 · UART on the board (2 h)

1. Compute `clks_per_bit` for 115200 baud at 125 MHz, and the actual baud rate that produces. Work out the percentage error. (UART tolerates a few percent; yours will be far smaller.)
2. Build a top module that sends the byte `0x48` ("H") once per button press, using the debounced press from S2 as `valid` .
3. Constrain `tx` to a Pmod pin from the master XDC. Wire it to the USB-UART adapter's **RX**, and connect **ground to ground** before anything else.
4. Open a serial terminal on the laptop at 115200 8N1 — `python -m serial.tools.miniterm /dev/ttyUSB0 115200` needs nothing extra installed — and press the button. An `H` should appear.

- Put the scope on the `tx` pin and capture one byte. Measure the bit period and match the waveform against your R4 S3 drawing, bit by bit. This is exactly the W2 S5 skill, on a signal you designed.

Done when: an `H` appears on each press, and the scope capture matches your drawing.

S5 · Saturday: "Hello" from a state machine (4 h)

- Extend the design to send a whole string — `Hello\r\n` — on each press. You need a small ROM of bytes, an index counter, and a state machine that waits for `ready`, sends the next byte, and stops at the end. This is the transmit side of W5's packet framer in miniature.
- Simulate the whole thing in cocotb first. Reuse your S3 decoder to collect the bytes and assert the string arrives intact.
- Build, program and test on the board.
- Add a second mode, selected by a switch, that sends the string at 1,000,000 baud instead — `clks_per_bit` of 125. Does your adapter keep up? Record the result either way; it tells you whether the adapter is usable as a bus probe later.
- Write `ramp/r4/README.md`: what you built, the waveforms, and the three bugs you hit and how you found each. Those bug notes are the first entries in what becomes your W13 STAR stories.

Done when: `Hello` arrives on every press at 115200 baud, verified in simulation first.

Failure modes

Symptom	Likely cause	Fix
Hardware Manager finds no target	JTAG cable drivers not installed	Run <code>install_drivers</code> with <code>sudo</code> (W0 S3), then unplug and replug the board
Bitstream builds but the LED does nothing	Clock pin not constrained, or wrong pin	Check the XDC against the master file; read the implementation warnings
Terminal shows garbage characters	Baud mismatch, or TX and RX swapped	Recompute <code>clks_per_bit</code> ; FPGA <code>tx</code> goes to adapter RX
Nothing at all in the terminal	No common ground	Ground to ground first, always
Press counter jumps erratically	Missing synchroniser or debouncer	That is S2's lesson; check both are in the path
Works in simulation, not on the board	Reset polarity, or an unconstrained pin	Check critical warnings in the implementation report

R5 · UART receiver, the PS-PL bridge, and Gate 0 (12 h)

Goal: a UART receiver, your first AXI-Lite register read from Python on the PYNQ, and an honest check that you are ready for the main plan. **You will have by Sunday:** the board echoing whatever you type, with a byte count readable from a Jupyter notebook.

S1 · UART receiver in simulation (2 h)

- Write `ramp/r5/uart_rx.sv`. Put the two-flip-flop synchroniser on the input first. Detect the falling edge of the start bit, wait half a bit period, check it is still low, then sample each data bit at its centre and check the stop bit.
- Output the byte with a one-cycle `valid` pulse, and flag a framing error if the stop bit is low.
- Test it in cocotb by connecting your R4 transmitter's output straight to the receiver's input and checking every byte from `0x00` to `0xFF` round-trips.
- Add a baud-mismatch test: drive the receiver from a transmitter running 2 % fast, then 4 % fast. Record where it starts failing. W5 S2 asks for exactly this number.

Done when: all 256 bytes round-trip, and you know your receiver's baud tolerance.

S2 · Echo on the board (2 h)

1. Build a top module: receiver from a Pmod pin (wired to the adapter's TX), transmitter to another Pmod pin (wired to the adapter's RX), each received byte sent straight back.
2. Type in the terminal and watch your characters echo. Paste a long paragraph and check nothing is dropped.
3. Add a small FIFO between receiver and transmitter, or explain in writing why you don't need one for an echo. (Hint: compare how long it takes to receive a byte with how long it takes to send one.)
4. Scope both lines at once and measure the delay from the end of a received byte to the start of its echo. That is your first latency measurement on logic you designed, and the same shape as W10's.

Done when: a pasted paragraph echoes intact, and you have a scope capture of receive-to-echo delay.

S3 · Your first AXI-Lite peripheral (2 h)

This session is a small preview of W1 and W6, and it is allowed to be rough.

1. Set the boot jumper back to **SD** and boot PYNQ. Confirm Jupyter works (W1 S1 covers this in detail).
2. In Vivado, use **Tools** → **Create and Package New IP** → **AXI4 peripheral** to generate a template with four 32-bit registers. Read the generated Verilog and find where a write from the processor lands in a register, and where a read is answered. Annotate those lines with comments in your own words.
3. Build a block design with the Zynq PS, your IP on `M_AXI_GP0`, and the automation connections. Generate the bitstream and export the `.hwh` file — W1 S4 explains the file names.
4. Load it in a notebook and read and write the registers:

```
from pynq import Overlay, MMIO
ol = Overlay("/home/xilinx/ramp/axi_regs.bit")
ip = ol.ip_dict["myip_0"] # your instance name may differ
mm = MMIO(ip["phys_addr"], ip["addr_range"])
mm.write(0x0, 0x12345678)
print(hex(mm.read(0x0)), hex(mm.read(0x4)))
```

Done when: a value written from Python reads back from the PL register.

S4 · Joining the two worlds (2 h)

1. Combine S2 and S3: put the echo UART inside your AXI-Lite IP, and make register 1 a **count of bytes received**, readable from Python.
2. Type characters in the serial terminal and watch the count rise in the notebook.
3. Add a write: register 2 sets a byte that the IP sends out of the UART when written. You now have Python commanding a UART through your own hardware — which is precisely what W6's `DefaultIP` driver does to the servo bus.
4. Write down every place in this design where two clocks meet, or an asynchronous signal enters. Everything runs on one clock except the UART input — which your synchroniser handles. That is the CDC answer W5 asks for, in miniature.

Done when: the notebook shows the received-byte count increasing as you type, and a register write sends a byte.

S5 · Saturday: Gate 0, honestly (4 h)

First, one optional hour on KiCad. Work through the KiCad getting-started tutorial: draw a small schematic, assign footprints, lay out a tiny board. W7 is not the week to meet the interface for the first time.

Then sit Gate 0. Do each item **without looking anything up**, timed, and record pass or fail honestly in `ramp/gate0.md`.

#	Task	Time limit	Tests readiness for
1	Write a function that validates a packet's checksum, plus three pytest tests	20 min	W3
2	Write a small class with a custom exception and a context manager that always cleans up	20 min	W3, W11
3	Read a CSV of numbers, print median and p99, save a labelled histogram	15 min	W3, W6, W10
4	Explain <code>async def</code> , <code>await</code> and <code>start_soon</code> in <code>cocotb</code> terms, in writing	10 min	W5
5	Write a 4-state FSM in SystemVerilog with correct <code><=</code> / <code>=</code> use, from a state diagram you draw	25 min	W5
6	Write a <code>cocotb</code> test for it, including one test you expect to fail	20 min	W5
7	Explain, in writing, why the UART input needs two flip-flops	5 min	W5, W13
8	Given a clock frequency and baud rate, compute <code>clks_per_bit</code> and the baud error	5 min	W5
9	Load an overlay on PYNQ and read an AXI-Lite register from Python	15 min	W1, W6
10	Find a pin name in the master XDC and constrain it correctly	5 min	W1, W6

Scoring and what to do with it.

- **Nine or ten passes:** you are ready. Start W1 on schedule.
- **Seven or eight:** start W1 anyway, but note the failed items. Revisit each in the first session of the week that needs it — the table says which.
- **Six or fewer:** take one more ramp-up week focused on the failed items, and take it from the holiday buffer (the schedule has two weeks there). Starting W1 unready costs more than a week later.

Finally, write a half-page in `ramp/gate0.md` on what was hardest, what surprised you, and how you learn best. Revisit it in W13; it is the opening of your lessons-learned.

Done when: `gate0.md` records a score, a decision, and the half-page reflection.

Failure modes

Symptom	Likely cause	Fix
Echo drops characters on a long paste	Transmitter busy when the next byte arrives	A small FIFO, or the arithmetic in S2 step 3
<code>overlay()</code> can't find your IP	<code>.hwh</code> not next to <code>.bit</code> , or base names differ	Same base name, same folder
Register reads always return zero	Wrong offset, or reading a register the template doesn't wire	Read the generated Verilog's address decode
Gate 0 feels artificial	It is timed and unaided on purpose	That is what W5 feels like; better to find out now

Month 1 · Foundations: know both boards better than their vendors expect

W01 · PYNQ-Z2 bring-up and the Zynq PS/PL split (12 h)

Competency: SoC architecture literacy and reproducible builds. **Prerequisites:** W0 complete; microSD written and checksummed. **Bench setup:** PYNQ-Z2, Ethernet to your router or direct to the laptop, scope with a short ground lead, bench supply at 12 V / 1 A limit for the barrel jack.

S1 · Boot the board and prove the base overlay (2 h)

1. Set the board jumpers: boot source to **SD**, power source to **REG** if you are feeding the barrel jack, or **USB** if powering from the 2 A adapter. Photograph the jumper positions — you will change them later and want the reference.
2. Insert the microSD, connect Ethernet, apply power. Watch the boot LEDs: the done LED asserts, then the four user LEDs flash in sequence when PYNQ's boot service comes up. If the LEDs never flash, the image is bad — reflash before debugging anything else.
3. Find the board. On a DHCP network try `http://pynq:9090`; on a direct laptop connection (a USB-Ethernet adapter is fine), give that interface a static address of `192.168.2.1/24` in Ubuntu's network settings — IPv4, Manual, no gateway — and browse to `http://192.168.2.99:9090`. Default credentials are `xilinx / xilinx`.
4. SSH in and capture the baseline:

```
ssh xilinx@pynq
uname -a; cat /etc/os-release
python3 -c "import pynq; print(pynq.__version__)"
free -h; df -h /
```

Paste that into `docs/pynq-baseline.md`.

5. Run the base-overlay notebooks under `base/board/`: LEDs, buttons, switches, and one Pmod example. Confirm each physically — press the button, see the LED. This proves the image, the bitstream loading path and the PL power rails in one go.
6. Read the PYNQ v3.1 changelog and release notes. Write a short list in `docs/pynq-baseline.md` of what differs from the v3.0-era tutorials you will find online. Two that bite: the Python version moved, and some `pynq.lib` import paths changed.

Done when: you can toggle a physical LED from a Jupyter cell, and the baseline file is committed.

S2 · The PS/PL split, read properly (2 h)

Reading, not typing. Take notes by hand; you will sketch this from memory on Saturday.

1. UG585, the AXI interconnect chapter: the **GP** ports (two master, two slave, 32-bit, low throughput, control), the **HP** ports (four, 64-bit, high throughput, DDR-attached, for DMA), and the **ACP** port (cache-coherent, 64-bit). For each, note the width, the typical use, and one reason *not* to use it.
2. The clocking chapter: how `FCLK_CLK0..3` are derived from the PS PLLs, what limits their frequency, and why the clocking wizard's requested frequency is not always what you get.
3. The boot flow: BootROM → FSBL → bitstream → U-Boot → Linux, and where PYNQ's `overlay()` sits relative to that (it reprograms the PL *after* Linux is up, through the FPGA manager).
4. Answer these four in writing, because they are interview questions verbatim:
 - Which AXI port would you use for the servo bus-master IP, and why not one of the others?
 - What is the bandwidth ceiling of a GP port at 100 MHz, 32-bit?
 - What has to be true for the PS to read a PL register safely while the PL is writing it?
 - What happens to PL outputs during the window when `overlay()` reloads the bitstream?

Question 4's last item matters for safety and returns in W12 — a bitstream reload deasserts your bus drivers and your load-switch hold, so note it now.

Done when: docs/ps-pl-notes.md exists with answers to all four questions, one page.

S3 · A scripted overlay, part one: write the Tcl (2 h)

The rule for the whole project: **no GUI clicks that are not also in a script**. If you explore in the GUI, export the Tcl afterwards and keep the script as the source of truth.

1. Write the counter RTL, rtl/counter32/counter32.sv :

```
module counter32 (  
    input  logic      clk,  
    input  logic      rstn,  
    output logic [31:0] count  
);  
    always_ff @(posedge clk) begin  
        if (!rstn) count <= '0;  
        else      count <= count + 1'b1;  
    end  
endmodule
```

2. Write overlay/hello/build.tcl . Start from this skeleton and fill in from your installed board files:

```

set proj hello
set part xc7z020clg400-1
create_project $proj ./build/$proj -part $part -force
# Board part name depends on which board files you installed.
# Run `get_board_parts` in the Tcl console and paste the exact string here.
set_property board_part tul.com.tw:pynq-z2:part0:1.0 [current_project]

add_files -norecurse ../../rtl/counter32/counter32.sv
add_files -fileset constrs_1 -norecurse ./hello.xdc

create_bd_design "system"
create_bd_cell -type ip -vlnv xilinx.com:ip:processing_system7 ps7
apply_bd_automation -rule xilinx.com:bd_rule:processing_system7 \
    -config {make_external "FIXED_IO, DDR" apply_board_preset "1" \
        Master "Disable" Slave "Disable"} [get_bd_cells ps7]

create_bd_cell -type module -reference counter32 cnt

create_bd_cell -type ip -vlnv xilinx.com:ip:axi_gpio gpio_cnt
set_property -dict [list CONFIG.C_GPIO_WIDTH {32} CONFIG.C_ALL_INPUTS {1}] \
    [get_bd_cells gpio_cnt]

create_bd_cell -type ip -vlnv xilinx.com:ip:axi_gpio gpio_led
set_property -dict [list CONFIG.C_GPIO_WIDTH {4} CONFIG.C_ALL_OUTPUTS {1}] \
    [get_bd_cells gpio_led]

apply_bd_automation -rule xilinx.com:bd_rule:axi4 \
    -config {Master "/ps7/M_AXI_GP0" Clk "Auto"} [get_bd_intf_pins gpio_cnt/S_AXI]
apply_bd_automation -rule xilinx.com:bd_rule:axi4 \
    -config {Master "/ps7/M_AXI_GP0" Clk "Auto"} [get_bd_intf_pins gpio_led/S_AXI]

connect_bd_net [get_bd_pins ps7/FCLK_CLK0] [get_bd_pins cnt/clk]
connect_bd_net [get_bd_pins ps7/FCLK_RESET0_N] [get_bd_pins cnt/rstn]
connect_bd_net [get_bd_pins cnt/count] [get_bd_pins gpio_cnt/gpio_io_i]

make_external -name fclk_probe [get_bd_pins ps7/FCLK_CLK0]

validate_bd_design
save_bd_design
make_wrapper -files [get_files ./build/$proj/$proj.srcs/sources_1/bd/system/system.bd] -top
add_files -norecurse ./build/$proj/$proj.gen/sources_1/bd/system/hdl/system_wrapper.v
set_property top system_wrapper [current_fileset]

launch_runs impl_1 -to_step write_bitstream -jobs 4
wait_on_run impl_1

```

3. Write `overlay/hello/hello.xdc` . Do **not** invent pin names. Download the PYNQ-Z2 master XDC from the board vendor, copy it into `overlay/` , and uncomment only the lines you need: the four LEDs and one Pmod pin for `fclk_probe` . Constrain the probe with a matching `IOSTANDARD LVCMOS33` .
4. Add a `README.md` in `overlay/hello/` that states the Vivado version it was proven on and the single command to rebuild:

```
vivado -mode batch -source build.tcl
```

Done when: the Tcl runs to the end of synthesis without errors. Implementation can finish while you sleep.

S4 · A scripted overlay, part two: load it and read it (2 h)

1. Collect the two artifacts. Paths for 2020.2 and later:

```

build/hello/hello.runs/impl_1/system_wrapper.bit
build/hello/hello.gen/sources_1/bd/system/hw_handoff/system.hwh

```

2. Rename both to the **same base name** and copy to the board. PYNQ matches `.hwh` to `.bit` by filename; a mismatch gives you a confusing "no such IP" error later.

```
scp system_wrapper.bit xilinx@pynq:~/overlays/hello/hello.bit
scp system.hwh xilinx@pynq:~/overlays/hello/hello.hwh
```

3. Load and inspect from Python:

```
from pynq import Overlay
ol = Overlay('/home/xilinx/overlays/hello/hello.bit')
print(ol.ip_dict.keys()) # expect gpio_cnt, gpio_led
```

4. Read the counter twice with a known delay and check the difference against the clock frequency — this is your first end-to-end PS↔PL proof:

```
import time
from pynq.lib import AxiGPIO
from pynq.ps import Clocks

cnt = AxiGPIO(ol.ip_dict['gpio_cnt']).channel1
a = cnt.read(); time.sleep(1.0); b = cnt.read()
delta = (b - a) & 0xFFFFFFFF
print(f"{delta/1e6:.2f} Mcounts/s vs FCLK0 {Clocks.fclk0_mhz:.2f} MHz")
```

Expect the two to agree within a percent or two — the error is your `sleep` and Python overhead, not the PL. If they disagree by an integer factor, you are reading a different clock than you think.

5. Drive the LEDs from `gpio_led` to prove the write path as well as the read path.
6. **The point of the week:** this is your evidence that a Vivado 2026.1 bitstream loads on a 2024.1-era PYNQ image. Record it explicitly in `docs/decisions/0002-toolchain-version-skew.md` with the version strings of both sides. If it *fails*, fall back to 2024.1 for the whole plan and record that as the decision with the error message attached — that document is worth more in an interview than a success would be.

Done when: the counter rate matches FCLK0, and the toolchain-skew decision record is written either way.

S5 · Saturday: instruments on the board (4 h)

1. **Board current.** Feed the barrel jack from the bench supply (check your board's accepted input range before choosing the voltage; PYNQ-Z2 documents a wide input) with a 1 A limit. Record supply current at three points: (a) Linux booted, PL empty; (b) base overlay loaded; (c) your `hello` overlay loaded. Three numbers, one table.
2. **PL clock on a pin.** Probe the Pmod pin carrying `fclk_probe` with the scope, ground lead as short as you can make it — use the probe's spring ground, not the long flying lead, or you will measure your ground loop instead of the edge.
 - Measure frequency and compare to `Clocks.fclk0_mhz`.
 - Measure 10–90 % rise time and overshoot. Note them; W8's signal-integrity work refers back to this as your baseline for what the board's own I/O looks like.
 - Change the requested FCLK0 in the Tcl to a value the PLL cannot hit exactly (try 33 MHz), rebuild, and measure again. The delta between requested and actual is the practical lesson from S2's clocking reading.
3. **Camera on the PS.** Plug the kit's wrist camera (the 480p one) into the board's USB port directly (hub comes later) and characterise what you actually get. Repeat afterwards with the 1080p external camera at 640×480 and at its native resolution — W4's camera budget needs both:

```
import cv2, time
cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
n, t0 = 0, time.time()
while n < 120:
    ok, frame = cap.read()
    if not ok: break
    n += 1
print(f"{n/(time.time()-t0):.1f} fps at "
      f"{int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))}x{int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))}")
cap.release()
```

Also run `v4l2-ctl --list-formats-ext -d /dev/video0` and save the output — it lists which resolution and format combinations the camera will actually negotiate, which is the input to W4's camera bandwidth budget. Note the CPU load during capture (`top` in another shell); MJPEG decode on the A9 is not free. 4. **Sketch the Zynq-7020 from memory** — PS with its two A9s, DDR controller, peripherals; the PL; the GP/HP/ACP ports between them; the FCLKs. Then open UG585 and mark what you got wrong in a different colour. Photograph both and put them in `docs/ps-pl-notes.md` . The corrections are the useful part. 5. Write `overlay/hello/README.md` properly: what it contains, how to rebuild from a clean checkout, which Vivado version it was proven on, and what the counter check proves. Then test it — move to a clean directory, clone your own repo, and run the rebuild command. If it fails, fix the README, not your memory.

Done when: the clean-checkout rebuild works, because that is the actual deliverable.

Measurements to record

Quantity	How	Expect	Lands in
Board current, PL empty / base / hello	Bench supply readout	Hundreds of mA, rising with PL load	<code>docs/pynq-baseline.md</code>
FCLK0 requested vs measured	Clocking wizard vs scope	Exact for 100 MHz, off for 33 MHz	<code>docs/ps-pl-notes.md</code>
FCLK0 rise time at the Pmod pin	Scope, spring ground	A few ns	<code>docs/ps-pl-notes.md</code>
Camera fps and format at 640×480	Script above + <code>v4l2-ctl</code>	30 fps MJPEG; YUY2 much lower	<code>docs/camera-baseline.md</code>
CPU load during capture	<code>top</code>	Note it; it constrains W4	<code>docs/camera-baseline.md</code>

Deliverables

-  `overlay/hello/` — `build.tcl` , `hello.xdc` , README, proven by clean-checkout rebuild
-  `docs/ps-pl-notes.md` — one page, four answered questions, corrected sketch
-  `docs/pynq-baseline.md` , `docs/camera-baseline.md`
-  `docs/decisions/0002-toolchain-version-skew.md`

Failure modes

Symptom	Likely cause	Fix
LEDs never flash at boot	Bad SD write or wrong boot jumper	Re-verify checksum, reflash, check jumper
<code>overlay()</code> raises about missing IP	<code>.hwh</code> base name differs from <code>.bit</code>	Rename both to one base name
Counter rate is an exact fraction of FCLK	Counter clocked from a different FCLK than probed	Check <code>connect_bd_net</code> , not the Python
Board resets when the camera is plugged in	USB inrush on an undersized supply	Use the 2 A adapter or the powered hub
Vivado cannot find the board part	Board files not installed	<code>get_board_parts</code> ; install files or drop to <code>-part</code> only

W02 · Both arms running, then characterised before you trust them (12 h)

Competency: lab measurement discipline. **Prerequisites:** W1 done; both supplies labelled; logic analyser or scope with UART decode. **Safety:** this is the first week the arms move. Clear a 60 cm radius around the follower. Keep one hand on the supply output button throughout every energised step.

S1 · Inventory the servos with ServoStudio (2 h)

The kit arrives assembled, so IDs and baud are already set. The job is to **verify and record** the factory configuration, not to reassign it — and to capture the protection settings W12 will build on.

1. **Rail check:** bench supply at 12.0 V on its display, **limit 2 A**, output off.
2. This session runs on the **Windows desktop**, because ServoStudio is Windows-only. Connect one BusLinker V3.0 to the desktop by USB-C. Set the communication jumper to the **Servo + USB** position shown in the BusLinker manual, connect the follower arm's bus cable, then connect power and switch the output on.
3. Open ServoStudio, select the port, **1,000,000 baud**, Connect, then **Scan** (IDs 1–253). Expect six servos. Record, for each: ID, model (HX-30HM), and firmware version.
4. Confirm the ID map runs 1–6 from base to gripper and matches what Hiwonder's LeRobot configuration expects:

ID	Joint	Follower servo	Leader servo
1	shoulder pan	HX-30HM	HX-10HM
2	shoulder lift	HX-30HM	HX-10HM
3	elbow flex	HX-30HM	HX-10HM
4	wrist flex	HX-30HM	HX-10HM
5	wrist roll	HX-30HM	HX-10HM
6	gripper	HX-30HM	HX-10HM

If an ID is wrong, fix it **one servo at a time, disconnected from the chain** — two servos sharing an ID produce garbled replies with no way to tell which one answered. 5. Open **Config** for each servo and screenshot every page into `docs/servo-config/`: ID and baud; operating mode (should be **Position**); offset; min/max angle; startup torque and torque limit; position and velocity PID; and the **Protection** page — under- and over-voltage thresholds, current maximum and hold time, overload threshold, and over-temperature threshold. These factory protection settings are the servo's own safety layer, and W12 has to know exactly what they are before it layers anything on top. 6. Repeat 2–5 for the leader arm with the second BusLinker. 7. Note two things for W3: whether settings survive a power cycle (the product documents say they do), and whether ServoStudio unlocks anything

before writing — watch the logic analyser in W3 to see it. 8. **Firmware policy.** ServoStudio can reflash servo firmware. Do not update anything now. Record the versions you have, and if you ever do update, write a decision record with before and after versions — a firmware change mid-project invalidates every earlier measurement on that servo.

Done when: `docs/arm-bringup.md` has the ID map with model and firmware version per servo, and `docs/servo-config/` has every configuration page for all twelve.

S2 · LeRobot calibration and first teleoperation (2 h)

1. **Rail check.** 12.0 V on the display, both arms. Check the `LEADER / FOLLOWER` flag on each cable against the BusLinker it goes to.
2. Follow **Hiwonder's SO-ARM101 LeRobot tutorial** for installation, not only the upstream LeRobot docs — they assume Feetech servos. While installing, find and write down how the HX servos are driven: upstream LeRobot's Feetech motor-bus code, a Hiwonder-supplied motor-bus class, or a fork. Put the answer and the evidence (file path, package name, commit) at the top of `docs/servo-notes.md`. It is the single most useful fact for W3, because it tells you how Feetech-like the protocol really is.
3. Identify which BusLinker is which:

```
lerobot-find-port      # check --help; the entry point name has changed between releases
```

Back on the laptop from here on. Record the two ports by their stable names under `/dev/serial/by-id/` — `ttyUSB0` and `ttyUSB1` can swap between reboots — and tape a label on each BusLinker. If the tool asks for a port, give it the `/dev/serial/by-id/...` path. 4. Run calibration for the follower, then the leader, following the prompts to move each joint through its range. Save the calibration files into the repo under `sw/calib/` — they are per-arm data and you will regret regenerating them. 5. Teleoperate: move the leader by hand, watch the follower track. Keep the follower's workspace clear. Stop at the first sign of a joint fighting itself. 6. Record a two-minute LeRobot dataset with the kit's wrist and external cameras attached to the **laptop** (not the board yet). The purpose is to see the whole pipeline work once, end to end, before you start replacing pieces of it. Note the dataset path and frame counts. 7. Write down three qualitative observations about the teleop feel: lag, deadband, any joint that overshoots. These become hypotheses that W3's latency numbers confirm or kill.

Done when: a two-minute dataset exists and both arms survived.

S3 · Current characterisation, follower (2 h)

The goal is a table that W4's power budget and W7's part selection are built on. Every number comes from the bench supply's own readout, with the limit set as a backstop.

1. Follower on 12 V, **limit 5 A**. Leader disconnected. For reference, Hiwonder specifies the HX-30HM at about **100 mA no-load and 3 A stall** — six stalled servos would be 18 A on paper, far beyond the kit's 5 A adapter. Your measurements will show how far real operation sits from that worst case, which is the point of the table.
2. Record current in each of these states, holding each for 10 s and noting mean and peak:

State	How to produce it	Record
Torque off, powered	Torque switch off in ServoStudio, all six	Quiescent draw
Idle, torque on, arm supported	Enable torque in a neutral pose	Holding current, no load
Holding against gravity, joint 2 loaded	Extend the arm horizontally	Worst static case
Fast move, all joints	A scripted sweep	Peak dynamic
Stall, < 2 s, soft stop	Command past a padded block	Peak, then stop immediately

3. The stall test is the dangerous one. Rules: keep it under two seconds, keep the supply limit at 5 A, use a soft block, and do it once per joint at most. If a servo gets hot to the touch, stop for the evening.

- The HX servos have their **own** current and overload protection, configured on the Protection page you screenshotted in S1. During a stall, one of three things ends it: the servo's protection, the supply limit, or you. **Record which one**, and the time it took. If the servo's protection trips first, you have measured its real reaction time — a number W12 needs and the datasheet does not give you.
4. Capture the *shape* of the fast-move transient on the scope across a small series shunt or with a current probe if you have one. You need the **duration** of the current step, not just its height: W7 sizes bulk capacitance as $C = I \cdot \Delta t / \Delta V$, and Δt comes from this capture. Without it you are guessing.
 5. Note ambient temperature and servo case temperature after the sequence.

Done when: the follower table is complete, including a measured Δt for the current step.

S4 · Current characterisation, leader; mechanical survey (2 h)

1. **Rail check**, then leader on 12 V, **limit 3 A**. Record: torque off; torque on idle; being back-driven by hand through its full range. The leader is normally back-driven with torque off, so the middle row matters less — but W7 has to size the leader rail's load switch and shunt for the worst case someone might command, so record it anyway.
2. Mechanical survey, torque off on both arms, written as observations with numbers where you can get them:
 - **Backlash at the gripper:** hold the base, wiggle the gripper, estimate the free play in millimetres at the tip. Do it for each joint locked in turn if you can. Hiwonder markets the HX-30HM as zero-backlash; whatever play you feel is therefore in the printed parts, the horns or the gear train downstream of the encoder. Note it now — W11 measures it properly and tests the claim.
 - **Sag:** which follower joints drop under gravity with torque off, and how far.
 - **Back-drive:** how freely each leader joint moves by hand. Hiwonder fits the leader with HX-10HM servos at a different gear ratio, so its joints should be noticeably easier; note any that are not, as that is a build problem to fix before W11 trusts the leader as a reference.
3. Photograph anything mechanically suspect — a loose horn screw, a cable strained at full reach. W9 and W11 will blame the electronics for mechanical faults you did not write down now.

Done when: leader table complete and the mechanical survey is in `docs/arm-bringup.md` with photos.

S5 · Saturday: the bus on the scope (4 h)

This is the session that feeds W3, W5 and W7, so take the captures carefully and save the raw waveform files, not just screenshots.

1. Set up the scope on the **follower** bus at the **last servo in the chain** — the far end is where reflections and loading are worst, and it is the point W7's series-resistor choice has to satisfy.
2. With teleop running, capture one complete packet. Save the waveform. From it, measure:
 - **Bit period.** Expect 1.00 μ s at 1 Mbps. If it is off by more than a percent or two, the servo's baud is not what you set.
 - **Rise and fall times**, 10–90 %, at the far end.
 - **Ringing amplitude and duration** on the fastest edge.
 - **Idle level**, as a voltage, and how long the line sits idle between master and servo transmissions. The voltage matters more than it looks: PYNQ-Z2 PL pins are 3.3 V and not 5 V tolerant, so if the HX bus idles near 5 V, W6's breadboard and W7's buffers both need a level translator. Record it prominently.
3. Zoom on the **turnaround**: the gap between the master's last stop bit and the first bit of the servo's reply. Measure it. This is the timing your RTL's direction-pin logic has to respect in W5, and the number you compare its simulation against.
4. Repeat all of it on the **leader** bus. The leader chain may be shorter or differently loaded; do not assume the numbers transfer.
5. Capture one **collision or error case** deliberately if you can produce one safely — for instance, pinging an ID that does not exist and observing the timeout gap. Knowing what "no reply" looks like on the wire saves an hour in W6.
6. Write `docs/servo-power-and-bus-report.md`, two pages:
 - Current tables, both arms, with the stall caveat and the measured step duration.
 - Scope shots: one packet, one turnaround zoom, one edge with ringing, per bus.
 - A short "what this constrains" section naming the W4 and W7 decisions each number feeds. That section is what makes it a report rather than a log.

Done when: the report exists with the "what this constrains" section written.

Measurements to record

Quantity	How	Feeds
Follower current: quiescent, idle, gravity-hold, move, stall	Bench supply, 10 s each	W4 power budget, W7 fuse and load switch
Current step duration Δt	Scope across shunt	W7 bulk capacitance sizing
Leader current: idle, back-driven	Bench supply	W7 leader rail switch and shunt
Which protection ended each stall, and when	Scope + ServoStudio	W12 defence-in-depth
Factory config and protection thresholds, all 12 servos	ServoStudio screenshots	W3 spec, W12
Bit period, rise/fall, ringing, both buses, far end	Scope	W5 baud generator, W7 series resistors
Master-to-servo turnaround gap	Scope zoom	W5 direction-pin timing
Bus idle-high voltage	Scope, DC-coupled	W6 and W7 signal-level decision
Backlash, sag, back-drive per joint	Hand survey	W11 characterisation baseline

Deliverables

-  docs/arm-bringup.md — ID map with model and firmware version, joint labels, photos, mechanical survey
-  docs/servo-config/ — every ServoStudio config page, all twelve servos
-  docs/servo-notes.md — how Hiwonder's LeRobot integration drives the HX servos
-  docs/servo-power-and-bus-report.md — two pages, tables, scope shots, constraints section
-  sw/calib/ — calibration files for both arms, committed
-  A two-minute LeRobot dataset, path recorded

Failure modes

Symptom	Likely cause	Fix
Garbled replies, random IDs answer	Two servos share an ID	Re-ID one servo at a time, off the chain
ID change does not survive power cycle	Write not committed to persistent storage	Check the protocol document's lock or save sequence; document it
ServoStudio scan finds nothing	Jumper not on Servo + USB, wrong baud, or no servo power	Check the jumper first, then 1,000,000 baud, then the supply
Stall ends before the supply limit	The servo's own protection tripped	Not a fault — record the time; it is data for W12
Follower joint fights itself in teleop	Calibration range wrong or joint sign flipped	Recalibrate that joint; check the ID map
Supply trips into current limit on power-up	Inrush into servo bulk caps	Raise limit slightly, or ramp the supply; note the inrush for W7
Bus edges look fine at the adapter, awful at the far end	Chain loading and reflections	This is the finding — it justifies W7's drivers
A servo runs hot after stall testing	Too long, too many repeats	Stop; let it cool; use the spare if it degrades

W03 · The HX servo protocol, from logic analyser to your own driver (12 h)

Competency: writing an interface spec others can build against, and measuring before asserting. **Prerequisites:** W2 complete; logic analyser with UART decode at ≥ 8 MS/s. **Sources, in order of authority:** (1) what you capture on the wire; (2) Hiwonder's *Magnetic Encoder Bus Servo Communication Protocol* document, saved in W0; (3) Hiwonder's LeRobot integration code, found in W2. Feetech material is **not** a source — the instruction names overlap, the register map does not have to. **Why clean-room:** you are going to implement this protocol in RTL in W5. You cannot debug RTL against a library you do not understand. The driver is not the deliverable — the *understanding*, captured as a spec, is.

S1 · Capture and dissect real traffic (2 h)

1. Probe both buses with the logic analyser: one channel per bus, common ground, sample at ≥ 8 MS/s. Configure a UART decoder at 1 000 000 baud, 8N1, on each channel.
2. Start LeRobot teleop and capture 2–3 seconds. Stop and save the raw capture into `docs/captures/` — committed, not left on the laptop.
3. Open Hiwonder's protocol document next to the capture. Find one complete transaction and decode it **by hand**, byte by byte, into a table. The table below is the layout to *expect* if the HX protocol follows the Dynamixel 1.0 / Feetech lineage its instruction names (Write, SyncWrite, RegWrite) suggest — treat every row as a hypothesis until both the document and the capture agree:

Offset	Byte	Meaning
0–1	FF FF ?	Header — confirm
2	ID	Servo ID; confirm the broadcast ID
3	LEN	Parameter count + 2
4	INSTR	Instruction
5...	PARAMS	Instruction-specific
last	CHK	Checksum — confirm the rule; $\sim(\text{ID}+\text{LEN}+\text{INSTR}+\Sigma\text{PARAMS}) \ \& \ 0\text{xFF}$ is the lineage default

If the header, length rule or checksum differ from this, nothing about the method changes — only the constants in S2's driver do. Write down which rows matched and which did not; that list is the first section of your spec.

- Verify the checksum arithmetic yourself on that packet with a calculator. If it does not come out, your byte extraction is wrong — fix it before moving on, because every later assumption rests on this.
- Identify each instruction present in the capture and tabulate it with its code **from Hiwonder's document**: Ping, Read, Write, RegWrite, Action, SyncWrite, and any others you see (a SyncRead, if Hiwonder implements one, would change W4's budget considerably — look for it). For each, note the parameter layout as observed.
- Answer the question the capture is really for: **what does the leader side actually do?** Expect reads only. Confirm it, and note the read cadence and which registers it reads. This decides whether W5's leader bus engine needs a transmit path beyond read requests.
- Measure from the capture, not from the datasheet: the master-to-servo turnaround gap, the servo's response delay, and the gap between consecutive transactions. Three numbers.

Done when: one packet is hand-decoded in `docs/hx-bus-spec.md`, every row of the frame table is marked confirmed or corrected, and the instruction inventory is written with codes cited to the Hiwonder document.

S2 · The driver core: framing, ping, read, write (2 h)

Write `sw/hx_min/bus.py`. Keep it dependency-light — this code gets reused in W6 against the PL backend, so no LeRobot imports. The constants at the top are the lineage defaults; **replace each one with the value from Hiwonder's document** and cite the section in a comment. If S1 found a different header or checksum, change `frame()` and `_read_status()` to match — the rest of the class does not care.

```

import serial

# Confirm every value against Hiwonder's protocol document (cite section numbers here).
HEADER      = b"\xff\xff"
BROADCAST   = 0xFE
PING, READ, WRITE, SYNC_WRITE = 0x01, 0x02, 0x03, 0x83

class BusError(Exception): pass
class BusTimeout(BusError): pass
class ChecksumError(BusError): pass
class StatusError(BusError):
    def __init__(self, sid, err): super().__init__(f"servo {sid} error 0x{err:02x}"); self.err = err

def checksum(body: bytes) -> int:
    return (~sum(body)) & 0xFF

class HxBus:
    def __init__(self, port, baud=1_000_000, timeout=0.005):
        self.ser = serial.Serial(port, baud, timeout=timeout)
        self.stats = {"tx": 0, "rx": 0, "timeout": 0, "checksum": 0}

    def frame(self, sid, instr, params=b"") -> bytes:
        body = bytes((sid, len(params) + 2, instr)) + params
        return HEADER + body + bytes((checksum(body),))

    def _read_status(self, nparams):
        need = 6 + nparams                # FF FF ID LEN ERR params CHK
        buf = self.ser.read(need)
        if len(buf) < need:
            self.stats["timeout"] += 1
            raise BusTimeout(f"got {len(buf)} of {need} bytes")
        if buf[0:2] != HEADER:
            raise BusError(f"bad header {buf[0:2].hex()}")
        body = buf[2:-1]
        if checksum(body) != buf[-1]:
            self.stats["checksum"] += 1
            raise ChecksumError(buf.hex())
        sid, _len, err = body[0], body[1], body[2]
        if err:
            raise StatusError(sid, err)
        self.stats["rx"] += 1
        return sid, body[3:]

    def txrx(self, sid, instr, params=b"", nparams=0):
        self.ser.reset_input_buffer()
        self.ser.write(self.frame(sid, instr, params))
        self.stats["tx"] += 1
        if sid == BROADCAST:
            return None
        return self._read_status(nparams)

    def ping(self, sid):
        try:
            self.txrx(sid, PING, b"", 0)
            return True
        except BusTimeout:
            return False

    def read(self, sid, addr, n):
        _, data = self.txrx(sid, READ, bytes((addr, n)), n)
        return data

    def write(self, sid, addr, data: bytes):
        self.txrx(sid, WRITE, bytes((addr,)) + data, 0)

```

1. Type it in rather than pasting, so the framing rules land.

2. Test `ping` against all six IDs on the follower and confirm six `True`. Then ping an unused ID and confirm `False` *within* the timeout, not after a long hang — the timeout path is as important as the success path.
3. Read present position — at the address the Hiwonder document gives — from ID 1 and sanity-check against the physical pose. Move the joint by hand with torque off and read again. Hiwonder defines 0–4095 as 0–360°, with 2047 at centre, so a joint near its middle should read near 2047.

Done when: all six ping, and position reads track physical motion.

S3 · Registers, byte order, sign, sync-write, and the memory map (2 h)

1. **Settle the byte order empirically.** Move joint 1 to roughly mid-range by hand, read the two position bytes, and see which interpretation gives a sensible number near 2047 that changes smoothly as you move the joint:

```
POS = 0x00 # REPLACE with the present-position address from Hiwonder's document
raw = bus.read(1, POS, 2)
print("LE:", int.from_bytes(raw, "little"), " BE:", int.from_bytes(raw, "big"))
```

Move the joint to three positions and check which interpretation is monotonic. Write the answer into the spec as a *measured* fact with the three readings as evidence, alongside what the document claims.

2. **Settle the sign.** HX servos support multi-turn absolute positions of roughly ± 30719 (about ± 7.5 turns), so position is a **signed** quantity, not an unsigned 12-bit one. Read it with `signed=True`, then answer three questions in the spec with evidence:
 - Does a joint moved past 4095 read 4096 and up, or wrap to 0?
 - What does a joint read after a power cycle? Hiwonder documents that the **turn count is cleared on power loss** and only the absolute position within one turn is kept — confirm it.
 - Which joint could ever cross the 0/4095 seam? **Wrist roll** is the candidate. If it can, W10's mapping block must handle the wrap, and you want to know that now rather than in W10.
3. Build the register table **from Hiwonder's document**, then verify each row on the hardware. Leave the address column blank until you have the document open — do not fill it from memory or from a Feitech table:

Addr	Name (Hiwonder's)	Bytes	Signed?	Verified how
	Torque enable			Write off, joint goes limp
	Operating mode			Reads Position mode
	Goal position			Write, joint moves there
	Goal speed / acceleration / time			Change it, move time changes
	Present position			Tracks hand motion
	Present speed			Non-zero during a move
	Present load or current			Rises under gravity load
	Present voltage			Matches the supply in 0.1 V units
	Present temperature			Matches the case, roughly
	Offset, min/max angle			Match the W2 ServoStudio screenshots
	Current max and hold time			Match the W2 screenshots
	Over-temperature, under/over-voltage thresholds			Match the W2 screenshots
	ID, baud			Match ServoStudio
	Lock / save-to-flash control			Needed to persist a change

The rows that match the ServoStudio screenshots from W2 are a free cross-check: if your read of the over-voltage threshold disagrees with what ServoStudio displayed, you have the address, width or scaling wrong. 4. Note the write sequence needed to change persistent settings, from the document and from a logic-analyser capture of ServoStudio doing it. Then compare the same addresses on an **HX-10HM** (leader) and an **HX-30HM** (follower): differences between the two models are a real finding for the spec. 5. Implement sync-write, the instruction the control loop actually depends on: 4. Implement sync-write, the instruction the control loop actually depends on:

```
def sync_write(self, addr, per_servo: dict, width=2):
    """per_servo: {id: value}. One packet, no replies."""
    params = bytearray((addr, width))
    for sid, val in per_servo.items():
        params.append(sid)
        params += int(val).to_bytes(width, "little", signed=True) # order from S3.1, sign from S3.2
    self.txrx(BROADCAST, SYNC_WRITE, bytes(params))
```

Check the parameter layout against Hiwonder's SyncWrite description — this layout (start address, data width, then ID + data per servo) is the lineage default. With it, the length arithmetic is: for N servos of W bytes each, the packet is $2 + 1 + 1 + 1 + 1 + 1 + N \cdot (W+1) + 1$ bytes; for N=6, W=2 that is **26 bytes**. If Hiwonder's goal-position write also carries speed or time per servo, W grows and so does the packet — recompute with your real W and write that number down, because W4's bus budget uses it. 6. Sync-write a small coordinated move to all six follower joints and watch them move together. Compare on the scope: one packet of the length you computed, not six separate writes. 7. Write unit tests in `sw/hx_min/test_bus.py` that need no hardware — feed known byte strings through the framing and parsing functions:

```
def test_checksum_known_packet():
    b = HxBus.__new__(HxBus)
    # Framing check only: 0x38 is an arbitrary address, not an HX register.
    # Body 01 04 02 38 02 sums to 0x41, so the lineage checksum is ~0x41 = 0xBE.
    assert b.frame(1, READ, bytes((0x38, 2))).hex() == "ffff0104023802be"

def test_status_checksum_error_raises(): ...
def test_short_read_raises_timeout(): ...
```

Recompute the expected hex from your own arithmetic and from the header and checksum rule you confirmed in S1; if they differ from the lineage defaults, so does this string. Deriving it is the exercise.

Done when: sync-write moves six joints in one packet, `pytest sw/hx_min` is green, byte order and sign are settled with evidence, and every register row is cited to the Hiwonder document and verified on hardware.

S4 · Latency of the USB path, measured properly (2 h)

This is the baseline the whole project is judged against. Measure it well.

1. Single-register round-trip, 1000 samples:

```
import time, numpy as np, matplotlib.pyplot as plt

lat = []
for _ in range(1000):
    t0 = time.perf_counter()
    bus.read(1, 56, 2)
    lat.append((time.perf_counter() - t0) * 1e6)    # μs

a = np.array(lat)
print(f"median {np.median(a):.0f} μs  p99 {np.percentile(a,99):.0f} μs  max {a.max():.0f} μs")
plt.hist(a, bins=100); plt.xlabel("round trip (μs)"); plt.ylabel("count")
plt.title("Present_Position read, USB path, n=1000")
plt.savefig("docs/figs/usb-read-latency.png", dpi=150)
np.savetxt("docs/data/usb-read-latency.csv", a, delimiter=",")
```

- Report **median, p99 and max**, not the mean. The distribution is the interesting part: the median is the protocol, the tail is the USB stack and the OS scheduler. Say so explicitly in the write-up — that sentence is the difference between a measurement and an insight.
- Repeat with the six-servo read loop and with the 26-byte sync-write, so you have per-transaction numbers for each of the three things the control loop does.
- Compute the **theoretical floor** from the wire alone: at 1 Mbps a byte is 10 μs, so an 8-byte request plus a 12-byte reply is 200 μs plus turnaround. Compare with your measured median. The gap is everything the USB path adds — and it is the number that justifies the whole project. Put the comparison in a two-row table.
- Note bytes on the wire per servo per loop iteration; W4 needs it.

Done when: the histogram is saved, the CSV is committed, and the measured-vs-theoretical table is written.

S5 · Saturday: end-to-end teleop latency, then write the spec (4 h)

- The measurement that matters.** With LeRobot teleop running, put scope channel 1 on the leader bus and channel 2 on the follower bus. Trigger on activity on the leader bus.
- Tap a leader joint sharply — a step input, not a smooth motion. Capture the interval from the leader bus transaction that *first reports the moved position* to the follower bus write that *commands the new goal*. That interval is your USB-path teleop latency.
- Repeat 20 times and record the spread. One capture is an anecdote; twenty gives you a median and a worst case. Save three representative waveforms.

4. Be honest about what this includes: the leader read, the BusLinker's CH341 USB round trip through the Linux USB stack, Python's loop, the follower write, and the wait for the next scheduled loop iteration. Decompose it if you can — you have the per-transaction numbers from S4, so subtract them and see what is left. What is left is the software loop, and it is usually most of it. That decomposition is the strongest single paragraph in the report.
5. Write `docs/hx-bus-spec.md v1`. Hiwonder's document is your source, but **yours is the one W5 builds to**, so write it as an implementer's contract: every fact cites either a section of Hiwonder's document or one of your capture files, and every place where your capture disagrees with the document is called out explicitly. Sections:
 - **Scope and physical layer** — half-duplex TTL, 3.3 V, 1 Mbps, 8N1, idle high, daisy-chained.
 - **Frame format** — the byte table from S1, with checksum defined as an equation.
 - **Instructions** — one subsection each for PING, READ, WRITE, SYNC_WRITE: parameter layout, reply or no reply, an annotated hex example.
 - **Register map** — the verified table from S3, with byte order and sign stated and evidenced, the multi-turn and power-cycle behaviour, and HX-10HM vs HX-30HM differences called out.
 - **Timing** — bit period, byte time, turnaround requirement, servo response delay, master timeout recommendation. Every number sourced to your own capture with the file name.
 - **Error handling** — status error bits, what a bad checksum looks like, what a missing reply looks like on the wire, recommended retry policy.
 - **Worked budget** — your measured sync-write length and the per-servo read, in bytes and microseconds. This is the bridge to W4.
 - **Deviations from the vendor document** — a short list. It is the most useful page in the spec for anyone who comes after you.
6. Write the one-page latency baseline: USB read histogram, teleop latency with spread, theoretical floor, and one sentence stating the target the PL path must beat.

Done when: someone who has never seen an HX servo could implement a master from your spec alone. Test it by re-reading it cold on Sunday and marking every place you had to remember something that is not written down.

Measurements to record

Quantity	Method	Feeds
Single read round-trip: median, p99, max	1000 samples, <code>perf_counter</code>	W6 comparison, requirements
Six-read loop and sync-write times	Same	W4 loop-rate budget
Theoretical wire floor vs measured	Arithmetic vs measurement	The project's justification
Teleop latency, 20 captures	Two-channel scope	The number W10 must beat
Turnaround and response delay	Logic analyser	W5 direction-pin timing
Bytes on wire per servo per iteration	Capture	W4

Deliverables

-  `sw/hx_min/` — driver plus hardware-free unit tests, green
-  `docs/hx-bus-spec.md v1` — the document W5 builds to, with a deviations section
-  `docs/latency-baseline.md` — one page, histogram, teleop spread, decomposition
-  `docs/captures/` , `docs/data/` , `docs/figs/` — raw evidence committed

Failure modes

Symptom	Likely cause	Fix
Position reads jump wildly	Wrong byte order	Settle it with the three-position test
Position reads jump by about 4096 after a power cycle	Turn count cleared on power loss	Expected — handle it in the mapping, per S3.2
Position reads as a huge positive number near a seam	Read as unsigned	Read signed; positions are multi-turn
A register value disagrees with ServoStudio	Wrong address, width or scale	Recheck against the document; ServoStudio is the cross-check
Checksum always fails	Including header bytes in the sum	Sum from ID to last param only
First read after write always times out	Adapter echoing your own transmission	Flush input after write, or filter the echo
Sync-write moves nothing	LEN arithmetic or broadcast ID wrong	Compare your packet byte-for-byte with a captured one
Latency histogram is bimodal	OS scheduling, USB polling, or buffering in the CH341 driver	The CH341 has no FTDI-style latency-timer setting — record the adapter chip, kernel version and driver as part of the baseline; repeating the measurement on the Windows desktop is a worthwhile comparison
Leader servo rejects a register the follower accepts	HX-10HM vs HX-30HM difference	That is a finding — put it in the spec
A Feetech address "almost works"	Borrowed register map	Stop; use Hiwonder's table only

W04 · Requirements, architecture and your first design review (12 h)

Competency: owning requirements and architecture. This is the week that most separates senior from mid-level, and it is almost entirely writing.

Prerequisites: W1–W3 measurements complete. Every number in this week must trace to one of them; a requirement with no measurement behind it is a wish.

S1 · Numbered requirements with verification methods (2 h)

1. Create `docs/requirements.md` with one table. Every row gets an ID, a statement, a rationale that names its source measurement, and a verification method from **Test** / **Analysis** / **Inspection** / **Demonstration**.
2. Do the **bus budget arithmetic explicitly** in the document, because it is what sets the headline requirement. The worked version below uses the lineage packet layout; **redo it with the framing and SyncWrite width you confirmed in W3**, and keep both versions visible so a reviewer can see the assumption changed:

At 1 Mbps, 8N1: 1 bit = 1 μ s, 1 byte = 10 bits = 10 μ s

Follower write, 6 servos $\times$ 2-byte goal, one SYNC_WRITE:

2 (FF FF) + 1 (FE) + 1 (LEN) + 1 (0x83) + 1 (addr) + 1 (width)
+ 6 $\times$ (1 + 2) + 1 (CHK) = 26 bytes = 260 μ s

Per-servo read of position+speed+load (6 bytes):

request 8 bytes = 80 μ s

reply 12 bytes = 120 μ s

turnaround + response delay $\approx$ 100 μ s $\leftarrow$ replace with your W3 capture

$\rightarrow \approx$ 300 μ s per servo, $\times$ 6 = 1.80 ms

Follower full read+write cycle $\approx$ 2.06 ms $\rightarrow$ ceiling $\approx$ 485 Hz

Leader bus: reads only $\approx$ 1.80 ms $\rightarrow$ ceiling $\approx$ 555 Hz

3. Set the target at **250 Hz** on both buses and state *why* in one sentence: 4.00 ms budget against a 2.06 ms worst case is 52 % bus utilisation, leaving room for retries, a slower response delay than measured, and a seventh transaction without redesign. A requirement whose margin you can justify is a senior requirement; one that sits at 95 % utilisation is a trap you set for yourself.
4. Seed the table with these and expand to $\sim$ 20 rows:

ID	Requirement	Rationale / source	Verif
REQ-001	Follower bus shall complete one read+write cycle at $\geq$ 250 Hz sustained	W3 budget; 52 % utilisation	T
REQ-002	Leader bus shall sample all six positions at $\geq$ 250 Hz	Symmetry with REQ-001	T
REQ-003	Loop period jitter shall be $\leq$ 1 % of the period, p99	Hardware scheduler claim	T
REQ-004	Leader-to-follower teleop latency shall be $\leq$ 1/3 of the W3 USB baseline	W3 measurement	T
REQ-010	The 12 V rail shall supply the W2 measured peak with $\geq$ 30 % margin	W2 current table	A + T
REQ-011	No servo shall see more than 12.6 V under any single supply fault	HX-30HM / HX-10HM rated 9–12.6 V; hazard hits all twelve at once	A + T
REQ-013	Leader and follower buses shall be physically and electrically distinguishable, so the PL cannot command the wrong arm	Both arms are 12 V and use the same connector	I
REQ-012	Rail droop during a worst-case current step shall be $\leq$ 0.5 V	W2 Δt capture	T
REQ-020	E-stop shall remove both rails with no firmware in the path	Safety	I + T
REQ-021	Loss of the PL watchdog shall drop both rails within 100 ms	Safety	T
REQ-022	Loss of leader reads shall freeze the follower within 50 ms, then drop torque	Safety	T
REQ-030	Two UVC cameras shall stream 640 $\times$ 480 MJPEG at 30 fps through one USB 2.0 host port	W1 camera baseline	T
REQ-040	A clean checkout shall rebuild the overlay with one command	Reproducibility	D

5. For every requirement, ask: *how would I fail this on purpose?* If you cannot answer, the verification method is not specific enough yet.

Done when: every row has a source and a verification method, and the bus arithmetic is shown rather than asserted.

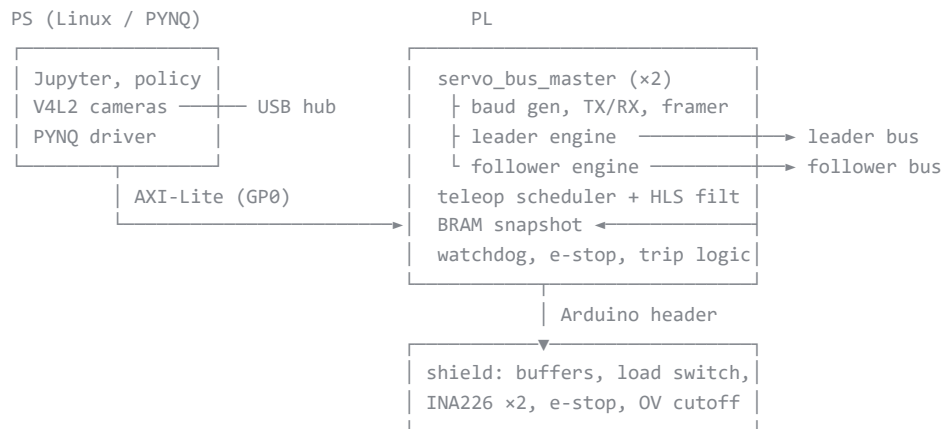
S2 · Power and camera budgets (2 h)

- Power budget**, in a table built from W2's measured currents — not datasheet maxima, which will oversize everything:
 - Follower 12 V rail: quiescent, realistic worst-case simultaneous load, absolute stall case. State which one you design to and why. Six simultaneous stalls is a fault condition the current limit handles, not an operating point the fuse must pass.
 - Fuse sizing: $I_{\text{fuse}} \geq 1.5 \times I_{\text{continuous_max}}$, then derate for ambient — most fuses lose about a quarter of their rating by 50 °C. Show both steps.
 - Bulk capacitance: $C = I \cdot \Delta t / \Delta V$, with I and Δt from W2's scope capture and ΔV from REQ-012. Compute it, then sanity-check the answer against what you can physically fit; if the number is implausibly large, the honest move is to relax ΔV and say so, not to quietly pick a nicer capacitor.
 - Leader 12 V rail: the same supply as the follower, behind its own load switch and shunt, sized from W2's leader table. Because both arms share one input, the fuse and input connector carry the **sum** of both arms' realistic worst case — state that sum explicitly.
 - Over-voltage.** Every servo in the kit is rated to 12.6 V, only 5 % above nominal. A TVS diode cannot protect that: a part with enough standoff to sit quietly at 12.6 V clamps well above 20 V. So the budget needs an **active cutoff** that opens both rails above a threshold between 12.6 V and your supply's worst-case tolerance — around 13.0 V — with hysteresis. Write the threshold, its tolerance stack (reference accuracy plus divider resistor tolerance), and the resulting worst-case trip range. If the range overlaps 12.6 V on the high side or your normal supply on the low side, the design does not work yet — tighten the tolerances before W7.
 - Note that the servos also have their own over-voltage protection, configurable per servo. Record why you are not relying on it alone: it is firmware, it acts after the voltage has already reached the part, and it protects the servo's logic but not necessarily its driver stage. That reasoning is a decision record in S4.
 - Connector and trace current ratings, with the derating you applied.
- Camera budget.** The Zynq-7020 PS has one USB 2.0 host port: 480 Mbit/s raw, realistically 280–320 Mbit/s of payload. The kit gives you a 480p wrist camera and a **1080p** external camera, so do the arithmetic for the 1080p case too.
 - Raw YUY2 at 640×480×30 is $640 \cdot 480 \cdot 2 \cdot 30 \cdot 8 = 147$ Mbit/s **per camera**. Two of those is 295 Mbit/s — nominally close, practically not survivable alongside everything else, and it lands entirely on the A9 to memcp.
 - MJPEG at the same resolution compresses roughly 10:1, so ~15 Mbit/s per camera. Two fit comfortably.
 - At 1080p the raw numbers are impossible — $1920 \cdot 1080 \cdot 2 \cdot 30 \cdot 8 \approx 995$ Mbit/s for YUY2 — and even MJPEG leaves the A9 decoding 1080p frames, which your W1 CPU-load measurement will likely rule out at 30 fps.
 - Conclusion: **MJPEG, both cameras, powered hub, external camera at 640×480 on the board**, or the external camera on the laptop when you need full resolution. Write the arithmetic and the conclusion, plus the decode cost you measured in W1.
 - Then state what *would* justify a PL video path: resolution or frame rate beyond what the A9 can decode, or a per-pixel operation in the loop. W10's stretch tests exactly this, so make the criterion measurable now.

Done when: docs/requirements.md contains both budgets with arithmetic shown, and the camera decision is recorded with its threshold for revisiting.

S3 · Architecture and the interface control document (2 h)

- Draw the block diagram — one page, in a text-diffable source format (Mermaid, Graphviz or draw.io XML) so it can be reviewed and versioned:



2. Mark on the diagram: every clock domain, every place data crosses PS↔PL, and every path that must work **without** the PS. That last annotation is the architectural claim of the whole project, so make it visible.
3. Write docs/icd.md v0.1 — the register map W5 implements and W6 binds to. Per-bus instances at a stride so the second bus is an offset, not a redesign:

Offset	Name	Access	Description
0x00	CTRL	RW	bit0 enable, bit1 soft reset, bit2 TX start
0x04	STATUS	RO	bit0 busy, bit1 rx_valid, bit2 timeout, bit3 checksum_err, bit4 tx_full, bit5 rx_empty
0x08	CONFIG	RW	baud divisor (clocks per bit), turnaround hold, reply timeout
0x0C	TX_DATA	WO	push one byte into TX FIFO
0x10	RX_DATA	RO	pop one byte from RX FIFO
0x14	FIFO_LVL	RO	{rx_level[15:0], tx_level[15:0]}
0x18	ERR_CNT	RO	{checksum[15:0], timeout[15:0]}, clear on write
0x1C	IRQ	RW	enable and latched status
0x1000	bus 1 base	—	same map, second instance

Reserve 0x2000 for the W10 teleop scheduler and 0x3000 for the BRAM snapshot now, so the map does not have to move later.

4. Add to the ICD: the **Arduino-header pin assignment** (TX, RX, DIR per bus; I2C for the INA226s; e-stop sense; watchdog output), taken from the PYNQ-Z2 master XDC by name, and the **connector pinouts**. Both arms use the same 5264-3P connector at the same voltage, so a swap no longer destroys anything — but it does send follower commands to the leader. Record how REQ-013 is met (distinct connector keying or orientation, colour, silkscreen, and net names) and accept, in writing, whichever residual risk remains.

Done when: docs/architecture.md and docs/icd.md v0.1 exist, and the register map is complete enough that you could hand it to another engineer to implement.

S4 · Risks, schedule and decision records (2 h)

1. docs/risks.md , one row per risk: description, likelihood, impact, owner, mitigation, trigger, and the response. Top risk is PCB fab and assembly lead time. Add: toolchain skew, USB bandwidth, wrong rail to the leader, RTL overrun, bench time squeezed, servo damaged in stall testing.

2. Draw the schedule with **W8 fab release on the critical path**. Mark the float each downstream week has. The holiday buffer after W8 already absorbs a normal two-week fab time. Identify what you would do if the boards were two weeks later still — that answer is the mitigation, and it is the swap of W9 and W10.
3. Write the decision records. Each is half a page, using the W0 template. At minimum:
 - 0003-four-layer-stackup.md — why four layers rather than two.
 - 0004-ina226-vs-xadc.md — I2C digital sense vs the Zynq's own ADC. Cover resolution, isolation from the PL design, sample rate, and what each costs you in W12's trip-time requirement.
 - 0005-watchdog-in-pl.md — why the hold-off is in PL and hardware rather than software.
 - 0012-active-over-voltage-cutoff.md — why an active cutoff rather than a TVS alone or the servos' own protection, and eFuse versus a discrete comparator-and-FET design.
 - 0006-cameras-on-ps.md — the S2 bandwidth arithmetic, with the threshold that would change the answer.
 - 0007-single-clock-domain.md — keeping the bus logic in the AXI clock domain to avoid a CDC, and what you would do instead if you needed a separate baud domain.
4. Each record must name at least one option you **rejected** and why. A decision record with one option is a description.

Done when: five decision records exist, each with a rejected alternative.

S5 · Saturday: run a real review on yourself (4 h)

Treat this as a rehearsal for W13. The purpose is to find problems, not to approve slides.

1. Write the review checklist first, before looking at your own work, so you cannot bend it:
 - Is every requirement verifiable, and does each name its method?
 - Does every number trace to a measurement with a file name?
 - Is the bus budget arithmetic correct if I redo it myself?
 - Does any single failure destroy hardware? (Trace the over-voltage path from the barrel jack to all twelve servos.)
 - What happens on bitstream reload, mid-move? On PS crash? On e-stop during a stall?
 - Is the register map implementable as written, with no "obvious" behaviour left unstated?
 - Can a stranger rebuild the overlay from the repo?
 - What is on the critical path, and what is its float?
2. Go through the checklist in one pass, writing findings as you go. Do not fix anything during the pass — reviewers who fix as they read stop reviewing.
3. Classify each finding: **blocker / major / minor / question**. Assign yourself an action and a due week for each.
4. Now fix the blockers. Leave the rest as tracked actions.
5. Write docs/reviews/2026-w04-architecture-review.md : attendees (you), scope, checklist used, findings with classification, actions with owners and dates, and the exit decision. Real minutes, real format.
6. **Rehearse the defence.** Close the laptop and state out loud, from memory: the loop-rate target and its margin, the teleop latency target and its baseline, the camera bandwidth conclusion, and the power headline. If you cannot, you do not own the architecture yet — reread and repeat. This is exactly what Gate 1 tests.

Done when: review minutes exist with classified findings, and you can recite the four headline numbers cold.

Deliverables

- docs/requirements.md — ~20 numbered, verifiable requirements with shown arithmetic
- docs/architecture.md — diagram in a diffable format, clock domains and PS-free paths marked
- docs/icd.md v0.1 — register map, pin assignment, connector pinouts and keying
- docs/risks.md — with triggers and responses
- docs/decisions/0003..0007 — each with a rejected option
- docs/reviews/2026-w04-architecture-review.md — classified findings, tracked actions

Failure modes

Symptom	Likely cause	Fix
A requirement you cannot test	Written as an aspiration	Rewrite with a number and a method, or delete it
Bus budget leaves no margin	Target set from the ceiling	Design to ~50 % utilisation and say why
Decision records read like descriptions	No rejected option	Add the alternative and the reason it lost
Review finds nothing	You reviewed with the intent to pass	Use the checklist literally; a clean first review is a bad sign
Diagram is a PNG only	Not reviewable, not diffable	Keep a text source in the repo

Gate 1. Architecture approved by your own review. You can state the loop-rate, teleop latency, camera bandwidth and power targets from memory and defend each with a measurement from W1, W2 or W3.

Month 2 · Build: RTL, verification, and a board of your own

W05 · RTL: half-duplex UART and the servo packet engine (14 h)

Competency: RTL plus verification. A senior engineer is asked "how do you know it works?" and answers with a testbench and coverage, not a demo. **Budget note:** 14 h — five standard sessions plus one extra 2 h block. **Build to the spec.** `docs/hx-bus-spec.md` v1 is the contract. If you find yourself guessing at behaviour, the spec has a hole; fix the spec first, then the RTL.

Architecture decision to make before you type. Two options for what crosses AXI:

- **Raw byte FIFO.** The PS pushes framed bytes; the PL only handles direction and timing. Simple RTL, but W10's hardware teleop loop would then need its own packet builder, duplicating logic.
- **Descriptor interface.** The PS writes `{id, instr, addr, len}` plus payload; the PL assembles the header, computes the checksum, and parses the reply. More RTL now, but W10's scheduler drives the *same* interface with no PS involved.

Choose the descriptor interface and write it up as `docs/decisions/0008-packet-engine-in-pl.md`. The reason is W10: an interface that only the PS can drive cannot become a hardware loop later. Noticing that in W5 rather than W10 is the senior move.

S1 · Baud generator and transmitter with direction control (2 h)

Start from your R4 transmitter. You wrote `uart_tx.sv` with this interface in R4 S3 and have it working on hardware. Promote it into `rtl/servo_bus_master/`, compare it with the listing below, and spend this session on what is new: the runtime baud divisor from `CONFIG` and the direction control. The time you save goes into S5's testbench, which is the part of this week that matters most.

1. Create `rtl/servo_bus_master/` with `uart_tx.sv`, `uart_rx.sv`, `packet_engine.sv`, `servo_bus.sv` (the per-bus top), and `servo_bus_master.sv` (the AXI-Lite wrapper instantiating N buses).
2. Baud generation: 100 MHz PL clock, 1 Mbps → **100 clocks per bit, exactly**. Make it a runtime parameter from `CONFIG` rather than a compile-time constant, so W9 can drop the baud to debug a marginal bus without a rebuild.
3. Transmitter:

```

module uart_tx (
    input logic      clk, rstn,
    input logic [15:0] clks_per_bit,
    input logic [7:0] data,
    input logic      valid,
    output logic      ready,
    output logic      tx,
    output logic      active // high from start bit to end of stop bit
);
typedef enum logic [1:0] {IDLE, START, DATA, STOP} state_t;
state_t state;
logic [15:0] cnt;
logic [2:0] bit_idx;
logic [7:0] sr;

assign ready = (state == IDLE);

always_ff @(posedge clk) begin
    if (!rstn) begin
        state <= IDLE; tx <= 1'b1; active <= 1'b0; cnt <= '0; bit_idx <= '0;
    end else begin
        case (state)
            IDLE: begin
                tx <= 1'b1; active <= 1'b0;
                if (valid) begin
                    sr <= data; cnt <= '0; bit_idx <= '0;
                    tx <= 1'b0; active <= 1'b1; state <= START;
                end
            end
            START: if (cnt == clks_per_bit - 1) begin cnt <= '0; tx <= sr[0]; state <= DATA; end
                    else cnt <= cnt + 1;
            DATA: if (cnt == clks_per_bit - 1) begin
                        cnt <= '0;
                        if (bit_idx == 3'd7) begin tx <= 1'b1; state <= STOP; end
                        else begin sr <= {1'b0, sr[7:1]}; bit_idx <= bit_idx + 1; tx <= sr[1]; end
                    end else cnt <= cnt + 1;
            STOP: if (cnt == clks_per_bit - 1) begin cnt <= '0; active <= 1'b0; state <= IDLE; end
                   else cnt <= cnt + 1;
        endcase
    end
end
endmodule

```

4. Direction control is the part that bites. `DIR` must be asserted before the start bit and held until the stop bit is **fully complete**, plus a configurable hold. Release too early and the last bit is truncated; release too late and you clip the servo's reply. Make the hold a `CONFIG` field in bit times, defaulting to the value you measured in W3.
5. Simulate `uart_tx` alone in a trivial testbench and confirm one byte on the wire: start low, eight data bits LSB first, stop high, `active` spanning exactly $10 \times \text{clks_per_bit}$.

Done when: one byte transmits correctly in simulation, with `active` measured in the waveform rather than assumed.

S2 · Receiver with mid-bit majority sampling (2 h)

Start from your R5 receiver, which already has the synchroniser, start-bit check and framing error. This session upgrades its single centre sample to a three-sample majority vote and wires out the error counters. You also measured its baud tolerance in R5 S1 — rerun that test after the change and compare.

1. Two-flop synchroniser on the incoming RX line first — it comes from a pad and is asynchronous to your clock. This is a real CDC and you will be asked about it in W13, so comment it as one.
2. Start-bit detection on the falling edge, then wait $\text{clks_per_bit} / 2$ to land in the centre of the start bit and confirm it is still low. A start bit that has vanished by mid-bit is noise, and rejecting it there saves you a garbage byte.
3. Sample each data bit at its centre by **majority vote of three samples** at `centre-1`, `centre`, `centre+1` clocks. At 100 clocks per bit those three samples span 30 ns — narrow enough to be one logical instant, wide enough to reject a single glitch.

4. Flag a **framing error** if the stop bit is not high. Count it. W9 and W11 both use the error counters as a bus health metric, so wire them out now rather than adding them later.
5. Simulate against a stimulus that drives bytes at the exact bit period, then at $\pm 2\%$ and $\pm 4\%$ to check your sampling margin. Record the baud tolerance you actually achieve — it belongs in the README and it is a good interview answer.

Done when: bytes round-trip through `uart_tx` → `wire` → `uart_rx` in simulation, and you know your baud tolerance as a number.

S3 · Packet engine: framing, checksum, parsing, timeout (2 h)

1. Transmit path state machine, driven by a descriptor: emit `FF FF`, then `ID`, `LEN`, `INSTR`, payload bytes from the FIFO, then the checksum. Accumulate the checksum in hardware as bytes leave — `sum <= sum + byte` from `ID` onward, emit `~sum`. Doing it as a running sum rather than a separate pass is what makes the engine streamable.
2. Receive path: hunt for `FF FF`, capture `ID`, `LEN`, `ERR`, then `LEN-2` parameter bytes, then verify the checksum. On success push the parameters into the RX FIFO and set `rx_valid`. On failure increment `checksum_err` and discard the frame — **never** push a frame that failed its checksum, because a silently-corrupted position is worse than a missing one.
3. Timeout counter: start on the transmit-complete edge, expire at `CONFIG.reply_timeout`. On expiry set `STATUS.timeout`, increment the counter, return the engine to idle so the next transaction is not blocked. Default the timeout to a few times the W3-measured response delay.
4. Three states to get right, and they are the ones that fail in hardware:
 - **Turnaround:** transmit complete → `DIR` release → receiver armed. The receiver must be armed *before* `DIR` releases or you lose the first bit.
 - **Broadcast:** `ID == 0xFE` expects **no reply**. Do not start the timeout; return to idle immediately. Sync-write is a broadcast, and a needless timeout per loop iteration costs you your loop rate.
 - **Late reply:** a reply arriving after the timeout must not be parsed as the *next* transaction's reply. Flush the receiver on timeout.
5. Write the timing diagram now, by hand, with the turnaround window annotated. It goes in the README and it is what you will hold next to the ILA capture in W6.

Done when: a full descriptor produces a correct byte sequence in simulation, checksum included, and a missing reply times out cleanly.

S4 · AXI-Lite wrapper, FIFOs, and two buses (2 h)

1. Generate the AXI-Lite slave skeleton from Vivado's "Create and Package New IP" flow, then replace its register block with your own implementing `docs/icd.md` v0.1. Keep the generated AXI handshaking; it is correct and it is not the interesting part.
2. Instantiate TX and RX FIFOs — depth 64 bytes each is ample for a six-servo SyncWrite plus margin, even if Hiwonder's goal write carries speed or time per servo (check against the packet length you recorded in W3). Expose levels in `FIFO_LVL` and overflow in `STATUS`.
3. Parameterise the top level for `N_BUS` and instantiate two, at the `0x0000` and `0x1000` offsets from the ICD. The address decode is `addr[12]` selecting the instance — trivially extensible to four buses later, which is worth one sentence in the README.
4. **Keep everything in the AXI clock domain.** The only CDC in the design is the RX input synchroniser from S2. Write that sentence into the README explicitly, because "are there any clock domain crossings?" is interview question 4 and the best answer is a design where the answer is short and provable.
5. Update `docs/icd.md` to v0.2 with anything you had to change while implementing. An ICD that never changes during implementation was not specific enough to begin with.

Done when: both bus instances respond at their offsets in simulation and the ICD matches the RTL.

S5 · Saturday: the behavioural servo model and the test suite (4 h)

The testbench is the deliverable that makes this a senior week. Budget more time here than on the RTL.

1. Write the behavioural HX servo model in Python as a `cocotb` coroutine, `test/cocotb/hx_servo_model.py`, built strictly from `docs/hx-bus-spec.md` — not from the Hiwonder document directly, because the spec is what you are verifying against. It must:

- Receive bytes at the configured baud and parse frames using the same rules as the spec.
- Answer `PING` and `READ` with correctly-framed status packets after a configurable response delay.
- Maintain a small register file so goal-position writes are visible to subsequent present-position reads, with **signed** 16-bit positions and the power-cycle behaviour you measured in W3 (turn count cleared).
- Be *tellable to misbehave*: `model.corrupt_checksum = True`, `model.silent = True`, `model.response_delay_us = 500`, `model.reply_truncated = True`. Fault injection built into the model from the start is what lets you test the error paths at all.

2. Write the tests. Aim for this list, one test per row:

Test	What it proves
<code>test_ping_all_ids</code>	Basic framing, both directions
<code>test_read_present_position</code>	Parameter extraction, endianness
<code>test_sync_write_six</code>	Broadcast path, no spurious timeout, the W3-measured packet length on the wire
<code>test_signed_position</code>	Negative and > 4095 positions survive a write/read round trip
<code>test_checksum_error_counted</code>	Bad frames discarded, counter increments
<code>test_no_reply_timeout</code>	Timeout fires, engine returns to idle
<code>test_late_reply_discarded</code>	Flush on timeout works
<code>test_back_to_back_writes</code>	No lost bytes between transactions
<code>test_tx_fifo_overflow</code>	Overflow flagged, not silently dropped
<code>test_rx_fifo_overflow</code>	Same on the receive side
<code>test_both_buses_concurrent</code>	Two instances, independent, no shared state
<code>test_turnaround_timing</code>	<code>DIR</code> release lands within spec of the last stop bit
<code>test_baud_tolerance</code>	$\pm 2\%$ model baud still decodes

- `test_both_buses_concurrent` is the one that finds real bugs — shared counters, shared FIFO pointers, a state machine accidentally common to both instances. Run both buses with different response delays so their transactions interleave rather than aligning.
- Assert on the **measured turnaround** in `test_turnaround_timing`: sample the simulation time of the last stop bit's end and of the `DIR` falling edge, and assert the gap is within the configured hold. This is the test that would have caught the bug W6 is about to find on real hardware.
- Makefile for the suite:

```
SIM      ?= icarus
TOPLEVEL_LANG ?= verilog
VERILOG_SOURCES = $(shell find ../../rtl/servo_bus_master -name '*.sv')
TOPLEVEL     = servo_bus_master
MODULE       = test_bus_master
COMPILE_ARGS += -g2012
include $(shell cocotb-config --makefiles)/Makefile.sim
```

If Icarus chokes on your SystemVerilog, switch `SIM=verilator` rather than dumbing down the RTL — and record the switch as a decision.

Done when: every test in the table passes, and at least one of them failed first and was fixed. A suite that passes on the first run was written to the implementation rather than to the spec.

S6 · Extra block: coverage, lint, documentation (2 h)

1. Functional coverage, tracked simply: a Python dict in the testbench counting which instructions and which error paths each test exercised, dumped at the end of the run. Report **instruction coverage** (of PING/READ/WRITE/SYNC_WRITE) and **error-path coverage** (checksum, timeout, framing, overflow) as percentages with the denominators stated. Do not quote a coverage number whose denominator you cannot name.
2. Lint with Verilator:

```
verilator --lint-only -Wall -Wno-fatal rtl/servo_bus_master/*.sv
```

Fix every warning or write a one-line justification next to the waiver. Latch inference, incomplete sensitivity and width-mismatch warnings are never to be waived; unused-signal warnings often can be. 3. Write `rtl/servo_bus_master/README.md`: block diagram, the hand-drawn timing diagram with the turnaround annotated, the register map, the CDC statement, the baud tolerance, how to run the tests, and the coverage summary. 4. Record the simulated turnaround time — last stop bit to `DIR` release — as a number. W6 compares the ILA capture against it.

Done when: lint is clean or justified, coverage is reported with denominators, and the README could stand alone.

Measurements to record

Quantity	Where from	Feeds
Simulated turnaround, stop bit → <code>DIR</code> release	Waveform assertion	W6 ILA comparison
Baud tolerance (% error still decoded)	Sweep test	README, interview
Instruction coverage, error-path coverage	Testbench counters	W13 review
Test count, pass rate	cocotb output	W13 review
Lint warnings: fixed vs waived	Verilator	W13 review

Deliverables

- `rtl/servo_bus_master/` — parameterised for N buses, two instantiated
- `test/cocotb/` — model with fault injection, twelve tests, Makefile
- Coverage summary with denominators
- README with timing diagram and CDC statement
- `docs/decisions/0008-packet-engine-in-pl.md`
- `docs/icd.md` v0.2

Failure modes

Symptom	Likely cause	Fix
Last transmitted bit truncated	<code>DIR</code> released during the stop bit	Hold <code>DIR</code> through stop bit plus configured hold
First reply byte always lost	Receiver armed after <code>DIR</code> release	Arm the receiver first
Sync-write costs a full timeout each loop	Broadcast waiting for a reply	Special-case <code>ID == 0xFE</code>
Second bus behaves oddly when the first is busy	Shared state between instances	<code>test_both_buses_concurrent</code> should have caught it — fix the test too
Icarus rejects the RTL	SystemVerilog constructs outside <code>-g2012</code>	Switch to Verilator; record the decision
Coverage reported as a bare percentage	No denominator	State what was covered out of what

W06 · Overlay integration, constraints, timing and on-chip debug (12 h)

Competency: FPGA integration, timing closure, and choosing the right instrument for each layer of the problem. **Prerequisites:** W5 suite green; breadboard buffers to hand. **Expect to find a bug.** The direction-turnaround bug is the classic one and it survives simulation because your model is too polite. Finding it here is the plan working, not the plan failing.

S1 · Package the IP and constrain the pins (2 h)

- Package `servo_bus_master` as an IP with a VLNV you control — `yourname:user:servo_bus_master:1.0`. Put the packaging in a Tcl script too; the no-GUI-clicks rule still applies.
- Extend `overlay/hello/build.tcl` into `overlay/v0.1/build.tcl`: same PS7 setup, add the IP repo path, instantiate the IP, `apply_bd_automation` to hang it off `M_AXI_GP0`.
- Constrain six pins in the XDC — `tx`, `rx`, `dir` for each bus. **Take the pin names from the PYNQ-Z2 master XDC**, uncommenting the Arduino-header lines you need, and cross-check them against `docs/icd.md`'s pin assignment. If the ICD and the XDC disagree, the ICD is what W7's schematic will be built from, so resolve the disagreement now and in the ICD.
- Set `IOSTANDARD LVCMOS33` on all six. Add a `set_property PULLUP true` on each `rx` so a disconnected bus idles high rather than floating — a floating receiver generates phantom start bits and will cost you an hour in S4 otherwise.
- Add `set_false_path` or proper input delay constraints for the asynchronous `rx` inputs. Leaving them unconstrained produces a clean timing report that is lying to you, and "which paths did you constrain and which did you exclude?" is a review question.

Done when: the block design validates and synthesis starts cleanly.

S2 · Build, close timing, and read the reports (2 h)

- Run implementation to bitstream. While it runs, open the previous run's reports rather than watching the progress bar. `report_timing_summary` and `report_utilization` are all this week needs and both are available on the Basic tier; some of Vivado's advanced timing-closure guidance is reserved for paid tiers, so if a tutorial points you at a report that refuses to run, that is why.
- Record from `report_timing_summary`: **WNS**, **TNS**, and the worst failing path if any. Record from `report_utilization`: LUT, FF, BRAM and DSP as absolute numbers and as a percentage of the 7Z020.
- If WNS is negative, fix it properly and document what you did:
 - Read the failing path first. Is it a long combinational chain (your checksum adder, or a wide comparator)? A high-fanout net (a reset, or an enable)? An I/O path with no delay constraint?

- Pipeline the offending logic, or register a high-fanout net, in that order of preference.
 - Retiming and over-constraining are last resorts and should be recorded as such.
 - Write `docs/decisions/0009-timing-closure.md` with the before/after WNS and what changed. Interview question 5 is exactly this, and a documented negative-to-positive story is worth more than a design that closed first time.
4. Archive the timing report into `docs/reports/` with the date and the git hash in the filename. A timing report you cannot tie to a commit is not evidence.
 5. Copy `.bit` and `.hwh` to the board under `overlay/v0.1/` with matching base names.

Done when: WNS is positive, the report is archived against a commit hash, and — if it ever went negative — the decision record exists.

S3 · The PYNQ driver, reusing the W3 tests (2 h)

1. Write `sw/pynq_hx.py` as a `DefaultIP` subclass so PYNQ binds it automatically:

```
from pynq import DefaultIP

class ServoBusMaster(DefaultIP):
    bindto = ['yourname:user:servo_bus_master:1.0']

    CTRL, STATUS, CONFIG = 0x00, 0x04, 0x08
    TX_DATA, RX_DATA, FIFO_LVL, ERR_CNT = 0x0C, 0x10, 0x14, 0x18
    BUS_STRIDE = 0x1000

    def __init__(self, description):
        super().__init__(description=description)

    def _off(self, bus, reg):
        return bus * self.BUS_STRIDE + reg

    def read_reg(self, bus, sid, addr, n):
        ... # write descriptor, poll STATUS.busy, drain RX_DATA

    def write_reg(self, bus, sid, addr, data: bytes):
        ...

    def sync_write_positions(self, bus, positions: dict):
        ...
```

2. **Confirm the arm before torque.** On start-up, ping IDs 1–6 on each bus and read each servo's model number (from the address Hiwonder's document gives). Refuse to enable torque unless the follower bus returns six HX-30HM and the leader bus six HX-10HM. Both arms use the same connector at the same voltage, so this check is what actually enforces REQ-013.
3. Poll `STATUS` rather than sleeping. A fixed `sleep` in the driver hides the latency you are trying to measure and caps your loop rate at whatever the sleep granularity is.
4. **Reuse the W3 unit tests against the new backend.** This is the payoff for writing them dependency-light: point the same test functions at a `ServoBusMaster` instead of a `HxBus` and they should pass unchanged. Where they do not, the difference is either a real bug or an unstated assumption in the spec — both worth writing down.
5. Add an interface-conformance test that runs against *either* backend, so W10 and W11 can use it as a regression.

Done when: the W3 tests pass against the PL backend with no hardware attached beyond the board itself (loopback `tx` to `rx` on the breadboard if needed).

S4 · ILA: find the bug the simulator was too polite to find (2 h)

1. **Instantiate one ILA core, with at most five probes.** On the Vivado Basic tier the ILA is available by instantiation only — one core, up to five probes, each up to 1024 bits wide — and the System ILA and the `mark_debug / Set Up Debug` insertion flow are not available. That limit shapes the design, which is a useful constraint to have met:
 - Add the ILA IP from the catalogue in your Tcl (`create_bd_cell -type ip -vlnv xilinx.com:ip:ila ...`) or instantiate it directly in RTL, and connect it to the PL clock.

- **Pack related signals into wide probes** rather than spending one probe per signal. A workable allocation:

Probe	Width	Contents
0	3	{dir, rx, tx} for one bus — the trigger probe
1	a few	packet engine state
2	8	checksum accumulator
3	16	timeout counter
4	16	{rx_level, tx_level} or the second bus's {dir, rx, tx}

- Record the allocation in the README and in `docs/decisions/0014-ila-within-basic-tier.md`. Being able to say "I had five probes, so I packed them this way and here's what I gave up" is a better interview answer than "I probed everything".
2. Rebuild, load, and capture a real transaction triggered on `dir` rising.
 3. Put the ILA capture **next to the W5 timing diagram and the W2 scope shot** of the same event. Three views of one thing: intent, simulation, reality. Differences between any pair are findings.
 4. Measure the real turnaround: `dir` falling edge to the first bit of the reply. Compare with the simulated number from W5 S6. Expect a discrepancy. Common causes, in the order they are usually true:
 - `DIR` released one bit time early because the stop bit was not counted.
 - The pad's output delay is not in your simulation, so the real release is later than modelled.
 - The receiver arms a cycle after `DIR` releases and eats the reply's start bit.
 5. Fix it in the RTL, **add a cocotb test that reproduces it**, and confirm the test fails on the old RTL and passes on the new. That sequence — hardware finding, regression test, fix — is the answer to interview question 3, and you want to have actually done it.
 6. Log the finding in `docs/eco.md` even though this is RTL rather than PCB. The ECO tracker opens in W8, but a finding is a finding; note it now and move it when the file exists.

Done when: simulated and measured turnaround agree, and a regression test exists that would have caught the original bug.

S5 · Saturday: real servos from the PL (4 h)

1. **Check the signal level first.** From your W2 scope captures, what level does the servo bus idle at? PYNQ-Z2 PL pins are 3.3 V and not 5 V tolerant. If the bus idles above 3.3 V, the receive path must go through a translator with its bus side at the servo's level (an SN74LVC1T45 with its VCCB side at that level), not straight into a PL pin.
2. **Build the interim bus drivers on the breadboard.** Per bus, you need a 3.3 V half-duplex node: the PL `tx` drives the bus through a tri-state buffer enabled by `dir`, and the bus feeds the PL `rx` continuously. Keep the wires short — this is a 1 Mbps bus on a breadboard and it is already marginal. Add the series resistor at the driver output now; you will size it properly in W7, but 22 Ω is a reasonable starting value. Use pre-crimped 5264-3P leads rather than bare jumpers into the servo connector.
 - **A fallback, and why it is only a fallback.** The BusLinker V3.0 has a TTL serial header and a communication jumper that lets an external controller drive the bus through the board's own half-duplex circuit. Wired to the PL's `tx` / `rx`, it would get servos moving tonight. But it hides exactly the thing this week tests: your `dir` timing, because the BusLinker handles direction itself. Use it only to separate "my RTL is wrong" from "my breadboard is wrong" if you get stuck, and never before checking its TTL logic level with a meter — **PYNQ-Z2 PL pins are 3.3 V and are not 5 V tolerant.**
3. **Rail check.** 12.0 V on the display; follower limit 5 A, leader limit 3 A, each arm on its own supply output through a 5264 breakout lead. Do **not** power the arms through a BusLinker while the breadboard drives the bus — the BusLinker's own driver sits on the same signal line and would fight yours. The breadboard carries **signals only** — do not run servo power through it.
4. Ground the board and the arm supplies together at one point before anything is energised. Different grounds across a 1 Mbps bus is the other classic breadboard failure.
5. Bring it up in strictly this order, stopping at the first failure:
 - **Loopback:** disconnect the servos, tie `tx` to `rx` through the buffer, transmit a packet and confirm you receive your own bytes. This proves the buffer and the direction logic with nothing at risk.

- **One servo, ping.** A single follower servo on the bus. Ping it. This is the first real byte exchange between your RTL and a real part.
 - **One servo, read position.** Move it by hand with torque off and confirm the value tracks.
 - **One servo, write goal.** Small move, 100 counts. Hand near the supply output button.
 - **Six servos, ping all.**
 - **Six servos, sync-write.** A small coordinated move. Confirm on the scope that it is one packet of the W3-measured length.
 - **Leader bus, read six.** Reads only.
6. **Python teleop on the PS over the PL IP.** A simple loop: read six leader positions, map them, sync-write six follower goals. Keep it deliberately naive — this is the PS-in-loop baseline, and its jitter is the thing W10 eliminates.
7. Measure, with the same method as W3 so the numbers are comparable:
- Read round-trip through the PL path, 1000 samples, same histogram script. Overlay it on the W3 USB histogram in one figure. That figure is the best single image in your whole design package.
 - Teleop latency, scope on both bus lines, 20 captures, same tap-the-joint method.
8. Update `docs/icd.md` to v0.2 (or v0.3) with whatever S3 and S4 changed, and archive the ILA captures into `docs/` .

Done when: both arms run from PL, and the two-histogram figure exists.

Measurements to record

Quantity	Method	Compare against
WNS, TNS, utilisation	Vivado reports, archived with git hash	Budget; W10 will add to it
PL-path read round-trip, n=1000	Same script as W3	W3 USB histogram — one overlaid figure
Teleop latency, PS in loop, n=20	Two-channel scope	W3 baseline; W10 target
Measured turnaround vs simulated	ILA vs waveform	W5 S6 number
Bus error counters over 10 min of teleop	ERR_CNT	W9 and W11 bus health

Deliverables

-  `overlay/v0.1/` — scripted build, XDC, driving both arms
-  `sw/pyng_hx.py` — `DefaultIP` driver passing the W3 tests
-  ILA captures in `docs/` , timing report archived against a commit
-  The overlaid USB-vs-PL latency figure
-  `docs/icd.md` v0.2+, `docs/decisions/0009-timing-closure.md` if timing ever failed
-  One new cocotb regression test from a hardware-found bug

Failure modes

Symptom	Likely cause	Fix
Phantom bytes with no servo attached	Floating <code>rx</code>	<code>PULLUP true</code> in the XDC
Ping works at 1 servo, fails at 6	Breadboard signal integrity, or shared-ground problem	Shorten wires, single-point ground; note it as justification for W7
Every reply times out, <code>tx</code> looks correct on the scope	<code>dir</code> polarity inverted vs the buffer's enable	Check the buffer's active level; fix in RTL not with a wire
Works cold, fails after minutes	Servo heating, or a marginal breadboard contact	Check the error counters first — they tell you which
Timing report is clean but the design fails	Asynchronous inputs unconstrained	Add input delays or explicit false paths, then re-read the report
PL read latency no better than USB	Driver sleeping rather than polling	<code>Poll STATUS</code>

W07 · Shield schematic in KiCad (14 h)

Competency: part selection, derating, and design-for-safety. Making a schematic reviewable is itself a senior skill. **Budget note:** 14 h — five standard sessions plus one extra 2 h block for breadboard prototyping. **Rule for the week:** every part must be in stock at LCSC with a named alternate, and every value must trace to either a W2 measurement or a formula written in the schematic notes. "Looked right" is not a sourcing strategy.

S1 · The 12 V input path (2 h)

Work from the connector inwards. Draw it as a left-to-right chain so the review reads naturally.

- Input connector.** XT30 or barrel, rated above your W4 continuous current — the sum of both arms — with derating. If you want the kit's own 12 V 5 A adapter to plug in for demos, add a matching jack in parallel, but design the current path for the bench supply.
- Reverse polarity protection.** P-channel MOSFET in the high side: source to the input, drain to the downstream rail, gate pulled to ground through $\sim 100\text{ k}\Omega$ with a 12 V–15 V zener clamping V_{GS} . Select on: $V_{DS} \geq 30\text{ V}$, $I_D \geq 2\times$ continuous, and $R_{DS(on)}$ low enough that $P = I^2 \cdot R$ is comfortably inside the package's dissipation. At 8 A, a 10 m Ω device dissipates 0.64 W — plan the copper for it in W8.
 - Note why a P-FET rather than a series diode: a Schottky at 8 A costs you $\sim 0.4\text{ V}$ and 3 W. Put that comparison in the schematic notes; it is a one-line trade-off a reviewer will ask about.
- TVS** across the input, for fast transients and ESD only. Choose the standoff voltage just above your over-voltage trip threshold so it never conducts in normal operation, and then **write down its clamping voltage next to the servos' 12.6 V maximum**. They will be nowhere near each other — a TVS at this standoff clamps somewhere above 20 V. That gap is the whole argument for S2's active cutoff, and seeing it on the sheet is what makes the argument obvious to a reviewer.
- Fuse**, sized in S1 of W4: $I_{fuse} \geq 1.5 \times I_{continuous_max}$, derated $\sim 25\%$ for a 50 °C ambient. Put the arithmetic in a schematic text block.
- Inrush limiting.** Either an NTC inrush limiter in series, or — better, since you have a load switch anyway — a controlled gate ramp on the load switch, where a gate capacitor sets dv/dt and therefore $I_{inrush} = C_{load} \cdot dv/dt$. Compute the ramp time from your bulk capacitance and a target inrush below the fuse's let-through.
- Bulk capacitance** at the value computed in W4, split between low-ESR electrolytic and ceramics. Note the electrolytic's ripple-current rating and its derating at temperature.

Done when: the input chain is drawn with every value's arithmetic in a text block on the sheet.

S2 · The over-voltage cutoff and both 12 V load switches (2 h)

Both arms run from the same 12 V input, so the shield has **no regulator for the servos** — it switches, protects and measures two copies of the same rail. The design effort that a 5 V buck would have taken goes into the over-voltage cutoff instead, because that is where the kit's real hazard lives.

1. **Choose the cutoff architecture** and record it in `docs/decisions/0012-active-over-voltage-cutoff.md` :
 - **Integrated eFuse** with adjustable over-voltage cutoff, current limit and controlled inrush in one part. Fewer parts and a characterised response time; check that the OVP threshold is adjustable down to about 13 V and that the part carries your W4 current per rail.
 - **Discrete**: a comparator with a precision reference watching a divider on the input, driving the load-switch gates off above threshold. More parts, more tolerance arithmetic, but every element is visible and you own the response time. Either is defensible. The decision record must state the trip threshold, the worst-case trip range from the W4 tolerance stack, the hysteresis, and the **measured** response time you will confirm in S6.
2. **Threshold and hysteresis**. Trip around 13.0 V, release a few hundred millivolts lower. Without hysteresis a supply sitting near the threshold makes the rails chatter, which is worse than either state. Check the low end of the tolerance range against your supply at maximum setting, and the high end against 12.6 V.
3. **Two independent load switches**, one per arm, each a high-side P-FET with its own gate drive (or one eFuse per rail). Selection criteria as in S1, plus: check the **safe operating area** during the inrush ramp, because that is where load switches actually die.
4. The over-voltage cutoff, the e-stop and the watchdog all act on the **same** hold path (S5), so any one of them kills both rails. Draw the hold signal as a clearly-labelled net — `RAIL_HOLD` — routed to both switches, so the review can trace it in one glance, and draw the over-voltage signal as a separate labelled net `OV_TRIP` that also reaches a PL input so the PL knows *why* the rails dropped.
5. Add a **power-good** or rail-sense divider from each switched rail back to a PL input. W9's bring-up and W12's fault injection both want to know, in the PL, whether the rail is actually up.
6. Decoupling pass: every IC gets its 100 nF at the pin, and bulk capacitance per switched rail. Write "decoupling reviewed" as a checklist item, not an assumption.

Done when: both rails are drawn end to end behind the over-voltage cutoff, the worst-case trip range is on the sheet, and you could point at any component and say where its value came from.

S3 · Servo buses, signal levels, and telling the two ports apart (2 h)

1. **Per bus**, two 3.3 V devices: a tri-state buffer driving the bus, enabled by the PL `DIR` pin, and a receive path from the bus back to the PL `RX`. `SN74LVC1G125` (active-low output enable) or `SN74LVC1T45` (direction-controlled translator) both work; pick one and note why in the schematic. The 1T45 gives you translation you do not strictly need; the 1G125 is simpler and cheaper.
 - Check the `DIR` polarity against your W5 RTL. If they disagree, fix the RTL, not the schematic — a bodge wire on the board is a worse answer than a rebuild.
 - **Check the signal level before choosing**. Look at the idle level you measured on the bus in W2. If the HX servos pull the line to 5 V, a 3.3 V buffer's input and the PL's `RX` pin are both at risk, and you need a translator with its bus side at 5 V — the 1T45 then earns its place. If the bus idles at 3.3 V, check the servo's input-high threshold against a 3.3 V driver instead. Write the measured level and the resulting choice on the sheet.
2. **Series resistor** at each driver output, 22–47 Ω , to slow the edge and damp the ringing you measured in W2. Choose the value from that measurement: more ringing, more resistance, at the cost of rise time. Put the W2 number in the schematic note so the choice is traceable.
3. **ESD diodes** at each bus connector, low-capacitance parts — a fat protection diode on a 1 Mbps line rounds your edges more than the series resistor does. Check the diode's capacitance against your total bus load.
4. **Pull-up** on each bus to 3.3 V so the line idles high when nothing is driving. Size it against the drivers' sink current.
5. **Connectors**. Each arm connects through one **5264-3P** port carrying GND, 12 V and the bus signal, in the pin order you confirmed in W0. Use the pitch you calipered.
 - Both ports are the same connector at the same voltage, so plugging the leader into the follower port destroys nothing. What it does do is let the PL command the leader as if it were the follower — moving the arm a person is holding. REQ-013 asks for the two to be distinguishable; meet it with the cheapest controls that work: `LEADER` and `FOLLOWER` in the largest silkscreen that fits, ports on opposite board edges, coloured cable sleeves, and net names `VBUS_FOLLOWER` / `VBUS_LEADER` , `BUS_FOLLOWER` / `BUS_LEADER` .

- Add a **software** check as the second layer: at start-up the driver pings each bus and confirms the model it finds (HX-30HM on the follower bus, HX-10HM on the leader bus) before enabling torque. That is a cheap, strong control, and it goes into W6's driver and W12's FMEA.
 - Record in the schematic notes why you did not use two different connector families: the cables are the kit's, and replacing them trades a minor hazard for a new supply-chain problem.
6. Confirm your Arduino-header pin assignment against `docs/icd.md` and against the master XDC one more time. A pin mismatch discovered in W9 costs you a rev; discovered now it costs a minute.

Done when: both bus channels are drawn with the signal-level decision recorded, the two ports are unmistakable on silkscreen and in the netlist, and the ICD, the XDC and the schematic agree pin for pin.

S4 · Sense: current, temperature, and test points (2 h)

1. **INA226 per rail**, high-side shunt, I²C to the PS.
 - Shunt value from the part's ± 81.92 mV full-scale: $R_{\text{shunt}} = 81.92 \text{ mV} / I_{\text{fullscale}}$. For a 10 A full scale that is 8.2 mΩ; for a 3 A full scale, 27 mΩ.
 - Shunt power: $P = I^2 \cdot R$. At 10 A into 8 mΩ that is 0.8 W, so a 2 W part in a 2010 or 2512 package with thermal relief planned for W8.
 - Set the I²C addresses differently via the A0/A1 strapping and **write both addresses on the schematic**. Two devices at one address is a classic and it is invisible until bring-up.
 - The **ALERT** pin is open-drain and is the fast trip path in W12: it needs a pull-up and a route to a PL input, not just to the PS. Note the conversion time you plan to configure — averaging buys noise performance and costs trip latency, and W12 has a reaction-time requirement to meet.
2. **Kelvin connections** on both shunts: the sense traces must connect at the shunt's own pads, not somewhere along the high-current copper. Draw them as separate nets in the schematic so W8's layout cannot accidentally merge them.
3. **NTC** near the follower load switch, into a divider to a PL analog input or to a spare INA226-adjacent ADC — whichever your W4 decision record chose. Note the divider values and the temperature-to-voltage curve you will use in software.
4. **Test points** — this is the cheapest thing on the board and the thing you will be most grateful for in W9. One on each of: 12 V input, 12 V after the reverse-polarity FET, follower switched rail, leader switched rail, the over-voltage comparator input (or eFuse OVP pin), `OV_TRIP`, 3.3 V, both bus lines, both `DIR` signals, `RAIL_HOLD`, and at least two grounds positioned for a scope probe's spring ground.
5. Write the expected voltage and tolerance for each test point straight into the schematic as a text block. W8's bring-up plan copies that table; having it on the schematic means the board and the plan cannot drift apart.

Done when: every test point has a documented expected value, and the two INA226 addresses are explicitly different on the sheet.

S5 · Saturday: safety chain, ERC, BOM, and a real schematic review (4 h)

1. **Draw the safety chain as its own sheet.** It is the most important page in the package and it should be readable alone.
 - The **e-stop** is a latching, normally-closed mushroom switch in series with the `RAIL_HOLD` path. Opening it removes the hold from both load switches with no firmware, no microcontroller and no PL logic in the path. Trace the path with your finger on the printed sheet and confirm there is nothing programmable in it — if there is, redraw.
 - The **watchdog**: a PL output must keep *toggle* to hold the rails on. AC-couple the toggle into a diode charge pump that holds the gate-drive transistor on; when toggling stops, an RC decays and the rails drop. Choose the RC for your REQ-021 drop-out time — 1 MΩ with 100 nF gives roughly a 100 ms time constant. State the computed drop-out time on the sheet.
 - A **level** output cannot do this job. If the PL hangs with the pin high, or the bitstream is cleared and the pin floats to a pull-up, a level-based hold stays on. Edge-based holding fails safe. Write that sentence on the schematic; it is interview question 9's answer.
 - The **over-voltage cutoff** breaks the hold directly in hardware — it must, because by the time software notices an over-voltage the servos have already seen it.
 - The **INA226 ALERT** and the over-temperature signal also break the hold, or drive a PL input that does. Decide which for each, and record the choice — this is the "which failure did you move from software to hardware?" question, and you want a deliberate answer for each one.

2. **ERC clean.** Run it, and fix every error and every warning or annotate the ones you are accepting. An ERC with sixty ignored warnings is the same as no ERC.
3. **BOM with alternates.** One row per part: designator, value, package, manufacturer part number, LCSC part number, **an alternate part number**, and a derating note (voltage, current, or power margin applied). Check stock for each at the time of writing and record the date — stock on the day you order is what matters, and a BOM with a date is honest about that.
4. **Run a formal schematic review on yourself**, checklist first, findings recorded, exactly as in W4:
 - Every IC decoupled at the pin?
 - Every unused input tied off, not floating?
 - Every pull-up and pull-down present and sized?
 - Every connector pinout checked against the mating part, including pin 1 orientation?
 - Can more than 12.6 V reach any servo by any path, including a single fault in the cutoff? Trace it, and name the single fault that would defeat it.
 - Is the bus signal level compatible with both the servo and the PL pin, per the W2 measurement?
 - Does the safety chain contain anything programmable? Trace it.
 - Are both INA226 addresses different?
 - Does the DIR polarity match the RTL?
 - Is every part in stock, with an alternate?
 - Is every component value traceable to a measurement or a formula on the sheet?
5. Write `docs/reviews/2026-w07-schematic-review.md` with classified findings and actions. Close the blockers before the week ends; carry the rest into W8 with dates.

Done when: ERC is clean, the BOM has alternates and a stock date, and the review minutes list closed blockers.

S6 · Extra block: prototype the parts you are least sure of (2 h)

Do not commit an untested sub-circuit to a two-week fab cycle.

1. **INA226 on the breadboard**, reading a known current through your chosen shunt value. Confirm the I²C address strapping, read a current, and compare against the bench supply's readout. Calibrate and note the error.
2. **The watchdog charge pump.** Drive it from a PL GPIO toggling at a few kHz, confirm it holds a transistor on, then stop the toggle and **measure the drop-out time on the scope**. Compare with your computed RC. This is the number REQ-021 is written against, and measuring it before layout means you can change the RC value for free.
3. **The e-stop contact.** Confirm it is normally-closed and latching, and confirm that pressing it opens the hold path. Measure the contact bounce; if it is long, the RC already handles it, but know the number.
4. **The over-voltage cutoff**, on the breadboard or an eFuse evaluation board, with **no servos attached** — a resistive load only. Ramp the bench supply slowly from 12.0 V upwards and record the trip voltage; ramp down and record the release voltage. Then step the supply abruptly from 12 V to 14 V and measure the **time from the step to the output falling** on the scope, and the peak voltage the output reached before it fell. That peak is the number that matters: it is what a servo would actually have seen.
5. **TVS and ESD parts:** confirm footprints and polarity markings against the physical parts, if they have arrived.
6. Record all five results in `docs/prototype-notes.md` with the schematic values they confirm or change.

Done when: the watchdog drop-out time and the over-voltage trip point, hysteresis, response time and output peak are all measured, not computed.

Measurements to record

Quantity	How	Feeds
Watchdog drop-out time	Breadboard + scope	REQ-021, W12 fault injection
INA226 reading vs supply readout	Breadboard	W9 calibration, W11 current tests
E-stop contact behaviour and bounce	Meter + scope	W12
Computed vs chosen shunt values and dissipation	Arithmetic on the sheet	W8 thermal relief
Over-voltage trip, release, response time, output peak	Breadboard + scope, resistive load	REQ-011, W9, W12
Servo bus idle level (from W2) and buffer choice	Scope	Signal-level decision

Deliverables

-  hw/shield/ — KiCad schematic, ERC clean, exported PDF
-  Safety chain on its own readable sheet
-  BOM with manufacturer and LCSC numbers, alternates, deratings, stock date
-  docs/reviews/2026-w07-schematic-review.md — classified findings, blockers closed
-  docs/prototype-notes.md — five breadboard results, including the over-voltage cutoff
-  docs/decisions/0012-active-over-voltage-cutoff.md with measured response time

Failure modes

Symptom	Likely cause	Fix
Two INA226s, one responds	Same I ² C address	Strap A0/A1 differently; catch it at ERC time
Bus idles low, phantom traffic	No pull-up, or buffer enabled by default	Add the pull-up; check <code>DIR</code> default state at reset
<code>DIR</code> inverted vs RTL	Buffer's OE is active-low	Fix in RTL before layout, never with a bodge
Connector does not mate with the servo cable	Pitch assumed rather than measured	This is why W0 calipered it
Safety chain has a PL signal in series	Convenience during drawing	Redraw; hardware chain must stand alone
Load switch dies on first power-up	Inrush outside SOA	Slow the gate ramp; recompute from bulk capacitance
Rails chatter with the supply near 13 V	No hysteresis on the cutoff	Add hysteresis; it is a design requirement, not a nicety
Cutoff trips at nominal 12 V	Tolerance stack too wide	Tighter reference and 0.1 % divider resistors; redo the W4 arithmetic
PL <code>RX</code> pin damaged	Bus idles at 5 V into a 3.3 V input	Translator with a 5 V bus side; check W2's idle level first

W08 · Layout, DFM, release, and the bring-up plan (14 h)

Competency: release discipline and planning around lead times. Seniors know a board on order is a schedule, not a hope.

Budget note: 14 h. The fab order goes out **this week** — it is the critical path, and every day of slip here is a day removed from W9.

S1 · Stack-up, outline, and placement (2 h)

1. **Four-layer stack-up**, and know why you chose it: L1 signal, **L2 solid ground**, L3 power, L4 signal. Use a standard fab stack-up (JLCPCB's 1.6 mm four-layer with a 7628 prepreg is the default) so impedance and cost are predictable. The reason for four layers is the solid ground plane on L2 — it gives every 1 Mbps bus line a continuous return path directly beneath it and it is what makes interview question 2 answerable.
2. **Board outline and the mechanical hazards.** This is where shields fail.
 - Match the Arduino-header mechanical outline exactly. Take the header positions from the PYNQ-Z2 mechanical drawing, and then **verify against the physical board with calipers**. The Arduino header's deliberate non-standard 0.05" offset on one connector has caught everyone at least once.
 - Mark keep-out zones for the PYNQ-Z2's **Ethernet jack, USB host port and HDMI connectors**, all of which are tall. Add a 3D check: place your tallest components (electrolytics, the inductor, the e-stop wiring) and view the board in 3D over a model of the PYNQ-Z2 if you have one; otherwise print the layout 1:1 on paper, cut it out, and physically hold it over the board.
 - Plan the e-stop as an **off-board switch on a cable** with a connector on the shield. A mushroom switch mounted on a shield is neither reachable nor safe.
3. **Placement, in this order:** connectors at the edges where the cables want to go, **LEADER** and **FOLLOWER** on opposite edges; the input protection chain (connector, reverse-polarity FET, TVS, over-voltage cutoff) in a straight line close to the input — copying the eFuse datasheet's reference layout if you chose one; the load switches with copper area for their dissipation; the shunts where the high-current path naturally passes; the bus buffers close to their connectors; then everything else.
4. Keep the **12 V high-current path physically away** from both bus lines. There is no switching regulator on this board, which removes the biggest noise source a shield usually has — but a servo stall is a multi-amp step, and the loop from the input capacitor through the load switch to the servo return still radiates. Keep that loop small. Route the over-voltage comparator's sense divider as a quiet signal, away from the current path, so load steps do not look like over-voltage.

Done when: placement is final and the 1:1 paper check confirms nothing collides with a tall connector.

S2 · Power routing and thermals (2 h)

1. Route the high-current 12 V path first, as **copper pours rather than traces**. Width from the KiCad calculator against IPC-2152 for your current and an acceptable temperature rise; expect to need several millimetres or a pour. Record the width and the assumed rise in the layout notes.
2. **Kelvin the shunts.** The sense traces leave from the shunt pads themselves as a tight differential pair, routed away from the current path, and they must not carry any load current. This is the single easiest thing to get wrong and it directly corrupts every current measurement in W11.
3. **Thermal relief and copper area** for the load-switch FETs and the shunts, sized against the dissipation you computed in W7. Put the copper on both outer layers with stitching vias if you need more.
4. **Via count for current.** A single via does not carry 8 A. Use a via array wherever the 12 V rail changes layer, and state the per-via current you assumed.
5. Keep the **ground plane on L2 unbroken** under the bus lines. If you must split the plane, do it where no signal crosses the split, and be able to point at every crossing and explain it.

Done when: the current path is poured, the shunts are Kelvined, and the L2 plane under the bus lines is continuous.

S3 · Signal routing and the return-path story (2 h)

1. Route both buses as short, direct traces on L1 with L2 ground directly beneath. At 1 Mbps with a ~5 ns edge, the electrical length matters less than the return path — but the return path always matters, and it is the question you will be asked.
2. Place the series resistors **at the driver**, not at the connector: their job is to damp the edge at its source.
3. Place the ESD diodes **at the connector**, before anything else: their job is to catch the event at the entry point.

4. Keep the two buses apart from each other and from the high-current 12 V copper. Where they must run parallel, add ground between them or increase spacing.
5. Route I²C with a clean return, and keep the pull-ups near the master end.
6. **Write the return-path answer now**, while it is in front of you, into `docs/layout-notes.md` : for a given bus trace, where does its return current flow, and what would change at 10 Mbps? At 1 Mbps the return spreads; at 10 Mbps it hugs the trace, and the cost of a plane split rises sharply. That paragraph is interview question 2 answered in your own words about your own board.
7. Silkscreen: voltage labels at both bus connectors, polarity at the input, pin 1 markers, test-point names, a version string (`rev-A`) and the date. Silkscreen is free and it is what makes W9 fast.

Done when: both buses are routed with continuous return paths and the return-path note is written.

S4 · DRC, DFM and the checks that catch the expensive mistakes (2 h)

1. **DRC against the fab's actual rules**, not KiCad's defaults. Import JLCPCB's design rules (minimum trace/space, annular ring, drill sizes, edge clearance) and run it clean.
2. **DFM pass**, by eye and by checklist:
 - Any acid traps or slivers?
 - Solder-mask slivers between fine-pitch pins?
 - Are all footprints checked against the actual datasheet drawings? Print the footprints 1:1 and lay the physical parts on them — this catches a wrong footprint in thirty seconds and is the highest-value DFM check available to you.
 - Is every polarised part's orientation marked on the silkscreen *and* consistent in the 3D view?
 - Are there fiducials, if you are ordering assembly?
3. **Netlist sanity, manually**. Open the netlist and grep for the hazards: confirm that **no path reaches either servo connector's 12 V pin without passing through the over-voltage cutoff**, that each switched rail reaches only its own arm's connector, and that `RAIL_HOLD` and `OV_TRIP` go where the safety sheet says. A few greps, a few minutes, and it is the check that protects twelve servos.
4. **3D render pass**. Look at the board from four angles against the PYNQ-Z2's tall connectors one final time.
5. Run the review checklist from W7 S5 again against the *layout* — a schematic review does not cover placement decisions.

Done when: DRC is clean against the fab's rules and the footprint print-out check is done with physical parts.

S5 · Saturday part one: release the fab package (2 h of the 4 h block)

Release is an event with a procedure, not a button.

1. Export the complete package into `hw/shield/fab/rev-A/` :
 - Gerbers (all layers, both masks, both silks, board outline)
 - Drill files, including plated/non-plated separation
 - Pick-and-place / centroid file
 - BOM in the assembler's required column format
 - Schematic PDF and a layout PDF
 - A `README.md` stating the fab, the stack-up, the finish, the quantity and the assembly scope
2. **Open the Gerbers in a separate viewer** — not KiCad's own — and look at every layer. A Gerber viewer catches export mistakes that the layout tool cannot see because it is showing you its internal model rather than the exported files.
3. Place the order. Choose **SMD assembly** so W9's time goes into measurement rather than soldering. Order a stencil for anything you will hand-place. Order **at least three boards**; the marginal cost is near zero and the second board is how you distinguish a design fault from an assembly fault in W9.
4. **Tag the release in git:**

```
git tag -a rev-A -m "Shield rev-A released to fab $(date -I)"
git push --tags
```

5. Record the order date, the promised ship date, and the tracking reference in `docs/risks.md` against the lead-time risk. Update the schedule with the real dates.
6. **Open `docs/eco.md` the same day.** The format: ID, date found, where found, description, classification (design / assembly / documentation), disposition (rev-B / bodge / no action), and status. Move the W6 RTL finding into it as ECO-001 if you have not already. The first real entry usually appears within an hour of release, and the discipline is to write it down rather than to feel bad about it.

Done when: the package is ordered, `rev-A` is tagged and pushed, and the ECO tracker exists.

S5b · Saturday part two: write the bring-up plan (2 h)

Write this **now**, while the board is fresh in your mind and before it arrives, because a plan written with the board on the bench is a plan written to justify what you already did.

Structure `docs/bringup-plan.md` as a sequence of numbered stages, each with entry conditions, steps, expected values with limits, and an explicit "stop if" condition.

Stage	Entry condition	Key steps	Expected	Stop if
0 · Inspection	Board in hand, unpowered	Visual under magnification; check orientation of every polarised part; continuity between every rail and ground	No rail shorted to ground	Any rail reads < 10 Ω to ground
1 · Bare power	Stage 0 clean, nothing plugged in	12 V in via bench supply, limit 500 mA , no arms, no PYNQ	Input current in tens of mA; 12 V at TP1	Current hits the limit
2 · Over-voltage cutoff	Stage 1 clean, still no load	Ramp input 12.0 V upwards; record trip and release; then a 12→14 V step, measuring the output peak	Trip inside the W4 range; output peak below 12.6 V	Trips outside the range, or output ever exceeds 12.6 V
3 · Load switches	Stage 2 clean	Assert <code>RAIL_HOLD</code> manually; verify both rails switch	Both rails follow the hold	Either rail cannot be switched off
4 · Safety	Stage 3 clean	E-stop opens both rails; stop the watchdog toggle	Rails drop; measure the time	Rails stay up
5 · Bus loopback	Stage 4 clean, PYNQ mated	Loopback each bus with no servos	Own bytes received on both	Either bus fails loopback
6 · One servo	Stage 5 clean	Rail check. One follower servo, ping, read, small move	Correct replies, clean move	Any error counter increments
7 · Six servos	Stage 6 clean	Ping six, sync-write a small move	One 26-byte packet, six joints move	Any timeout
8 · Leader	Stage 7 clean	Rail check. Leader on the <code>LEADER</code> port; driver confirms six HX-10HM before torque; read six	Positions track hand motion	Wrong model found on a bus, or anything unexpected
9 · Thermal	Stage 8 clean	Ten minutes of continuous teleop, then thermal survey	All parts within your derating	Any part above its derated limit

For each stage, also write: the tools needed, the safety steps, and what "done" means. Add the expected-value table copied from the W7 schematic text blocks so the plan and the board agree by construction.

Done when: a stranger could run stage 0 through 9 from the document alone, and every stage has a "stop if".

S6 · Extra block: prepare for the wait (2 h)

Boards take about two weeks, and the holiday buffer covers most of that. Do this block before the holidays start, so the bench is ready on 4 January and nothing has to be remembered.

1. Decide now what you will do if the boards are late: the risk register says swap W9 and W10, and W10 runs fine on the breadboard buffers. Set the trigger date — boards not in hand by **Monday 4 January** means W10 goes first — and put it in the schedule.
2. Set up the W9 bench in advance: supplies labelled and limited, scope probes compensated, thermal camera or IR thermometer to hand, the bring-up plan printed on paper with space to write measured values beside expected ones. Bring-up with a paper checklist and a pen is faster than bring-up with a laptop.
3. Pre-write the empty `docs/bringup-report.md` with the measured-vs-expected table already laid out, rows for every test point. Filling in a prepared table is how you avoid finishing bring-up with a working board and no record of it.
4. If time remains, start reading ahead for W10: the C++ primer and Vitis HLS example noted in W10 S3.

Done when: the bench is set up, the checklist is printed, and the empty report table exists.

Deliverables

-  `hw/shield/fab/rev-A/` — complete, Gerber-viewer checked, ordered
-  `rev-A` tag pushed, order dates in the risk register
-  `docs/eco.md` opened, first entry made
-  `docs/bringup-plan.md` — ten stages, limits, stop-if conditions
-  `docs/layout-notes.md` — trace widths, return-path answer, stack-up rationale
-  Empty `docs/bringup-report.md` table, bench prepared, checklist printed

Failure modes

Symptom	Likely cause	Fix
Shield fouls the Ethernet or HDMI connector	Height not checked in 3D	The 1:1 paper cut-out check prevents this
Header does not align	Arduino header's offset spacing assumed	Calipers on the real board
Current readings wrong by a constant factor	Shunt not Kelvined	Catch it in S2; it is unfixable after fab
Cutoff trips during a servo stall	Sense divider routed alongside the current path	Route the divider as a quiet signal; filter it lightly
Gerbers missing a layer	Exported from a stale plot config	Open them in an independent viewer
Boards arrive, assembly missing parts	Stock changed between BOM and order	The alternate column in the W7 BOM is why
No record of bring-up afterwards	No prepared table	Pre-write it during the wait

Gate 2. Rev-A released and on order. The overlay drives both arms from PL through the breadboard buffers. The bring-up plan exists before the boards arrive.

Holiday buffer · 21 December 2026 – 3 January 2027 (2 weeks)

The schedule deliberately puts two light weeks here. Rev-A was released at the end of W8, so the boards are in fab and in transit over exactly the period you would lose to the holidays anyway. Nothing on the critical path waits on you.

What this buffer is for, in priority order:

1. **Rest.** It is a real option, and the plan assumes you take most of it. W9 onwards is the most demanding stretch of the project.
2. **Absorbing slip.** If W1–W8 ran late, finish them here. If Gate 0 sent you back for an extra ramp-up week, that week came out of here — the buffer then shrinks to one week, which still covers the fab time.
3. **Optional read-ahead,** a couple of hours at most: the C++ primer and Vitis HLS example in W10 S3, and a first look at *Debugging: The 9 Indispensable Rules* before bring-up.

What it is not for: starting bring-up early if the boards arrive on 30 December. Bring-up done tired and between family commitments is how a board gets damaged. The boards will keep until 4 January; the bring-up plan in W8 is written so that nothing has to be remembered.

One date to note now. Chinese New Year 2027 falls on 6 February, and PCB fabs and shipping slow down for one to three weeks around it. Any rev-B order must go out by the third week of January, or it slips into March. W9's rev-B list is written in time for that — decide in W9 S5 whether a rev-B order is worth placing before the deadline.

Month 3 · Prove: bring-up, real-time control, evidence, safety

W09 · Board bring-up and the rev-B list (12 h)

Competency: methodical bring-up and root-cause. "It works now" is not a finding. **Prerequisites:** boards in hand; docs/bringup-plan.md printed on paper; bench prepared in W8 S6. **The rule for the week:** follow the plan **literally**, in order, and write the measured value next to the expected value for every single line before moving on. The temptation to skip to "does the arm move" is the exact instinct this week exists to train out of you.

If the boards are late, swap this week with W10 as the risk register says. That is a planned response, not a slip.

S1 · Stages 0–2: inspection, bare power, over-voltage cutoff (2 h)

1. **Stage 0, inspection.** Under magnification, go component by component against the layout: orientation of every polarised part, every IC's pin 1, any bridged fine-pitch pin, any missing part. Photograph anything that looks wrong before touching it.
2. **Continuity:** every rail to ground with a meter. Anything below about $10\ \Omega$ is a short and stops the stage. Also check the follower rail to the leader rail and 12 V to 3.3 V — a solder bridge between rails is the failure that destroys the most parts at once.
3. **Stage 1, bare power.** 12 V in from the bench supply, **limit 500 mA**, nothing else connected — no PYNQ, no arms. The low limit is the whole point: if there is a fault, the supply finds it instead of the board.
 - Record input current. Tens of milliamps is healthy.
 - Measure 12 V at its test point.
 - If the supply goes into limit: turn it off, and go find the short. Do not raise the limit "to see". Raising the limit is how a diagnosable fault becomes a dead board.
4. **Stage 2, over-voltage cutoff.** Still no PYNQ and no arms — a power resistor on each switched rail as a dummy load. This is the most important stage on the board, because it is the one that protects twelve servos at once.
 - Ramp the input slowly from 12.0 V and record the trip voltage; ramp down and record the release. Compare both with the W4 worst-case range and the W7 breadboard numbers.
 - Step the supply from 12 V to 14 V (or use a second supply and a switch) with the scope on a switched rail, and measure the **peak voltage the rail reaches before the cutoff acts** and the time to fall. The peak must stay below 12.6 V. If it does not, the board is not safe to connect to servos, and that is a rev-B blocker — log it and stop.
 - Measure static ripple on the input rail with the scope **AC-coupled, 20 MHz bandwidth limit, spring ground**. Without the bandwidth limit and the short ground you will measure probe pickup and conclude your rail is noisy.

5. Write every number into the printed table as you go, then transcribe into `docs/bringup-report.md` at the end of the session.

Done when: stages 0–2 are complete with measured values recorded, or a fault is found and logged as an ECO.

S2 · Stages 3–4: load switches and the safety chain (2 h)

The safety chain gets tested **before** any servo is connected. Testing it afterwards means testing it with hardware at risk.

1. **Stage 3.** Assert `RAIL_HOLD` manually — a jumper to the appropriate level, or the PL driving it once the PYNQ is mated. Verify both rails switch on and off, and that they switch *independently of each other only if that was your design intent*. Measure the switched rail voltage against the unswitched one; the drop across the FET at low current should be negligible.
2. Measure the **turn-on ramp** on each rail with the scope. Compare with the ramp you designed in W7 S1. A much faster ramp than intended means your gate capacitor is not doing what you think, and it means the inrush protection is not there.
3. **Stage 4, e-stop.** Press it. Both rails must drop. Measure the time from contact opening to the rail falling below a stated threshold — pick a threshold and use the same one everywhere (for example, below 1 V). Record it per rail.
4. **Stage 4, watchdog.** With the PL toggling the watchdog pin, confirm the rails are held. Stop the toggle — in software, then by clearing the bitstream, which is the more realistic failure — and measure the drop-out time each way. Compare with the W7 S6 breadboard measurement and with REQ-021.
5. Clearing the bitstream is the interesting test, because that is what happens when someone runs `overlay()` while the arms are energised. If the rails do not drop, you have found a real safety hole; log it as an ECO with a high classification and decide whether it is a rev-B change or a procedural control.

Done when: e-stop and watchdog drop-out times are measured for both rails, in both the software-stop and bitstream-clear cases.

S3 · Stages 5–7: buses, one servo, then six (2 h)

1. Mate the shield to the PYNQ-Z2. Check the alignment physically before pressing down, and check that nothing fouls the tall connectors — the 1:1 paper check in W8 predicted this, now verify it.
2. **Stage 5, loopback.** With no servos attached, loop each bus back on itself and transmit a packet. You should receive your own bytes on both buses. This tests the buffers, the `DIR` logic, the series resistors and the connectors with nothing at risk.
3. Compare the bus edges on the **shield** against the W2 breadboard and far-end captures. This is the payoff measurement of the whole PCB: rise time, ringing amplitude, and the cleanliness of the turnaround should all be visibly better. Capture and save all three side by side — that figure belongs in the W13 review.
4. **Stage 6.** Rail check. One follower servo. Ping, read position with the joint moved by hand, then a 100-count move. Watch `ERR_CNT` throughout; any increment is a stop condition, not a curiosity.
5. **Stage 7.** Six servos. Ping all six, then a sync-write coordinated move. Confirm on the scope it is one 26-byte packet.
6. Run ten minutes of continuous six-servo traffic and record the error counters. Zero errors is the expectation; anything else is characterised now, not later.

Done when: six follower servos run from the shield with zero bus errors over ten minutes, and the three-way edge comparison figure is saved.

S4 · Stage 8 and root-cause discipline (2 h)

1. **Rail check, and bus identity.** 12.0 V on the display. The leader goes on the `LEADER` port — both ports carry the same voltage on the same connector, so the only things distinguishing them are your silkscreen, your cable flags and the driver's model check. Confirm the driver refuses to enable torque if you deliberately plug the leader into the follower port first (arms clear, torque not yet enabled). Then connect it correctly and read all six positions.
2. Run the PS-side teleop loop over the shield. Both arms, your own board, your own RTL.
3. **Now handle the anomalies.** By now you will have a list. Work each one with this discipline:
 - **Reproduce** it deliberately before investigating. An intermittent you cannot trigger is not yet a bug, it is a rumour.
 - **Classify** it: design, assembly, or firmware. That classification determines who fixes it and when, and it is the first thing a reviewer asks.

- **Root-cause** it before patching. Write down the mechanism, not the symptom. "Bus errors on joint 6" is a symptom; "the pull-up is too weak for the loaded chain, so the idle level sits below V_IH at the far end" is a cause.
 - **Then** decide the disposition: rev-B change, bodge on this board, software workaround, or no action with a reason.
 - **Log it in** `docs/eco.md` with all of the above.
4. Resist the two failure modes of bring-up: patching without root-causing, and root-causing forever without dispositioning. Timebox each anomaly; if it is not understood in 45 minutes, log what you know, note the next experiment, and move on.
5. A useful sanity technique when stuck: **the second board**. Assemble or power the second board and see if the fault follows. A fault on one board is assembly; a fault on both is design. This is why W8 ordered three.

Done when: every anomaly is logged with a classification and a disposition, even the unresolved ones.

S5 - Saturday: thermal survey, the rev-B list, and the report (4 h)

1. **Stage 9.** Ten minutes of continuous teleop with the follower doing real work — moving under load, not idling. Then survey temperatures immediately, before anything cools:
 - Both shunts
 - The over-voltage cutoff (eFuse or comparator-driven FETs)
 - Both load-switch FETs
 - The bus buffers
 - The NTC's reading, compared against your IR measurement at the same spot — that comparison calibrates the NTC and makes W12's over-temperature trip meaningful
2. Record ambient alongside every reading; a temperature without an ambient is not a measurement. Compute the rise above ambient for each part and compare against the derating you applied in W7. A thermal image of the powered board is the single most convincing figure in a bring-up report — take one if you have the camera.
3. **Ripple under motion.** Measure both rails' peak-to-peak ripple while the follower is moving, AC-coupled and 20 MHz limited. Compare with the static measurement from S1 and with REQ-012's droop limit. The dynamic number is the one that matters and it is the one people forget to take.
4. **Write the rev-B list.** Go through `docs/eco.md` and pull out everything dispositioned as a rev-B change. For each, state the change and the reason in one line. Then order the list by whether it is a **safety**, **function**, or **convenience** change — that ordering is what a change-control board would ask for.
5. **Mark up the schematic.** Annotate the KiCad schematic with the rev-B changes visibly marked (a coloured text layer or a dedicated notes block). A rev-B list that lives only in a markdown file gets lost; one marked on the schematic is in front of you the next time you open it.
6. **Write `docs/bringup-report.md` properly.** Structure:
 - Board identity: rev, serial, assembly date, which of the three boards this is
 - Measured vs expected, every test point, every stage — the pre-written table filled in
 - The three-way bus edge comparison figure
 - Thermal survey with ambient and rise
 - Anomaly log: symptom, reproduction, root cause, classification, disposition
 - Rev-B list, ordered by safety / function / convenience
 - One paragraph: what surprised you. That paragraph is what makes it a report written by an engineer rather than generated by a script.

Done when: the report is complete including the "what surprised me" paragraph, and the rev-B list is marked on the schematic.

Measurements to record

Quantity	Method	Against
Every test point, every stage	Meter / scope, paper checklist	Expected values from W7 schematic
Rail ripple, static and under motion	Scope AC, 20 MHz, spring ground	REQ-012
Over-voltage trip, release, and rail peak on a 12→14 V step	Scope, dummy load	REQ-011, W7 breadboard
E-stop time to rail-off, both rails	Scope	REQ-020
Watchdog drop-out: software stop and bitstream clear	Scope	REQ-021, W7 breadboard number
Bus edges: breadboard vs far-end vs shield	Scope, three captures	W2 baseline
Bus error counters over 10 min	ERR_CNT	Zero expected
Component temperatures and rise over ambient	IR / thermal camera	W7 deratings
NTC reading vs IR at the same spot	Both	W12 trip calibration

Deliverables

-  docs/bringup-report.md — complete, measured vs expected, surprises paragraph
-  docs/eco.md — every anomaly with root cause, classification, disposition
-  Rev-B list, ordered, and marked on the schematic
-  Three-way bus edge comparison figure
-  Thermal survey with ambient

Failure modes

Symptom	Likely cause	Fix
Supply goes to limit at stage 1	Short, or reversed polarised part	Find it at 500 mA; never raise the limit to "see"
Cutoff trips at nominal 12 V	Divider tolerance, reference error, or too little hysteresis	Measure the comparator or OVP node; compare with the W4 tolerance stack
Rail peak exceeds 12.6 V on a supply step	Cutoff too slow for the edge	Rev-B blocker; faster part or a clamp stage — do not connect servos meanwhile
Driver enables torque on the wrong arm	Model check missing or reading the wrong address	Fix before any further motion; it is the REQ-013 control
Ripple looks terrible	Long probe ground, no bandwidth limit	Spring ground, 20 MHz limit, remeasure
Rails stay up when the bitstream is cleared	Watchdog holds on a level, not an edge	Serious finding — ECO it, decide hardware vs procedure
Bus errors only with six servos	Loading, pull-up, or termination	Compare edges at the far end; this is a real design finding
Fault present on one board only	Assembly	Use the second board to separate design from assembly
Everything works, nothing recorded	Skipped the paper checklist	The checklist <i>is</i> the deliverable

W10 · Hardware teleoperation: leader to follower without the PS (14 h)

Competency: real-time architecture and PS-versus-PL partitioning, argued with measurements. **Budget note:** 14 h — five sessions plus one extra 2 h block. **The claim you are proving:** the Linux side can be slow and jittery without the arms noticing. Everything this week is in service of measuring that claim rather than asserting it.

S1 · The loop timer and the scheduler state machine (2 h)

- Loop timer.** A down-counter from `CONFIG.loop_period`, at 100 MHz. For the 250 Hz target from REQ-001, that is $100\text{e}6 / 250 = 400_000$ clocks, a 4.000 ms period. Make the period a register so you can sweep the rate in S5 and find the real ceiling.
- Scheduler state machine** in `rtl/teleop_sched/`. One iteration does:
 - Issue six leader reads (round-robin, one at a time — the bus is serial)
 - Pass the six positions through the mapping block (S2)
 - Issue one follower sync-write of six goals
 - Issue one follower read in round-robin, so a full set of follower telemetry accumulates over six iterations
- The round-robin telemetry read is the design decision worth writing down: reading all six follower joints every iteration costs 1.8 ms and blows the budget, whereas one per iteration costs 300 μ s and gives you a complete refresh at $250/6 \approx 42$ Hz. The PS does not need 250 Hz telemetry; the control loop does. Record it as `docs/decisions/0010-telemetry-round-robin.md`.
- Overflow handling.** If an iteration is not finished when the timer expires, do not start a second one on top of it. Count the overrun in a status register and skip. An overrun counter that stays at zero is evidence; a loop that silently overlaps itself is a mystery generator.
- Add the scheduler's registers to the ICD at the `0x2000` base reserved in W4: enable, loop period, mapping-block base, overrun count, iteration count, and a per-joint status.

Done when: the scheduler runs in simulation against the W5 servo model, issuing the right sequence at the right period, with the overrun counter working.

S2 · Mapping, limits, and the BRAM snapshot (2 h)

1. **Per-joint scale, offset and limit block.** For each of the six joints: `follower_goal = clamp(offset + (leader_pos * scale) >> shift, min, max)`. Keep it fixed-point and keep the arithmetic trivially reviewable — this block sits between a human's hand and a motor, and clever arithmetic here is a liability. Positions are **signed** multi-turn values on HX servos; carry them signed through the block, and use the W3 finding on the 0/4095 seam to decide whether any joint (wrist roll, most likely) needs wrap handling here.
2. The **limit** is a safety function, not a convenience. It must clamp in hardware, it must be settable only from the PS, and it must default to a safe narrow range on reset rather than to full travel. Write down what the reset default is and why. The HX servos also hold their own min/max angle limits (the W2 screenshots); set yours inside theirs, so the servo's limit is a backstop rather than the thing that actually stops the arm.
3. Add a per-joint **enable** so you can bring up one joint at a time in S5 without rewiring anything. Bring-up granularity designed in advance is what makes the hardware session fast.
4. **BRAM snapshot.** A dual-port BRAM the scheduler writes and the PS reads over AXI in one burst: six leader positions, six follower positions, six speeds, six loads, plus iteration count, overrun count and error counters.
 - The PS reading a block the PL is writing needs a coherency story. The simplest correct one: double-buffer, with the scheduler writing buffer A while the PS reads buffer B, and a register telling the PS which is current. Alternatively, a sequence counter written before and after each update, which the PS checks for equality. Pick one, implement it, and be able to explain it — this is interview question 4's territory and "I just read it and hoped" is a bad answer.
5. Update the ICD to v0.3 with the scheduler and snapshot regions.

Done when: the mapping block is verified in simulation across its clamp boundaries, and the snapshot coherency scheme is implemented and documented.

S3 · The HLS trajectory filter (2 h)

C++ is a third language, but a small one here. HLS uses a restricted subset of C++: functions, fixed-size arrays, `for` loops, integer and `ap_fixed` types — no dynamic memory, no classes you need to write. Before starting, spend 45 minutes on a C++ basics tutorial (variables, functions, loops, arrays, header files) and build one of the Vitis HLS example projects unchanged, reading its synthesis report. Your Python and Verilog now carry most of the weight: the filter's logic is the same as a few lines of Python, and its report reads like the Vivado reports you know from W6. The whole kernel is about thirty lines.

1. Create the Vitis HLS project under `rtl/hls_traj/`. The filter takes a leader position stream and produces a follower goal stream, doing two things:
 - **Rate limiting:** `out = prev + clamp(target - prev, -max_delta, +max_delta)`, where `max_delta` is the maximum change per loop iteration. At 250 Hz, `max_delta` in counts per iteration converts directly to a joint speed limit — compute that conversion and state the resulting deg/s, because that is the number a human understands.
 - **Smoothing:** a first-order IIR, `y += (x - y) >> k`, with `k` a runtime parameter. A shift-based IIR is cheap and its behaviour is obvious.
2. Use `ap_fixed` with a width and integer-part chosen from the 0–4095 position range plus headroom for the filter state. Write down the format you chose (for example `ap_fixed<20,14>`) and why — quantisation in a rate limiter shows up as a joint that creeps.
3. **Justify the filter against the servo's own.** HX servos already apply a trapezoidal acceleration profile and PID internally, set by their Accel and Speed parameters. So the honest question is not "does the HLS filter smooth the motion?" but "does it add anything the servo's profile does not?" Plan the S5 experiment now: the same leader jerk with (a) the HLS filter on and the servo profile at its fastest, (b) the filter bypassed and the servo profile tuned slower, (c) both. Possible answers include that the PL filter bounds the *commanded* step (which the servo's profile cannot see coming), makes the shaping identical across all six joints, and keeps it under your control rather than in servo firmware. If the measurement shows it adds nothing, say so in `docs/decisions/` — a component removed on evidence is a better story than one kept on faith.
4. When teleop is off, the same block **interpolates PS-sent waypoints**, so the PS can command motion through exactly the path the leader would have used. One block, two sources, selected by a register.

5. Pragmas: `#pragma HLS PIPELINE II=1` on the per-joint loop, `s_axilite` for the parameters, and a stream or array interface for the joint data. Keep the interface simple; an HLS block with an elaborate interface is hard to integrate and harder to review.
6. **Read the HLS report and understand it**, do not just check it built:
 - Latency and initiation interval in cycles, converted to microseconds at 100 MHz
 - LUT, FF, DSP and BRAM usage
 - Which loop, if any, failed to achieve $II=1$, and why
7. Write those numbers into `docs/performance.md`. Being able to say "the filter adds 1.2 μ s and 3 DSPs" is what makes the HLS decision defensible; "I used HLS" is not.

Done when: the filter synthesises with a report you can explain line by line, and its latency in microseconds is written down.

S4 · Integration, build, and timing (2 h)

1. Add the scheduler, the mapping block and the HLS IP to the block design in `overlay/v1.0/build.tcl`. Everything stays scripted.
2. **Re-allocate the single ILA core.** The Basic tier allows only one, so it moves from W6's bus signals to the scheduler: probe 0 `{loop_tick, dir_follower, dir_leader}` as the trigger, probe 1 the scheduler state, probe 2 the iteration counter, probe 3 the overrun counter, probe 4 the joint currently being read. Keep W6's allocation in a Tcl variable so you can switch back with one line when you need to debug the bus again. You will want it in S5.
3. Constrain the **loop-tick GPIO** to a Pmod pin for the scope. This pin is your primary instrument for the week.
4. Build. Re-run the timing and utilisation reports and **compare with W6's numbers** — the design has grown by a scheduler, a mapping block and an HLS kernel, so the interesting figure is the delta. Archive it against the commit hash as before.
5. If WNS goes negative here, it is most likely the mapping block's multiply-and-clamp chain or the HLS block's interface. Pipeline it and record the change.

Done when: `overlay/v1.0` builds with positive WNS, and the utilisation delta from W6 is recorded.

S5 · Saturday: bring it up, then measure the claim (4 h)

1. **Bring up incrementally**, using the per-joint enable from S2. Rail check first, arms clear, hand near the e-stop.
 - Scheduler enabled, **all joints disabled**: confirm the loop tick on the scope at the expected period, and confirm the leader reads are happening on the bus, with nothing being commanded.
 - **One joint enabled**, with a deliberately tight limit range. Move that leader joint slowly. Confirm the follower joint tracks within the clamp.
 - Widen the limits for that joint. Then enable joints one at a time.
 - Finally all six, with the PS doing nothing but watching the snapshot.
2. **Measure loop period and jitter.** Scope on the loop-tick pin, persistence or histogram mode, over at least 10,000 ticks. Record mean period, and jitter as peak-to-peak and as p99. Compare against REQ-003. This is the measurement that supports the "hardware scheduler" claim, so take it carefully and save the screenshot.
3. **Measure leader-to-follower latency in hardware.** Same method as W3 and W6 so the numbers are comparable: scope on both bus lines, from the leader read packet that first carries the moved position to the follower write packet that carries the new goal. Twenty captures, median and worst case.
4. **Prove the claim about the PS.** This is the experiment the whole week exists for, and it is easy to skip:
 - Run the hardware teleop loop and load the PS heavily — compile something, run a busy loop on both cores, copy a large file.
 - Measure loop jitter and teleop latency again under that load.
 - Then do the same with the **W6 PS-in-loop** overlay for comparison.
 - The PS-in-loop numbers should degrade badly and the PL numbers should not move. That contrast, in one table, is the strongest result in your entire design package. If the PL numbers *do* move, find out why — it means something in your loop still depends on the PS, and finding that is even more valuable.
5. **Run the filter experiment from S3:** the same sharp leader input under the three filter/profile combinations, follower trajectory logged each time. Write the result into `docs/decisions/0013-hls-filter-vs-servo-profile.md`.
6. **Measure the waypoint path too:** worst-case latency from the PS writing a waypoint to the first byte on the follower bus. This is the number that matters for W13's policy inference, where the PS is in the loop by necessity.
7. Build the comparison table in `docs/performance.md`:

Path	Median latency	p99	Loop jitter	Under PS load
USB + Python (W3)			n/a	
PL bus, PS in loop (W6)				
PL hardware loop (W10)				

Every cell filled from a measurement, each with its capture file named.

Done when: the three-path table is complete, including the under-load column, and the jitter screenshot is saved.

S6 · Extra block: the video stretch, or the write-up (2 h)

Take the stretch only if S5 finished cleanly. The write-up is not optional.

Stretch: capture wrist-camera frames on the PS with V4L2, push them through an HLS Sobel or colour-threshold kernel over AXI DMA, and compare frames per second against the same filter in OpenCV on the A9.

1. Build the HLS kernel with AXI-Stream in and out, and a DMA in the block design.
2. Measure both paths at the same resolution: frames per second, and CPU load during each.
3. Be honest in the write-up about the DMA and copy overhead. For a single cheap filter at 640×480 the PL path frequently **loses**, because you pay two memory copies to save one cheap operation. Publishing a negative result with the crossover analysis — at what resolution, frame rate or operation count does PL win? — is a better artifact than a rigged win.
4. Write `docs/decisions/0011-video-path.md` with the measurement and the criterion from W4 S2 that it tests.

Either way: finish `docs/performance.md` with the three-path table, the jitter numbers, the HLS resource and latency figures, and a paragraph on what the latency is actually limited by. That last paragraph answers interview question 7, and the answer is usually "the serial bus itself at 1 Mbps" — at which point the follow-up, "what would it take to halve it?", has a real answer: a faster bus rate if the servos support it, or splitting the six servos across more buses so reads happen in parallel. You have a two-bus design and a parameterised `n_bus` top level, so you can say exactly what that would cost.

Done when: `docs/performance.md` is complete with the limiting-factor paragraph.

Measurements to record

Quantity	Method	Against
Loop period mean and jitter (p2p, p99), $n \geq 10\,000$	Scope on loop-tick	REQ-003
Leader-to-follower latency, hardware, $n=20$	Two-channel scope	W3 and W6 baselines, REQ-004
All three paths under heavy PS load	Repeat both above	The central claim
Waypoint to first bus byte, worst case	Scope	W13 policy inference
HLS latency (cycles $\rightarrow$ μ s), LUT/FF/DSP/BRAM	HLS report	<code>docs/performance.md</code>
Utilisation and WNS delta vs W6	Vivado reports	Budget
PS vs PL frames per second (stretch)	Both paths, same resolution	<code>0011-video-path.md</code>

Deliverables

- `overlay/v1.0/` — hardware teleop loop, scripted build
- `rtl/teleop_sched/`, `rtl/hls_traj/` with the HLS report committed
- `docs/performance.md` — three-path table, under-load column, limiting-factor paragraph

-  docs/decisions/0010-telemetry-round-robin.md , 0013-hls-filter-vs-servo-profile.md , and 0011-video-path.md if attempted
-  docs/icd.md v0.3

Failure modes

Symptom	Likely cause	Fix
Follower jumps on enable	No rate limit applied on the first iteration	Initialise the filter state from the follower's current position, not zero
Loop tick period correct, latency poor	Scheduler waits a full iteration before acting on a fresh read	Reorder: read leader, then write follower, in the same iteration
Overrun counter climbing	Period too short for the transaction set	Lengthen the period, or drop a transaction; do not ignore it
PS snapshot reads show impossible values	No coherency scheme	Double-buffer or sequence-counter, as designed in S2
PL loop degrades under PS load	Something still depends on the PS	Excellent finding — trace it and write it up
Joint creeps slowly with the leader still	Quantisation in the fixed-point filter	Widen <code>ap_fixed</code> , or add a deadband; record the format change
A joint whips almost a full turn	Position crossed the 0/4095 seam and the mapping treated it as a huge step	Handle wrap in the mapping block, per the W3 seam finding

Gate 3. Overlay v1.0 teleoperates the follower from the leader entirely in PL on your own rev-A board, at the W4 target rate, with a latency table that compares the hardware loop against the PS-in-loop and USB paths.

W11 · The characterisation bench, built on your ATE habits (12 h)

Competency: design verification ownership — turning a bench into evidence against numbered requirements, with an independent measurement to back the self-reported one. **Prerequisites:** overlay v1.0 running; OBSBOT on a tripod; printed ArUco marker; bench supplies on SCPI. **The principle:** an encoder can only report what the servo *believes*. The OBSBOT is there to disagree with it. Where they agree, you have a number you can defend; where they disagree, you have the most interesting finding of the week.

S1 · The pytest harness and instrument fixtures (2 h)

1. Build `test/bench/conftest.py` with session-scoped fixtures, in the same shape as your RF ATE sessions:

```
import pytest, datetime, pathlib, pyvisa

@pytest.fixture(scope="session")
def session_dir():
    d = pathlib.Path("data") / datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
    d.mkdir(parents=True)
    return d

@pytest.fixture(scope="session")
def psu_12v():
    rm = pyvisa.ResourceManager()
    psu = rm.open_resource(PSU_12V_ADDR)
    psu.write("*RST")
    psu.write("CURR 5.0")          # limit before voltage, always
    psu.write("VOLT 12.0")
    psu.write("OUTP ON")
    yield psu
    psu.write("OUTP OFF")

@pytest.fixture(scope="session")
def arm(psu_12v, psu_5v):
    ol = Overlay("overlay/v1.0/teleop.bit")
    ...
    yield handle
    handle.torque_off_all()      # teardown always de-energises
```

2. Three habits carried over from ATE, each worth stating explicitly in the README:

- **DUT identity** recorded at session start: board serial, git hash of the overlay, servo IDs and their firmware revisions, ambient temperature. A result without a DUT identity cannot be compared with a later result.
- **Timestamped data directory** per session, raw CSV written as it is measured, never only at the end. A crashed run should still leave its data.
- **A log file per session** capturing instrument setup commands and any exception. When a result looks wrong three weeks later, the log is what tells you whether the supply was configured correctly.

3. Teardown must **always** de-energise, including on test failure. Use fixtures, not `try/finally` scattered through tests.

4. Write the naming convention to match your existing DUT compensation files so the two bodies of data live together:

`{serial}_{test}_{date}.csv` .

Done when: `pytest test/bench -k smoke` runs, powers up, records identity, powers down, and leaves a timestamped directory with a log.

S2 · The optical instrument: calibration and pose (2 h)

An uncalibrated camera is not an instrument. Do this properly or the "independent measurement" claim collapses.

1. Calibrate the OBSBOT. Print a chessboard, mount it flat and rigid, capture 20 views across the frame and at varied angles:

```
import cv2, numpy as np, glob
CB = (9, 6); SQ = 0.025          # inner corners, metres
objp = np.zeros((CB[0]*CB[1], 3), np.float32)
objp[:, :2] = np.mgrid[0:CB[0], 0:CB[1]].T.reshape(-1, 2) * SQ
objpoints, imgpoints = [], []
for f in glob.glob("calib/*.png"):
    img = cv2.imread(f); g = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    ok, c = cv2.findChessboardCorners(g, CB, None)
    if ok:
        c = cv2.cornerSubPix(g, c, (11,11), (-1,-1),
                               (cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_MAX_ITER, 30, 1e-3))
        objpoints.append(objp); imgpoints.append(c)
rms, K, dist, _, _ = cv2.calibrateCamera(objpoints, imgpoints, g.shape[::-1], None, None)
print("reprojection RMS (px):", rms)
np.savez("calib/obsbot.npz", K=K, dist=dist, rms=rms)
```

Record the **reprojection RMS**. Below about 0.5 px is good; above 1 px means recapture with better coverage. That number is your camera's stated accuracy and it belongs in the report.

2. **Marker detection and pose**, using the current OpenCV API — `estimatePoseSingleMarkers` was deprecated, so use `solvePnP` with explicit object points:

```
d = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
det = cv2.aruco.ArucoDetector(d, cv2.aruco.DetectorParameters())
S = 0.040 # marker side, metres — measure it
half = S/2
obj = np.array([[-half, half, 0], [half, half, 0], [half, -half, 0], [-half, -half, 0]], np.float32)

corners, ids, _ = det.detectMarkers(gray)
ok, rvec, tvec = cv2.solvePnP(obj, corners[0], K, dist, flags=cv2.SOLVEPNP_IPPE_SQUARE)
```

3. **Characterise the instrument before you use it.** Clamp the marker so it cannot move, capture 500 frames, and compute the standard deviation of the reported position. That is your optical noise floor. If it is larger than the repeatability you are trying to measure, the optical measurement cannot resolve the thing you want it to, and you must fix that first — better lighting, a larger marker, a closer camera, or frame averaging.
4. Fix the tripod position and **mark it on the floor with tape**. Every optical result is relative to this pose; moving the camera mid-campaign invalidates comparisons.
5. Record: camera intrinsics file, reprojection RMS, marker size measured with calipers, optical noise floor in mm, camera-to-arm distance.

Done when: the optical noise floor is measured and is comfortably smaller than the encoder resolution referred to the gripper.

S3 · Per-joint tests, part one: repeatability and backlash (2 h)

Test the vendor's claims, not just the servo. Hiwonder specifies the HX-30HM with a 12-bit magnetic encoder, **0.3° accuracy**, and describes it as **zero backlash**. Write those three claims at the top of the report and make this week's data answer each one directly, with the optical instrument as the referee. A characterisation report that confirms or refutes a datasheet claim with independent evidence is exactly the kind of artifact a senior reviewer remembers.

1. **Resolution reference.** 4096 counts over 360° is 0.088° per count. Referred to the gripper at radius r , one count is $r \cdot 0.088^\circ \cdot \pi/180$. Compute it for your arm — it is the yardstick every repeatability number is judged against.
2. **Repeatability, 200 cycles, both instruments simultaneously.** For each joint: move away to a fixed excursion, return to the same commanded target **from the same direction**, settle, then record both the encoder position and the optical position.

```
def repeatability(joint, target, away, n=200):
    enc, opt = [], []
    for _ in range(n):
        goto(joint, away); settle()
        goto(joint, target); settle()
        enc.append(read_position(joint))
        opt.append(optical_position())
    return np.array(enc), np.array(opt)
```

Report **standard deviation and 3σ** , in counts, in degrees, and in millimetres at the gripper, for both instruments. Three units because three different audiences read the report.

3. **Backlash.** Approach the same target from both directions and take the difference. On a magnetic-encoder servo the *encoder* backlash should be near zero — the encoder sits on the output — so any difference the encoder shows is interesting, and any difference only the optics show is backlash in the horn, the printed parts or the joint, not the servo. That split is the answer to the vendor's zero-backlash claim. Do it ten times per direction and report the mean and spread. Compare against your W2 hand estimate — that comparison is satisfying and it validates the W2 survey as a method.

4. The **encoder will look better than the optics**, always. The gap is the sum of the optical noise floor, the marker mounting compliance, and the real mechanical error the encoder cannot see — gear backlash downstream of the encoder, horn slip, frame flex. Decompose it as far as you can: you measured the optical noise floor in S2, so subtract it in quadrature and see what remains.
5. Save raw per-cycle data, not just statistics. W13 reviewers ask to see the distribution.

Done when: joint 1 has a complete repeatability and backlash result from both instruments, with the units converted three ways.

S4 · Per-joint tests, part two: dynamics, current, temperature, bus health (2 h)

1. **Record the servo's motion profile first.** HX servos shape every move with their own Accel and Speed parameters. Read and record both for every joint before any dynamic test, and hold them fixed for the whole campaign — a step response is meaningless without the profile that produced it. Run the step tests twice: once at the kit's settings, once at the fastest profile the servo allows, so you can separate the servo's control loop from its trajectory generator.
2. **Step response.** Reconfigure the scheduler to read a **single** joint every iteration, giving you 250 Hz sampling on that joint rather than the round-robin's 42 Hz. This is exactly why the scheduler's read target is a register, and it is worth noting in the report as a design decision that paid off.
 - Command a step of a defined size, log position at 250 Hz, then compute **10–90 % rise time, overshoot %, and settling time to ± 2 %**.
 - Repeat for three step sizes — small, medium, large. The servo's internal control is nonlinear and a single step size tells you almost nothing.
3. **Current versus holding load.** With the INA226 on the follower rail, hold the joint at a fixed position and add known loads (a set of known masses at a known radius gives you torque). Plot current against torque. The slope is effectively the motor constant as seen at the rail, and the intercept is the quiescent overhead.
4. **Temperature rise over a 30-minute duty cycle.** Define the duty cycle precisely (for example: a 60° move every 2 s at 50 % of maximum speed) and log servo-reported temperature plus rail current throughout. Report the rise above ambient and the time constant. State the duty cycle in the report — a temperature rise without its duty cycle is meaningless.
5. **Bus error rate over an hour.** Run continuous traffic and log `ERR_CNT` and the transaction count. Report as **errors per million transactions**, separated into checksum errors and timeouts, because they have different causes. Run it on both buses.
6. Each of these is a separate pytest test with its own CSV output and its own pass/fail limit drawn from `docs/requirements.md`. A test without a limit is a measurement, not a verification.

Done when: all five test types run on joint 1 and produce CSVs with pass/fail evaluated against requirements.

S5 · Saturday: teleop tracking, the full campaign, and the datasheet (4 h)

1. **Teleop tracking test.** Move the leader through a defined trajectory — ideally a repeatable one, so drive the *leader* from a recording rather than by hand if you can. Log leader position and follower position at the loop rate, then compute:
 - **Tracking error** over time: RMS and maximum, per joint
 - **Effective lag**, by cross-correlating the two traces and finding the peak offset. Compare that lag with the W10 scope-measured latency. If they disagree, one of them is wrong and finding out which is worth the time.
 - Plot leader and follower overlaid, plus the error trace beneath.
2. **Run the full suite on all six follower joints.** This is mostly waiting, so use the time to draft the report's narrative sections.
3. **Report generation, automated.** A script that reads the session's CSVs and emits `docs/characterisation-report.md` with one page per joint:
 - Joint identity, servo ID, firmware revision
 - The three vendor claims — resolution, 0.3° accuracy, zero backlash — each marked confirmed, refuted or not resolvable, with the evidence
 - Servo motion-profile settings used
 - Repeatability: encoder vs optical, σ and 3σ , three units, with the distribution plotted
 - Backlash, both methods
 - Step response table across three step sizes
 - Current versus torque plot
 - Temperature rise with the duty cycle stated

- Bus error rate
- Pass/fail against each relevant requirement ID

4. The **encoder-versus-optical** section is the report's centrepiece. Write it as an argument, not a table:

- Where they agree, and within what bound
- Where they disagree, by how much, and your explanation
- Which you believe for which purpose — the encoder for control, the optics for truth about where the gripper actually is
- What you would do to resolve the remaining disagreement with more time That section is interview question 8 answered in advance, in your own numbers.

5. Commit the raw data. All of it. A characterisation report whose raw data is not in the repo is a claim, not evidence.

Done when: the report generates from committed raw data with one command, and the encoder-versus-optical argument is written.

Measurements to record

Quantity	Units reported	Limit from
Repeatability σ and 3σ , encoder and optical	counts, degrees, mm at gripper	Requirements, vendor's 0.3°
Servo Accel / Speed settings per joint	register values	Required context for every dynamic result
Backlash per joint, both directions	counts and mm	W2 hand survey for comparison
Rise time, overshoot, settling, $\times 3$ step sizes	ms, %, ms	Requirements
Current vs holding torque	A vs N·m, slope and intercept	W2 table
Temperature rise over a stated 30-min duty cycle	°C above ambient, time constant	W7 deratings
Bus error rate, per bus	errors per 10^6 transactions, split by type	Zero expected
Tracking RMS and max error, effective lag	counts, ms	W10 latency
Optical noise floor, reprojection RMS	mm, px	Instrument qualification

Deliverables

-  test/bench/ — pytest suite with instrument fixtures and always-de-energising teardown
-  docs/characterisation-report.md — one page per joint, auto-generated
-  The encoder-versus-optical argument, written out
-  Raw CSVs and calibration files committed
-  Camera intrinsics with reprojection RMS, optical noise floor stated

Failure modes

Symptom	Likely cause	Fix
Optical result noisier than the effect measured	Uncalibrated camera, small marker, poor lighting	Fix the instrument before trusting any result
Repeatability improves after the first 20 cycles	Mechanical settling or thermal warm-up	Add a warm-up phase and say so in the method
Step response looks identical at every step size	Sampling too slow	Single-joint read mode at 250 Hz
Temperature result not reproducible	Duty cycle not defined	Define it numerically, state it in the report
Results differ between sessions	Camera moved, or ambient changed	Tape the tripod; record ambient in DUT identity
Test leaves the arm energised after a failure	Cleanup in the test body	Move de-energising into the fixture teardown

W12 · Safety: FMEA, hardware trips, and fault injection (12 h)

Competency: safety thinking proven with measurements, and the judgement to decide which failures go in hardware versus software. **Prerequisites:** overlay v1.0 on the rev-A board; e-stop and watchdog verified in W9. **Standing rule for the week:** every fault-injection test runs with **the arms clear of everything** — no workpiece, no fixture, nothing within reach. You are deliberately causing failures; assume each one produces the worst motion it could.

S1 · The FMEA (2 h)

- Build the table in `docs/fmea.md` (or a spreadsheet, committed as CSV so it diffs). One row per failure mode, with columns: item, function, failure mode, local effect, system effect, **severity (1–10)**, cause, **occurrence (1–10)**, current detection, **detection (1–10)**, **RPN = S×O×D**, mitigation, and post-mitigation RPN.
- Cover the whole system, not just the board. Seed with these and expand:

Item	Failure mode	System effect
Follower bus	Stuck low	No commands delivered; follower holds last goal indefinitely
Follower bus	Open / cable pulled	Same, plus no telemetry
Leader bus	Cable pulled mid-teleop	Follower keeps last leader position; holds pose
Follower rail	Over-current (jam or collision)	Servo overheats, mechanical damage, fire risk at the extreme
Watchdog	PL hangs, toggling stops	Rails must drop
PS	Linux crash or kernel killed mid-move	Nothing commands the loop; PL must be self-sufficient
Bitstream	Reloaded while arms energised	All PL outputs indeterminate during reload
Thermal	Load switch or servo over-temperature	Derating exceeded, eventual failure
Supply	Brown-out on the 12 V rail	Servos reset mid-move; multi-turn count cleared, so positions after recovery may jump
Supply	Over-voltage — bench left high, wrong adapter on the same barrel plug	All twelve servos above their 12.6 V maximum at once
Connectors	Leader and follower cables swapped	PL commands the leader as the follower; the arm a person is holding moves under power
Mechanical	Joint limit exceeded via a bad mapping	Self-collision or frame damage
Gripper	Commanded closed on a hand	Injury

3. Score honestly. The over-voltage row should score near the top of the severity scale — one fault damages every servo in the kit — and before the W7 cutoff its occurrence is set by nothing more than a label on a supply, which is exactly why the cutoff exists. The cable-swap row is lower severity but high occurrence, because the two connectors are identical; its mitigations are the silkscreen and the driver's model check. Show the **post-mitigation** RPN so the table demonstrates the value of the controls you built.
4. Pick the **top five by RPN** to mitigate this week. State why the others are accepted: already mitigated by design, low severity, or out of scope with a named reason.
5. For each of the top five, decide **hardware or software**, and write the reason. The general principle: anything whose severity is high and whose detection depends on software running correctly belongs in hardware, because the failure modes you most need to catch are the ones where software has already stopped working. That sentence is interview question 9.

Done when: the table is complete with pre- and post-mitigation RPNs, and the top five have a hardware/software decision with reasons.

S2 · Implement the trips, part one: current and watchdog (2 h)

1. **Over-current trip.** The INA226 `ALERT` pin, configured with an over-limit threshold, drives a PL input that opens the follower load switch. Two things to decide and record:
 - The **threshold**, from the W2 and W11 current data: above the legitimate worst case with margin, below anything that damages a servo.
 - The **conversion time and averaging**, which set the trip latency. More averaging is quieter and slower. Compute the expected latency from the datasheet's conversion time and your averaging setting, then measure it in S4 and compare.
2. Implement the PL side: latch the trip, open the switch, set a status bit, and require an **explicit PS acknowledgement to re-arm**. A trip that clears itself is a trip that hides a recurring fault.

3. **Watchdog** kicked by the PS: a register the PS must write within a timeout, whose expiry stops the PL's watchdog toggle and therefore drops both rails via the hardware chain from W7. Note the two-stage structure — PS feeds PL, PL feeds hardware — and that each stage fails safe independently. That layering is worth a diagram in the architecture document.
4. Set the PS-to-PL timeout from a real number: how long can the PS legitimately be busy? Use the jitter you measured under load in W10 S5, with margin.

Done when: both trips are implemented, latched, and require an explicit re-arm.

S3 · Implement the trips, part two: leader silence and servo heartbeat (2 h)

1. **Leader-silence detector.** If leader reads stop arriving — cable pulled, servo dead, bus shorted — the follower must not keep tracking a stale position. Two stages:
 - **Freeze** immediately: hold the current follower goal, stop applying new leader input. Target within 50 ms per REQ-022.
 - **Drop torque** after a longer interval, so the arm does not hold a pose indefinitely under a fault. Choose the second interval deliberately: dropping torque makes the arm fall, which for a loaded arm may be worse than holding. Decide which failure you prefer for *your* arm, and write the reasoning down — this is a genuine safety trade-off with no universal right answer, and having reasoned about it is the point.
2. **The servos' own protection layer.** HX servos carry configurable protection — a current maximum with a hold time, an overload threshold with a reduced torque after tripping, an over-temperature threshold, and under- and over-voltage thresholds — which you recorded from ServoStudio in W2. Treat them as the innermost layer of defence in depth:
 - Set each threshold deliberately, below the factory default where your W11 data justifies it, and record the new values and the reasoning.
 - Change thresholds with ServoStudio on the Windows desktop, or through your own driver on the laptop once W3's register table covers them.
 - Verify each one actually fires, on a **spare** servo on the bench rather than in an arm: force an overload and measure time-to-trip; heat it gently and confirm the temperature trip. Assume nothing works until you have seen it work.
 - Check the Hiwonder protocol document for a command-timeout or "lost communication" behaviour. If one exists, enable and test it — it is the servo-side answer to a silent PL. If none exists, say so in the FMEA: that gap is why the PL leader-silence detector matters.
3. **Bitstream-reload hazard.** From W1 S2 and W9 S2 you know what happens to PL outputs during a reload. Decide the control: a procedural rule (de-energise before loading an overlay), a hardware default (pull-downs that make the load switch open when the PL is not driving), or both. A pull-down that fails safe is worth more than a procedure. If the board needs a rev-B change for this, that is exactly what the rev-B list is for.
4. Add every trip's status to the BRAM snapshot so the PS can see *which* trip fired, and add a counter per trip type.

Done when: all four mitigations are implemented, with per-trip status visible to the PS.

S4 · Fault injection, part one (2 h)

Every injection follows the same four-step procedure, and each produces one number.

Procedure: (1) arms clear, e-stop within reach; (2) scope armed on the relevant signal — usually the switched rail, or the follower bus; (3) inject the fault; (4) measure the time from the fault to the defined response, and record it against the requirement.

1. **Pull the leader cable mid-move.** Scope the follower bus. Measure: last leader read → follower freeze (no new goals). Then measure freeze → torque drop. Against REQ-022.
2. **Pull the follower cable mid-move.** Measure what the system detects and how fast. Note that the follower servos now hold their last goal with no way to command them — write down what that means and whether it is acceptable.
3. **Kill the Jupyter kernel** during teleop. This is the PS-crash proxy. Two things to observe: does the PL loop keep running (it should — that is the whole claim), and does the PS watchdog fire and drop the rails as designed? Both are correct behaviours and you need to state which you *want*. If the PL loop is self-sufficient, should a PS crash really drop the rails? Argue it, decide, and configure accordingly.
4. **Force a current trip** by lowering the threshold in software until normal motion trips it. Measure fault-to-rail-off on the scope. Compare against the latency you computed from the INA226's conversion time in S2. A measured value well above your computed one means your averaging setting is not what you think.

Done when: four faults injected, each with a measured reaction time recorded against its requirement.

S5 · Saturday: the rest of the injections, and the write-up (4 h)

1. **E-stop during a stall.** The worst realistic case: the arm pushing hard, maximum current, then the e-stop. Measure both rails' fall time under load and compare with W9's unloaded measurement. Loaded fall times are longer, and this is the number that actually matters.
2. **Brown-out.** Ramp the supply down slowly and observe what the servos do as the rail falls. Find the voltage at which behaviour becomes unpredictable. This tells you whether you need an under-voltage lockout, which may be a rev-B item.
3. **Over-temperature**, if you can produce it safely: heat the NTC locally (a heat gun at distance, carefully) and confirm the trip fires at the expected temperature, using the NTC-vs-IR calibration from W9.
4. **Over-voltage injection.** Never on the arms. First with the dummy loads from W9, then with **one spare servo** on the follower port and nothing else. Step the supply from 12 V to 14 V and capture the switched rail: the peak it reaches, and the time until it is off. The peak must stay under 12.6 V. That capture is the verification evidence for REQ-011, and it is the single best slide in the W13 safety section.
5. **Cable swap.** Arms clear, torque off. Plug the leader into the follower port and start the driver. It must refuse to enable torque and report which model it found on which bus. Screenshot the refusal — it is REQ-013's evidence.
6. **Bitstream reload with arms energised** — with the arms clear and supported, and the current limits low. Observe what actually happens to the rails and to the servos. Record it. If the answer is bad, that is a genuine finding and it goes to the top of the rev-B list.
7. Write `docs/fault-injection.md` :

Fault	Injection method	Response required	Requirement	Measured	Pass
Leader cable pulled	Physical	Freeze ≤ 50 ms, then torque drop	REQ-022		
Follower cable pulled	Physical	Detect and report			
PS killed	<code>kill -9</code> on the kernel	PL loop continues; watchdog per design	REQ-021		
Over-current	Lower threshold	Rail off			
E-stop under stall	Physical	Both rails off $\leq$ limit	REQ-020		
Brown-out	Supply ramp	Defined behaviour			
Over-temperature	Local heating	Rail off			
Over-voltage	12→14 V step, spare servo only	Rail off with peak < 12.6 V	REQ-011		
Cable swap	Leader on follower port	Torque refused, models reported	REQ-013		
Servo overload	Forced stall, spare servo	Servo's own protection trips	W2 config		
Bitstream reload	<code>Overlay()</code> while live	Fail safe			

8. Add the **safety section** to `docs/architecture.md` : the layered diagram (over-voltage cutoff and hardware chain → PL trips → PS watchdog → servo-internal protection), which layer catches which failure, and the measured reaction time of each. A safety argument with measured numbers attached to each layer is unusual in a portfolio and it will be noticed.

Done when: every row of the table has a measured number and a pass/fail, and the architecture document has its safety section.

Deliverables

-  docs/fmea.md — full table, pre- and post-mitigation RPN, top five selected with reasons
-  Four mitigations implemented in PL with latched status and explicit re-arm
-  docs/fault-injection.md — nine faults, measured reaction times, pass/fail against requirements
-  Safety section in docs/architecture.md with the layered diagram
-  The over-voltage injection capture, and the cable-swap refusal screenshot
-  Any new rev-B items added to docs/eco.md

Failure modes

Symptom	Likely cause	Fix
Trip clears itself	No latch	Latch, require explicit PS re-arm
Trip latency far worse than computed	INA226 averaging set high	Recompute, reconfigure, re-measure
Follower keeps tracking after the leader is unplugged	Silence detector keyed on stale data	Key it on successful reads, not on the last value
PS crash drops rails, but you wanted it not to	Policy not decided	Decide deliberately and configure; either answer is defensible, indecision is not
Cannot produce a fault safely	Test design	Reduce energy: lower current limit, slower speed, supported arm
FMEA rows all score similarly	Scoring not discriminating	Anchor the scales: write what 3, 6 and 9 mean for each of S, O and D

W13 · Final design review, portfolio, and interview readiness (12 h)

Competency: communication and leadership — running a review, defending trade-offs, and teaching the design to others.

Prerequisites: everything. This week assembles rather than creates. **The test:** a stranger, given twenty minutes and your repo, should understand what you built, why, and how well it works.

S1 · Assemble the package and make the repo navigable (2 h)

1. Collect the design package into a coherent order, with docs/README.md as its index:

#	Document	Answers
1	requirements.md	What was it supposed to do?
2	architecture.md	How is it built, and why that way?
3	icd.md	What are the interfaces?
4	hx-bus-spec.md	What does it talk to, and where does it differ from the vendor document?
5	hw/shield/ schematic + layout	What did you build?
6	rtl/ + coverage summary	How do you know the RTL works?
7	bringup-report.md	Did the board work, and what did you find?
8	performance.md	How fast is it, versus what?
9	characterisation-report.md	How good is it, measured two ways?
10	fmea.md + fault-injection.md	How does it fail, and how fast does it catch it?
11	eco.md	What changed, and how was each found?
12	decisions/	Why each choice, and what lost?
13	lessons-learned.md	What would you do differently?

2. Write the **top-level README** for a stranger, not for yourself: what this is, one photograph of the working system, the headline results as a short table, the repo map, and how to rebuild. Put the headline numbers in the first screen — latency versus baseline, loop jitter, repeatability, safety reaction times. Someone scanning for thirty seconds should get the result.
3. Write `docs/lessons-learned.md` from the Sunday logs you have been keeping since R1, and reread `ramp/gate0.md` first — the distance between that half-page and where you are now is a story in itself. Structure it as: what went as planned, what did not, what you would change in the schedule, and the three things you learned that you did not expect to. Honest entries beat polished ones.
4. **Test the navigation.** Open the repo in a fresh browser window and try to answer three questions using only the links: *What is the teleop latency? Why four layers? What happens if the leader cable is pulled?* If any takes more than two clicks, fix the index.

Done when: the three-question navigation test passes.

S2 · Build the review deck (2 h)

Twenty minutes, roughly 15–18 slides. Structure it the way a review board expects, which is not the way you built it:

1. **Problem and requirements** (2 slides) — what the SO-ARM101 ships with, what you set out to change, the four headline requirements with their margins.
2. **Architecture** (2) — the block diagram, and the PS-versus-PL partitioning decision with the measurement that decided it.
3. **Design** (4) — the bus-master IP with its timing diagram; the shield with the safety chain sheet; the hardware teleop loop; the HLS filter with its resource numbers.
4. **Verification** (3) — the cocotb suite and coverage; the ILA-versus-simulation-versus-scope comparison; the bug that hardware found and the regression test that now catches it.
5. **Results** (3) — the three-path latency table including the under-PS-load column; the loop jitter; the characterisation summary with encoder versus optical.
6. **Safety** (2) — the layered diagram, the FMEA top five, the fault-injection table with measured times, and the over-voltage capture.
7. **Reflection** (2) — the rev-B list, lessons learned, what you would do with six more weeks.

Two rules: **every claim carries its number**, and **every number names where it came from**. A slide that says "low latency" is a slide a reviewer will stop you on. A slide that says "hardware loop: 1.8 ms median, versus 14 ms for the USB path, scope-measured on both bus lines, n=20" is a slide that ends the question.

Done when: every results slide cites its measurement source.

S3 · Record the talk (2 h)

1. Record a full twenty-minute run-through, screen plus voice. Do not stop to fix mistakes on the first take.
2. Watch it back with a timer and a notepad. Look for: sections that ran long, claims you made without the number, places you said "basically" or "kind of", and any slide where you had to apologise for the content.
3. Cut ruthlessly and record a second take. Two takes is the right number — a third is polish, and polish is not what this artifact is for.
4. Upload it, link it from the README.

Done when: a clean twenty-minute recording is linked from the top-level README.

S4 · STAR stories and the grilling (2 h)

1. Write ten STAR stories in `docs/star-stories.md` — Situation, Task, Action, Result, with the **Result carrying a number**. Cover at least:
 - A trade-off you made (PS versus PL, or INA226 versus XADC)
 - A bug you root-caused (the turnaround bug, or a bring-up anomaly)
 - A schedule risk you managed (fab lead time, and the W9/W10 swap plan)
 - A review you ran that found real problems
 - A requirement you pushed back on or changed after measuring
 - A measurement that changed a decision
 - A safety failure you moved from software to hardware (over-voltage is the obvious one — the servos protect themselves in firmware, and you explain why that was not enough)
 - A vendor claim you tested independently (the zero-backlash and 0.3° claims)
 - A negative result you published anyway (the video path, if you did the stretch)
 - Something you got wrong and fixed
2. **Get grilled.** Hand a colleague `docs/interview-bank.md` and give them thirty minutes to attack. Brief them properly: their job is to find the answer you do not have, not to be encouraging.
3. Record which questions you answered with a number, which with a document, and which with a shrug. The shrugs are your remaining work. Note them in the lessons-learned file rather than quietly forgetting them.
4. Update the CV line. It should name the artifact and a number: custom Zynq overlay and PCB driving a robot arm pair, hardware control loop at 250 Hz with measured latency $N\times$ better than the vendor path, full design package published.

Done when: the shrug list is written down.

S5 · Saturday: the LeRobot stretch, or consolidation (4 h)

Take the stretch if the package is complete. If anything above is unfinished, finish it first — a complete package beats an incomplete package with a policy demo bolted on.

1. **Record a dataset with your hardware teleop loop**, the kit's wrist and external cameras both streaming MJPEG through the powered hub (the external camera at 640×480 on the board, per W4), using the LeRobot recording tools pointed at your overlay. Record enough episodes of one simple task to train something — a pick-and-place is the usual choice.
2. **Train an ACT policy** on whichever machine has the NVIDIA GPU — most likely the Windows desktop, where LeRobot training runs fine. Copy the dataset across and pin the same LeRobot version on both machines. This is mostly waiting; use it to write S5's note.
3. **Run inference on the laptop with your overlay as the actuator backend** over the network. The PS is back in the loop here by necessity, which is exactly why W10 measured the waypoint-to-bus-byte latency — you know what that costs, and you can state it.

4. **Write the note that makes this interesting**, `docs/policy-note.md` : what did the deterministic loop change about the recorded data? Hypotheses worth testing against your own recordings:
- Timestamp jitter in the dataset should be much lower, since the actions were applied on a hardware schedule rather than a Python one.
 - Action-to-observation alignment should be tighter and more consistent.
 - The follower trajectory should be smoother, because the HLS rate limiter shaped it. Check each against the data rather than asserting it. A short, measured note about dataset quality is a more distinctive artifact than a policy that works, because everyone has a policy that works.
5. Finally, **archive the whole project**: tag `v1.0` , confirm every raw data file is committed, confirm the README's headline table matches the reports, and write the final Sunday log entry.

Done when: the repo is tagged and the README's headline numbers match the underlying reports exactly.

Deliverables

-  Public repo, navigable, headline results on the first screen
-  Review deck, every claim numbered and sourced
-  Twenty-minute recorded talk, linked
-  `docs/star-stories.md` — ten stories, each with a numeric result
-  `docs/lessons-learned.md` including the shrug list from the grilling
-  Updated CV line
-  Stretch: `docs/policy-note.md` with measured claims about dataset quality

Failure modes

Symptom	Likely cause	Fix
Deck runs to 35 minutes	Built in build order, not review order	Cut design detail; reviewers want problem, results, safety
A reviewer stops you on slide 3	Unquantified claim	Every claim carries its number
Cannot answer an interview-bank question	Genuine gap	Write it in lessons-learned; do not pretend
README's numbers differ from the reports	Copied by hand at different times	Generate the table, or check it as the last act
Stretch consumed the week, package unfinished	Priorities inverted	Package first, always

Gate 4. You can walk a stranger through the design package in 20 minutes, answer every question in the interview bank with a number or a document, and both arms run safely from PL on your own board.

Competency matrix

What a senior hardware role is usually screened for, and where each week exercises it. Use it to check for gaps as you go.

Competency	1	2	3	4	5	6	7	8	9	10	11	12	13
Requirements & architecture	.	.	.	•	.	.	.	.	.	•	.	•	•
Lab measurement & instrumentation	•	•	•	.	.	•	.	.	•	•	•	•	.
Interface specification	.	.	•	•	.	•	.	.	.	.	.	.	.
RTL design	•	.	.	.	•	•	.	.	.	•	.	•	.
Verification & coverage	.	.	.	.	•	•	.	.	.	.	•	•	.
FPGA integration & timing closure	•	.	.	.	.	•	.	.	.	•	.	.	.
Schematic & part selection	.	.	.	•	.	.	•	.	•	.	.	.	.
PCB layout, SI & DFM	.	.	.	.	.	.	.	•	•	.	.	.	.
Power design & derating	.	•	.	•	.	.	•	.	•	.	.	.	.
Bring-up & root-cause	.	•	.	.	.	•	.	.	•	.	.	•	.
Real-time & performance analysis	.	.	•	•	.	•	.	.	.	•	.	.	.
Vision & high-bandwidth data paths	•	.	.	•	.	.	.	.	.	•	•	.	•
Test automation & DVT	.	.	•	.	•	.	.	.	.	.	•	•	.
Safety & FMEA	.	.	.	•	.	.	•	.	.	.	.	•	.
Release, ECO & schedule discipline	.	.	.	•	.	.	.	•	•	.	.	.	•
Reviews, documentation & communication	•	•	•	•	.	.	•	.	•	.	•	.	•

Kit and tools

Item	Used from	Notes
Hiwonder SO-ARM101 follower (6 × HX-30HM)	W2	In hand. 12 V; servos rated 9–12.6 V, 3 A stall each.
Hiwonder SO-ARM101 leader (6 × HX-10HM)	W2	In hand. Also 12 V; different gear ratio for back-driving.
Kit cameras: 480p wrist, 1080p external	W1	In hand (Standard/Advanced kits). Dataset cameras in W2 and W13.
Spare OBSBOT camera, tripod, printed ArUco marker	W2	In hand. Becomes the independent optical instrument in W11.
BusLinker V3.0 boards (one per arm)	W2	CH341 USB-serial; reference path for the W3 latency baseline; TTL header is a W6 fallback only.
Kit 12 V 5 A adapter	W0	Demos only; every test runs from the current-limited bench supply.
Spare HX-30HM and HX-10HM	W0	For stall, over-voltage and protection tests — never test destructively on the arms.
ServoStudio and CH341 driver	W0	Servo inventory, config, protection settings, firmware versions.
PYNQ-Z2, 32 GB microSD, Ethernet, powered USB hub	W1	PYNQ v3.1.1 image (2024.1 toolchain baseline). The hub is for the two cameras on the single USB 2.0 host port.
Bench supply, 12 V at ≥ 8 A with current limit, adjustable to ~ 14 V	W2	Every servo test runs through the limit; the upper range is for over-voltage testing on dummy loads and spares only.
Oscilloscope, ≥ 100 MHz, two channels	W1	Bus edges, rail ripple, loop-tick jitter, teleop latency, fault reaction times. Use the spring ground, not the flying lead.
Logic analyser (sigrok-compatible or Saleae)	W3	UART decode at 1 Mbps needs ≥ 8 MS/s; two channels for the two buses.
Two 74LVC1G125 / SN74LVC1T45 breakouts	W6	Interim bus buffers on a breadboard until the shield arrives.
INA226 breakout, NTC, e-stop switch, TVS, XT30, eFuse or comparator parts	W7	Prototype sense, safety and over-voltage parts before committing them to layout.
Power resistors as dummy loads	W7	Over-voltage cutoff testing without risking servos.
Thermal camera or IR thermometer	W9	Optional but makes the bring-up report far stronger.
Calipers	W0	For the 5264 connector pitch and the marker size. Both matter more than they sound.
Dedicated Linux laptop (Ubuntu LTS), ≥ 16 GB RAM, ≥ 200 GB free SSD, Ethernet or USB-Ethernet	W0	Primary host for everything except ServoStudio.
Windows desktop	W2	ServoStudio; W13 training if it has the GPU; fallback host (needs its own Vivado license).
Vivado + Vitis 2026.1 (Basic tier), KiCad 8+, cocotb, Verilator or Icarus, pytest, pyvisa + pyvisa-py, OpenCV	W0	Basic is free, covers the 7Z020, runs on Linux; node-locked, renewed yearly; one ILA core, five probes.

Reading list, paced to the weeks

Zynq and PYNQ

- *The Zynq Book* (free PDF), chapters on PS/PL and AXI — W1.
- PYNQ v3.1 changelog and release notes, so you know what differs from older tutorials — W1.
- AMD UG585 Zynq-7000 TRM, interconnect and clocking chapters — W1, W6.
- PYNQ docs: overlay design methodology, default IP, DMA — W1, W6, W10.
- UG903 Vivado constraints, UG906 timing analysis — W6.
- UG1399 Vitis HLS user guide, ap_fixed and pipelining — W10.
- Vitis Vision library overview and an AXI-stream video example, only if you take the W10 stretch — W10.

Arms, servos, cameras

- Hiwonder SO-ARM101 LeRobot tutorials, and the upstream LeRobot SO-101 docs for context (they describe the Feetech build, not yours) — W2.
- Hiwonder BusLinker V3.0 user manual and ServoStudio guide — W2.
- Hiwonder HX-30HM and HX-10HM product documents — W2, W4.
- Hiwonder *Magnetic Encoder Bus Servo Communication Protocol* — W3. Your primary source for every register and packet format.
- A TI or Analog Devices application note on eFuses and over-voltage protection — W7.
- AMD UG973 (release notes, installation and licensing) for 2026.1, and the Vivado licensing FAQ for what the Basic tier includes — W0.
- cocotb documentation and the UART example — W5.
- OpenCV camera calibration and ArUco detection tutorials, current API — W11.
- LeRobot dataset recording and ACT training guide — W13.

Senior craft

- Horowitz & Hill, *The Art of Electronics*, power and interfacing chapters — W7.
- Johnson & Graham, *High-Speed Digital Design*, chapters 1–6 — W8.
- Ott, *Electromagnetic Compatibility Engineering*, grounding chapters — W8.
- Agans, *Debugging: The 9 Indispensable Rules* — W9.
- An FMEA primer such as the AIAG/VDA handbook overview — W12.

Interview bank

Questions a senior panel would ask about this exact project. Each should be answerable with a number or a document from the repo by W13. The week that produces the answer is noted.

1. Walk me through the power budget. How did you size the fuse and bulk capacitance, and what derating did you apply? Why two switched rails from one input rather than one rail for both arms? — W4, W7
2. Why four layers? Where is the return current for the 1 Mbps bus lines, and what would change at 10 Mbps? — W8 S3
3. How did you verify the bus-master RTL before it touched hardware? What is your coverage, and what did it miss that the ILA found? — W5, W6 S4
4. Are there any clock-domain crossings in the design? How are they handled, and how would you prove they are safe? — W5 S2, S4; W10 S2
5. Timing closure: what did you do the first time WNS went negative? — W6 S2
6. How do you decide what runs in PL versus the PS? Give the measurement that decided it for the teleop loop, and the one that decided it for the cameras. — W10 S5, W4 S2
7. What is your teleop latency limited by, and what would it take to halve it? — W10 S6

8. Your encoder repeatability and your optical repeatability disagree. Which do you believe, and why? — *W11 S5*
9. Describe the FMEA. Which failure did you move from software to hardware and why? — *W12 S1*
10. What is the first thing you do when a new board lands on your bench? What is the last? — *W9*
11. What changed between rev A and rev B, and how did each ECO get found? — *W8 S5, W9 S5*
12. How would you take this shield to production? Test points, ICT, calibration, traceability, and how you stop an over-voltage supply reaching twelve 12.6 V servos. — *W7, W12*
13. You built on a Vivado release newer than the PYNQ image. What can break, and how did you prove it did not? — *W1 S4*
14. How do you run a design review so that it finds problems rather than approves slides? — *W4 S5, W7 S5*
15. Tell me about a measurement that changed a decision on this project. — *W13 S4*
16. What would you cut if you had six weeks instead of thirteen, and what would you never cut? — *W13*
17. The servos are rated to 12.6 V and already have their own over-voltage protection. Why did you build an active cutoff, and what peak voltage does a servo actually see on a 12→14 V step? — *W4, W7, W9, W12*
18. Your servos are not the ones the open-source project documents. How did you establish the protocol, and where did your captures disagree with the vendor's document? — *W3*

Risks and how to slip gracefully

Risk	Trigger	Response
Vivado Basic license lapses or stops matching the laptop	Renewal date approaching; network adapter changed	Renew a month early; regenerate the node-locked license if the host ID changes. Without a valid license you cannot open the designs.
Laptop fails mid-project	Any hardware fault	Everything is in git; the Windows desktop is the fallback host, with the rebuild steps in <code>docs/toolchain.md</code> .
Gate 0 finds real gaps	Six or fewer Gate 0 passes	Take one extra ramp-up week from the holiday buffer. Do not start W1 unready.
Ramp-up runs slow	Any R week not closed by Sunday	Finish it the following Monday, and compress W0 installs; do not skip the cocotb or AXI-Lite sessions — they are what W5 and W6 stand on.
PCB fab and assembly take longer than the holiday buffer	Boards not in hand by Mon 4 Jan	Swap W9 and W10. The hardware teleop loop runs fine on the breadboard buffers; bring-up waits for the board.
Rev-B needed, and Chinese New Year (6 Feb 2027) closes fabs	Rev-B list has safety items	Decide in W9 S5 and order by the third week of January, or accept a bodged rev-A for the portfolio.
Vivado 2026.1 and the 2024.1-era PYNQ image disagree	A 2026.1 bitstream fails to load in W1	Custom overlays are unaffected; only a base-overlay rebuild needs IP upgrades. Fall back to 2024.1 for the plan and record the reason as a decision.
Two cameras saturate the single USB 2.0 host port	Dropped frames at W4's target	Drop to MJPEG at 640×480, or run the OBSBOT on the laptop for W11 and W13; only the wrist camera needs to be on the board. Write the bandwidth arithmetic down either way.
Over-voltage supply reaches the servos	Any near miss at all	Label every supply and adapter in W0, build the active cutoff in W7, verify its peak in W9, and keep the rail check in every procedure. Treat a near miss as a finding, not as a lucky escape.
HX protocol differs from the lineage defaults more than expected	W3 frame table has corrections	Constants and <code>frame()</code> absorb header and checksum changes; a different packet structure costs W5 time — borrow from W7's slack, and do not skip the spec.
Hiwonder's LeRobot support depends on a vendor fork that drifts	W2 finds a fork, or W13 finds it broken	Pin the version in <code>requirements.txt</code> in W2; your own <code>sw/hx_min</code> is the fallback actuator backend for W13.
Leader and follower cables swapped	Any occurrence	Silkscreen and cable flags in W7; the driver's model check refuses torque.
RTL takes longer than one week	W5 S5 not green by Saturday	Borrow from W7 by prototyping the shield with breakout boards. Do not skip the cocotb suite; it is the senior-level part of W5.
Bench time squeezed by work	Two consecutive weeks under 8 h	Protect W4, W7 and W13. They are mostly writing and can be done anywhere, and they carry the most interview weight.
A servo is damaged in stall testing	Any servo running hot to the touch	Keep stall tests under two seconds behind the supply current limit, do destructive and protection tests on the spare HX-30HM and HX-10HM, not the arms. Stop for the evening.
Bring-up finds a blocking design fault	Rev-A unusable for a required measurement	Fall back to the breadboard path for W10 and W11; W12's hardware trips are the only work genuinely blocked. Re-order

Risk	Trigger	Response
		rather than wait.

Weekly ritual regardless of slips: 30 minutes every Sunday to log what shipped, update the risk register, and adjust the next week. Start it in R1 — the ramp-up weeks are where the habit forms. That log becomes the lessons-learned section in W13.

Appendix A · Standing procedures

These are referenced by every week. Write them once, follow them every time.

A1 · Rail check

Before any arm is connected to any supply or board:

1. Say it out loud: **twelve volts, and the display agrees.**
2. Read the voltage on the supply's display, not the setting you remember.
3. Confirm nothing above 12.6 V is plugged into, or within reach of, anything that feeds the arms.
4. Check the LEADER / FOLLOWER cable flag against the port or BusLinker it is going into.
5. From W9 onwards, confirm the shield's over-voltage cutoff has not been bypassed or bodged.

A near miss on this procedure is an entry in `docs/eco.md`, not a shrug.

A2 · Power-on sequence

1. Current limit set **first**, output off.
2. Voltage set second.
3. Verify both on the display.
4. Output on, and watch the current reading for the first two seconds. Inrush that hits the limit means stop, not retry.

A3 · Scope hygiene

1. Compensate the probe against the calibration output at the start of any session where a number matters.
2. Use the **spring ground** for anything above about 1 MHz, never the flying lead.
3. For ripple: AC coupling, 20 MHz bandwidth limit.
4. Save the waveform file, not only a screenshot. A screenshot cannot be re-measured.
5. Name the file after the measurement and the date, and commit it.

A4 · Energised-work rules

1. Arms clear of everything within their reach.
2. E-stop within arm's reach of you, tested that session.
3. Hand near the supply output button for any first-time motion.
4. First motion of any new configuration is small — 100 counts, one joint.
5. Nobody else's hands anywhere near the follower.

A5 · Anomaly handling

1. Reproduce deliberately before investigating.
2. Classify: design / assembly / firmware.
3. Root-cause the mechanism, not the symptom.

4. Disposition: rev-B / bodge / software workaround / no action with a reason.
5. Log in `docs/eco.md` , including unresolved ones with the next experiment noted.
6. Timebox to 45 minutes before logging and moving on.

Appendix B · The measurement ledger

Every number this project depends on, where it comes from, and what consumes it. Keep this table updated as you go; it is the fastest way to spot a requirement with nothing behind it.

#	Measurement	Taken in	Consumed by
M01	Board current: PL empty / base / custom	W1 S5	Power budget
M02	FCLK requested vs actual, rise time at pin	W1 S5	Clocking understanding, W8 SI baseline
M03	Camera fps, format, CPU load	W1 S5	W4 camera budget
M04	Follower current: quiescent / idle / gravity / move / stall	W2 S3	W4 power budget, W7 fuse, load switch, shunt
M05	Current step duration Δt	W2 S3	W7 bulk capacitance
M06	Leader current: idle / back-driven	W2 S4	W7 leader rail switch and shunt
M07	Bus bit period, rise/fall, ringing, far end, both buses	W2 S5	W5 baud gen, W7 series resistors
M08	Master-to-servo turnaround and response delay	W2 S5, W3 S1	W5 direction timing, W4 bus budget
M09	Backlash, sag, back-drive per joint	W2 S4	W11 baseline
M10	USB read round-trip: median, p99, max	W3 S4	The project's justification
M11	Teleop latency, USB path, n=20	W3 S5	REQ-004 baseline
M12	Simulated turnaround, stop bit to DIR release	W5 S6	W6 ILA comparison
M13	Baud tolerance	W5 S2	README, interview
M14	Coverage: instruction and error path, with denominators	W5 S6	W13 review
M15	WNS, TNS, utilisation	W6 S2, W10 S4	Budget, interview Q5
M16	PL-path read round-trip, n=1000	W6 S5	Overlaid figure vs M10
M17	Teleop latency, PS in loop, n=20	W6 S5	Three-path table
M18	Measured turnaround on hardware	W6 S4	vs M12
M19	Watchdog drop-out time (breadboard)	W7 S6	REQ-021
M20	INA226 reading vs supply readout	W7 S6	W11 current tests
M21	Every test point, measured vs expected	W9 S1–S3	Bring-up report
M22	Rail ripple, static and under motion	W9 S1, S5	REQ-012
M23	E-stop and watchdog drop-out, on the board, both rails	W9 S2	REQ-020, REQ-021
M24	Bus edges: breadboard vs far end vs shield	W9 S3	The PCB's payoff figure
M25	Component temperatures and rise over ambient	W9 S5	W7 deratings
M26	NTC vs IR at the same spot	W9 S5	W12 over-temp trip
M27	Loop period mean and jitter, $n \geq 10\,000$	W10 S5	REQ-003
M28	Teleop latency, hardware loop, n=20	W10 S5	REQ-004, three-path table
M29	All three paths under heavy PS load	W10 S5	The central claim
M30	Waypoint to first bus byte, worst case	W10 S5	W13 policy inference

#	Measurement	Taken in	Consumed by
M31	HLS latency and resources	W10 S3	performance.md
M32	Optical noise floor, reprojection RMS	W11 S2	Instrument qualification
M33	Repeatability $\sigma/3\sigma$, encoder and optical, per joint	W11 S3	Characterisation report
M34	Backlash per joint, both instruments	W11 S3	vs M09
M35	Step response $\times$ 3 sizes, per joint	W11 S4	Characterisation report
M36	Current vs holding torque	W11 S4	vs M04
M37	Temperature rise over a stated duty cycle	W11 S4	W7 deratings
M38	Bus error rate per 10^6 transactions, both buses	W11 S4	Bus health
M39	Tracking RMS/max error and effective lag	W11 S5	vs M28
M40	Reaction time for each injected fault	W12 S4, S5	REQ-020, 021, 022
M41	Factory config and protection thresholds, all twelve servos	W2 S1	W3 cross-check, W12
M42	Which protection ended each stall, and its time	W2 S3	W12
M43	Servo bus idle level	W2 S5	W6 and W7 signal-level decision
M44	Over-voltage trip, release, response and rail peak	W7 S6, W9 S1, W12 S5	REQ-011
M45	Servo Accel / Speed profile per joint	W11 S4	Context for every dynamic result
M46	HLS filter vs servo-profile experiment	W10 S5	Decision 0013

If a requirement in docs/requirements.md has no ledger entry pointing at it by W13, it was never verified. Find it before a reviewer does.

Appendix C · Weekly cadence template

Copy this into the Sunday log each week.

```
## W<nn> · <dates>

**Hours actually spent:** <n> of <budget>
**Shipped:** <files, tagged>
**Measurements added to the ledger:** <M-numbers>
**Open from this week:** <items, with next action>
**Risk register changes:** <added / retired / triggered>
**Next week's first session starts with:** <one concrete step>
**What surprised me:**
```

The last two lines matter most. The first-session prompt removes the cost of restarting; the surprise line is where lessons-learned comes from in W13.

Plan drafted 10 September 2026; expanded to step-by-step detail; revised 21 September 2026 for the Hiwonder kit and a five-week Python and Verilog ramp-up, running 21 September 2026 to 7 February 2027. Toolchain: PYNQ v3.1.1 (2024.1 baseline) with Vivado 2026.1. Written for the Hiwonder SO-ARM101 kit (HX-30HM follower, HX-10HM leader, BusLinker V3.0). Register addresses and packet formats come only from Hiwonder's protocol document, confirmed on the wire in W3 against your servo firmware versions; this plan deliberately names none. Pin assignments are always taken from the PYNQ-Z2 master XDC, never from this document.
